

2026

Ricardo

Aqui son las dedicatorias pero hay que ponerlas hasta el final

Roman

Estas notas fueron elaboradas con fines académicos.

A pesar del cuidado puesto en su redacción,
es posible que existan errores u omisiones.

Cualquier corrección, sugerencia o comentario será bienvenido en los correos correo:

ricardo.leon@cimat.mx

carlos.roman@cimat.mx

Índice general

1	Lógica Proposicional	3
1.1	¿Qué es la lógica proposicional?	3
1.2	Validez, satisfacibilidad y contradicción	8
1.3	Consecuencia y equivalencia	11
1.4	Demostraciones formales	13
1.5	Demostración por inducción	24
1.5.1	Inducción matemática	25
1.5.2	Inducción sobre la complejidad de las fórmulas	27
1.6	Formas normales	29
1.7	Fórmulas de Horn	34
1.8	Resolución	38
1.8.1	Cláusulas	39
1.8.2	Resolventes	40
1.8.3	Completitud de la resolución	42
1.9	Completitud y compacidad	46
2	Estructuras y lógica de primer orden	55
2.1	El lenguaje de la lógica de primer orden	55
2.2	La sintaxis de la lógica de primer orden	56
2.3	Semántica y estructuras	59
2.4	Ejemplos de estructuras	67
2.4.1	Grafos	67
2.4.2	Bases de datos relacionales	70
2.4.3	Órdenes lineales	72
2.4.4	Sistemas numéricos	73
2.5	El tamaño de una estructura	74
2.6	Relaciones entre estructuras	81
2.6.1	Encajes	81

2.6.2	Subestructuras	85
2.6.3	Diagramas	89
2.7	Teorías y modelos	91

1 Lógica Proposicional

1.1 ¿Qué es la lógica proposicional?

En lógica proposicional, las fórmulas atómicas son proposiciones. Cualquier afirmación vale. Por ejemplo

A="Aristóteles ha muerto,"

B="Barcelona está a orillas del Sena," y

C="Courtney Love es alta"

son fórmulas atómicas. Las fórmulas atómicas son los elementos básicos que se utilizan para construir proposiciones. En cualquier lógica, una proposición se considera un tipo concreto de fórmula. En la lógica proposicional, no existe distinción entre estos dos términos. Utilizamos "fórmula" y "proposición" indistintamente.

En la lógica proposicional, al igual que en todas las lógicas que estudiamos, cada proposición es verdadera o falsa. A la proposición se le asigna, en consecuencia, un *valor de verdad* de 1 o 0. En el ejemplo anterior, podemos asignar el valor de verdad 1 a la fórmula A y el valor de verdad 0 a la fórmula B. Si tomamos la proposición C al pie de la letra, su veracidad es discutible. Quizá, tendría más sentido permitir valores de verdad entre 0 y 1. Podríamos asignar un 0,75 a la afirmación C si la señorita Love es más alta que el 75 % de las mujeres estadounidenses. La lógica difusa permite tales valores de verdad, pero las lógicas clásicas que estudiamos no.

De hecho, el contenido de las proposiciones no es relevante para la lógica proposicional. De ahora en adelante, las fórmulas atómicas se denotarán únicamente mediante las letras mayúsculas A, B, C, ... (posiblemente con subíndices) sin hacer referencia a lo que estas proposiciones realmente dicen. La veracidad de estas fórmulas no nos concierne. La lógica proposicional no es el estudio de la verdad, sino de la relación entre la verdad de una afirmación y la de otra.

El lenguaje de la lógica proposicional contiene palabras como "no", "y", "o", "implica" y "si y solo si". Estas palabras se representan mediante símbolos:

$\neg$ para "no", $\wedge$ para "y", $\vee$ para "o",
 $\rightarrow$ para "implica", $\leftrightarrow$ para "si y solo si".

Como ocurre siempre que se traduce de un idioma a otro, esta correspondencia no es exacta. A diferencia de sus equivalentes en inglés, estos símbolos representan conceptos precisos e invariables. El significado de una palabra en inglés, por el contrario, siempre depende del contexto. Por ejemplo, $\wedge$ representa un concepto similar, pero no idéntico, a "y". Para las fórmulas atómicas A, B y $A \wedge B$ siempre significa lo mismo que $B \wedge A$. Esto no siempre es así con la palabra "y". La oración

Se puso muy enferma y fue al médico.

no tiene el mismo significado que

Fue al médico y se puso muy mal.

Del mismo modo, $\vee$ difiere de "o". En el lenguaje coloquial, el uso de "A o B" suele excluir la posibilidad de que se den tanto A como B. En lógica proposicional, $A \vee B$ siempre significa o bien A, o bien B, o bien tanto A como B.

Debemos definir con precisión los símbolos $\neg, \wedge, \vee, \rightarrow$ y $\leftrightarrow$. Nos enfrentamos al dilema de cómo definir la primera de un lenguaje (¡sin recurrir a ninguna otra palabra!). Por este motivo, tomamos los símbolos $\neg$ y $\wedge$ como *primitivos*. Definimos los demás símbolos en función de estos dos. Aunque no definimos $\neg$ y $\wedge$ en función de los demás símbolos, sí describimos la semántica de estos símbolos de forma inequívoca.

Antes de describir la semántica del lenguaje, abordaremos la sintaxis. Mientras que la semántica se ocupa del significado, o interpretación, de las oraciones en el lenguaje, la sintaxis se ocupa de la gramática del lenguaje. La sintaxis de la lógica proposicional nos indica qué cadenas de símbolos son admisibles como fórmulas. Naturalmente, cualquier fórmula atómica es una fórmula. Además, contamos con las dos reglas siguientes.

- (R1) Si F es una fórmula, entonces $\neg F$ es una fórmula.
- (R2) Si F y G son fórmulas, entonces $(F \wedge G)$ es una fórmula.

Definición 1.1: Negación y conjunción

La fórmula $\neg F$ es la negación de F y la fórmula $(F \wedge G)$ es la conjunción de F y G .

Definición 1.2

Una cadena finita de símbolos es una fórmula de la lógica proposicional si y solo si se construye a partir de fórmulas atómicas mediante la aplicación de las reglas (R1) y (R2).

Ejemplo 1.1

$\neg(\neg(A \wedge B) \wedge \neg C)$ es una fórmula, mientras que $((A \neg \wedge)B(C \neg$ no lo es. ▲

Obsérvese que hemos restringido la definición de fórmula a los símbolos primitivos $\neg$ y $\wedge$. Si tuviéramos que escribir la sintaxis de la lógica proposicional a un ordenador, esta definición de fórmula sería suficiente. Sin embargo, para que las fórmulas resulten más comprensibles para los humanos, incluimos los demás símbolos ($\vee, \rightarrow$, y $\leftrightarrow$) que se definirán más adelante. Podemos considerar las fórmulas que incluyen estos símbolos como abreviaturas de fórmulas más complicadas que solo contienen $\neg$ y $\wedge$. La inclusión de estos símbolos hace que las fórmulas sean más fáciles de leer (para nosotros, los humanos).

Asimismo, con el fin de facilitar la lectura, utilizamos ciertas convenciones. El uso de estas abreviatur-

ras y convenciones modifica en cierta medida nuestra noción de fórmula. Una de estas convenciones es la siguiente:

(C1) Si F o (F) es una fórmula, entonces consideremos que F y (F) son la misma fórmula.

Es decir, podemos omitir los paréntesis más externos. Esto amplía nuestra definición de fórmula. Técnicamente, según la definición anterior, $A \wedge B$ no es una fórmula. Sin embargo siguiendo la convención (C1), no distinguimos entre $A \wedge B$ y la fórmula $(A \wedge B)$.

El uso de la convención (C1) da lugar a algunas ambigüedades que abordaremos a continuación. Supongamos que, en (R1), F denota $A \wedge B$ (lo cual, según (C1), es una fórmula). Entonces, $\neg F$ no representa la fórmula $\neg A \wedge B$. Más bien, $\neg F$ denota la fórmula $\neg(A \wedge B)$. Como veremos, $\neg A \wedge B$ y $\neg(A \wedge B)$ no significan lo mismo. Del mismo modo, $F \wedge G$ denota la fórmula $(F) \wedge (G)$.

El uso de (C1) también requiere precaución a la hora de definir el concepto de "subfórmula". Una subfórmula de una fórmula F (considerada como una cadena de símbolos) es una subcadena de F que es a su vez una fórmula. Sin embargo, debido a (C1), no todas esas subcadenas son subfórmulas. Por lo tanto, no queremos tomar esta propiedad como la definición de "subfórmula". En su lugar, definimos "subfórmula" de la siguiente manera.

Definición 1.3: Subfórmula

Las siguientes reglas definen las subfórmulas de una fórmula.

Cualquier fórmula es una subfórmula de si misma.

Cualquier subfórmula de F es también una subfórmula de $\neg F$.

Cualquier subfórmula de F o G es también una subfórmula de $(F \wedge G)$.

Ejemplo 1.2

Sean A y B fórmula atómicas y sea F la fórmula $\neg(\neg A \wedge \neg B)$.

La fórmula $A \wedge \neg B$ aparece como una subcadena de F , pero no es una subfórmula de F . No hay forma de construir la fórmula F a partir de la fórmula $A \wedge \neg B$. Las subfórmulas de F son A , B , $\neg A$, $\neg B$, $(\neg A \wedge \neg B)$ y $\neg(\neg A \wedge \neg B)$. ▲

Una vez descrita la sintaxis de la lógica proposicional, pasamos ahora a describir la semántica. Es decir, explicamos cómo interpretar las fórmulas. No solo debemos describir la semántica de los símbolos " $\wedge$ " y " $\neg$ ", sino que también debemos explicar como interpretar las fórmulas en las que estos símbolos aparecen juntos. Para ello, establecemos el *orden de las operaciones*. La función de los paréntesis es determinar qué subfórmulas deben tenerse en cuenta en primer lugar al interpretar una fórmula. Si no hay paréntesis, entonces utilizamos la siguiente regla:

$\neg$ tiene prioridad sobre $\wedge$.

Por ejemplo, la fórmula $\neg(A \wedge B)$ significa "no tanto A como B ". Los paréntesis nos indican que el símbolo " $\wedge$ " en esta fórmula tiene prioridad sobre " $\neg$ ". La fórmula $\neg A \wedge B$, por otro lado, tiene una interpretación diferente. A falta de paréntesis, aplicamos la regla de que $\neg$ tiene prioridad sobre $\wedge$.

Por lo tanto, esta fórmula significa "tanto no A como B ".

La semántica de la lógica proposicional viene definida por esta regla, junto con las Tablas 1.1 y 1.2. Estos son ejemplos de *tablas de verdad*. Cada fila de estas tablas asigna valores de verdad a fórmulas atómicas y proporciona los valores de verdad resultantes para fórmulas más complejas. Por ejemplo, la tercera fila de la Tabla 1.1 nos indica que, si A es verdadera y B es falsa, entonces $(A \wedge B)$ es falsa. Vemos que $(A \wedge B)$ tiene el valor de verdad 1 solo si tanto A como B tienen el valor de verdad 1, lo que se corresponde con nuestra noción de "y". Del mismo modo, la Tabla 1.2 nos indica que $\neg A$ tiene el valor de verdad opuesto al de A , lo que corresponde con nuestra noción de negación.

Utilizando estas dos tablas de verdad, podemos hallar tablas de verdad para cualquier fórmula. Esto se debe a que toda fórmula se construye a partir de fórmulas atómicas mediante las reglas (R1) y (R2). Supongamos, por ejemplo, que queremos hallar una tabla de verdad para la fórmula $\neg(\neg A \wedge \neg B)$. Dado los valores de verdad de A y B , podemos utilizar la Tabla 1.2 para hallar los valores de verdad de $\neg A$ y $\neg B$. Dados los valores de verdad de $\neg A$ y $\neg B$, podemos utilizar la Tabla 1.1 para hallar el valor de verdad de $(\neg A \wedge \neg B)$. Por último, podemos volver a consultar la Tabla 1.2 para hallar el valor de verdad de $\neg(\neg A \wedge \neg B)$. La tabla de verdad resultante para esta fórmula se muestra en la Tabla 1.3.

Obsérvese que las fórmulas que aparecen en la parte superior de la Tabla 1.3 son precisamente las subfórmulas del Ejemplo 1.2. En esta tabla vemos que la fórmula $\neg(\neg A \wedge \neg B)$ tiene valor de verdad 1 si y solo si A o B tiene valor de verdad 1. Esta fórmula se corresponde con la noción de "o" que se ha comentado anteriormente. El símbolo $\vee$ se utiliza para denotar esta útil noción.

Tabla 1.1 Tabla de verdad de $A \wedge B$

A	B	$(A \wedge B)$
0	0	0
0	1	0
1	0	0
1	1	1

Tabla 1.2 Tabla de verdad de $\neg A$

A	$\neg A$
0	1
1	0

Tabla 1.3 Tabla de verdad de $A \vee B$

A	B	$\neg A$	$\neg B$	$(\neg A \wedge \neg B)$	$\neg(\neg A \wedge \neg B)$
0	0	1	1	1	0
0	1	1	0	0	1
1	0	0	1	0	1
1	1	0	0	0	1

Tabla 1.4 Tabla de verdad de $(A \rightarrow B)$

A	B	$\neg A$	$(B \vee \neg A)$	$(A \rightarrow B)$
0	0	1	1	1
0	1	1	1	1
1	0	0	0	0
1	1	0	1	1

Definición 1.4: Disyunción

El símbolo $\vee$ se define de la siguiente manera: para cualquier fórmula F y G , $(F \vee G)$ es una abreviatura de $\neg(\neg F \wedge \neg G)$. La fórmula $(F \vee G)$ se denomina disyunción de F y G .

Otras dos abreviaturas que resultan útiles son las siguientes.

Definición 1.5

Los símbolos $\rightarrow$ y $\leftrightarrow$ se definen de la siguiente manera:

$(F \rightarrow G)$ es la abreviatura de $(G \vee \neg F)$, y

$(F \leftrightarrow G)$ es la abreviatura de $((F \rightarrow G) \wedge (G \rightarrow F))$.

Anteriormente señalamos que el símbolo $\rightarrow$ se corresponde con la palabra "implica" y que el símbolo $\leftrightarrow$ se corresponde con la expresión "si y solo si". Una vez más, estas correspondencias no son más que recursos mnemotécnicos para la semántica de los símbolos. Por ejemplo, $(A \leftrightarrow B)$ es verdadero si A y B tienen los mismos valores de verdad y, en caso contrario, es falso. Así pues, $\leftrightarrow$ se comporta exactamente igual que la expresión "si y solo si". La relación entre $\rightarrow$ e "implica" es un poco más tenue. Consideremos la tabla de verdad para $(A \rightarrow B)$ (Tabla 1.4).

Veamos que $(A \rightarrow B)$ es verdadero a menos que A sea verdadero y B sea falso. En particular, $(A \rightarrow B)$ es verdadero siempre que A sea falso. Por lo tanto, en lógica, una proposición falsa implica cualquier cosa. Esto difiere del uso coloquial de la palabra "implica". No diríamos "Que Barcelona esté a orillas del Sena implica que Aristóteles esté muerto" o, lo que es aún más flagrante, "Que Barcelona esté a orillas del Sena implica que Barcelona no esté a orillas del Sena". Sin embargo, $(A \rightarrow B)$ y $(A \rightarrow \neg A)$ son enunciados verdaderos de la lógica proposicional siempre que A sea falso.

Una vez introducidos los nuevos símbolos, debemos determinar el orden de operación de dichos

símbolos. A la hora de evaluar el valor de verdad de una fórmula, debemos saber el orden en el que hay que proceder. En lugar de clasificar todos los símbolos en una jerarquía, establecemos una única regla:

$\neg$ tiene prioridad sobre $\wedge$, $\vee$, $\rightarrow$ y $\leftrightarrow$

Además, los paréntesis indican el orden en el que hay que proceder.

Ejemplo 1.3

Consideremos la fórmula $((\neg A \rightarrow B) \wedge C) \vee \neg(A \vee D)$. Llamemos esta fórmula F . Supongamos que sabemos que los valores de verdad de A , B , C y D son 1, 0, 1 y 0, respectivamente. Para calcular el valor de verdad de F , comenzamos con la subfórmula $\neg A$ (usando la Tabla 1.2), ya que $\neg$ tiene prioridad. A continuación, evaluamos la subfórmula $(\neg A \rightarrow B)$ (Tabla 1.4), que se encuentra en el conjunto de paréntesis más interno. A continuación calculamos los valores de verdad de $((\neg A \rightarrow B) \wedge C)$ y de $(A \vee D)$ (Tabla 1.1). Después, calculamos los valores de $\neg(A \vee D)$ (Tabla 1.2) y, por último, los de F (Tabla 1.3). Obtenemos los siguientes valores de verdad.

A	B	C	D	$\neg A$	$(\neg A \rightarrow B)$	$((\neg A \rightarrow B) \wedge C)$	$(A \vee D)$	$\neg(A \vee D)$	F
1	0	1	0	0	1	1	0	1	1

Esta es solo una fila de una tabla de verdad para esta fórmula. Podríamos elegir otros valores de verdad para A , B , C y D distintos de 1, 0, 1 y 0. Dado que hay dos valores posibles para cada una de estas cuatro fórmulas atómicas, hay $2^4 = 16$ formas de asignar valores de verdad a A , B , C y D . Por lo tanto, la tabla completa tiene 16 filas. ▲

La función de los paréntesis no es solo determinar el orden de las operaciones, sino también facilitar la lectura de las fórmulas. Con este fin, omitimos los paréntesis cuando no son necesarios. Ya hemos comentado la convención (C1) que nos permite omitir los paréntesis más externos de la fórmula (F). También utilizamos la siguiente convención.

(C2) Para cualquier fórmula F , G y H ,
consideramos que $F \wedge G \wedge H$ es la misma fórmula que $(F \wedge G) \wedge H$
y que $F \vee G \vee H$ es la misma fórmula que $(F \vee G) \vee H$.

Dado que las fórmulas $(F \wedge G) \wedge H$ y $F \wedge (G \wedge H)$ tienen las mismas tablas de verdad, no hay ambigüedad al omitir los paréntesis y escribir simplemente $F \wedge G \wedge H$. Por el contrario, $F \wedge G \vee H$ es ambigua y no está permitida como fórmula de la lógica proposicional. Las fórmulas $(F \wedge G) \vee H$ y $F \wedge (G \vee H)$ no tienen la mismas tablas de verdad.

1.2 Validez, satisfacibilidad y contradicción

Sea $S = \{A_1, \dots, A_n\}$ un conjunto de fórmulas atómicas. Sea $\mathcal{F}(S)$ el conjunto de todas las fórmulas que pueden construirse a partir de las fórmulas atómicas de S .

Definición 1.6: Asignación

Una *asignación* de $\mathcal{S}$ es una función

$$\mathcal{A} : \mathcal{S} \rightarrow \{0, 1\}.$$

Es decir, una asignación de $\mathcal{S}$ asigna valores de verdad a cada fórmula atómica de $\mathcal{S}$. Una asignación $\mathcal{A}$ de $\mathcal{S}$ se extiende naturalmente a todo $\mathcal{F}(\mathcal{S})$. Para cualquier fórmula F en $\mathcal{F}(\mathcal{S})$, una asignación $\mathcal{A}$ de $\mathcal{S}$ se corresponde con una fila única de la tabla de verdad de F . Definimos $\mathcal{A}(F)$ como el valor de verdad de F en esa fila. Una asignación $\mathcal{A}$ de $\mathcal{S}$ también se extiende a ciertas fórmulas que no están en $\mathcal{F}(\mathcal{S})$. Supongamos que F_0 es una fórmula que no está en $\mathcal{F}(\mathcal{S})$. Sea $\mathcal{S}_0$ el conjunto de subfórmulas atómicas de F_0 . Si cada extensión de $\mathcal{A}$ a $\mathcal{S} \cup \mathcal{S}_0$ tiene el mismo valor para F_0 , entonces definimos $\mathcal{A}(F_0)$ como este valor.

Ejemplo 1.4

Sean A y B fórmulas atómicas. Sea $\mathcal{A}$ la asignación de $\{A, B\}$ definida por $\mathcal{A}(A) = 1$ y $\mathcal{A}(B) = 0$. Entonces,

$$\begin{aligned}\mathcal{A}(A \wedge B) &= 0, \\ \mathcal{A}(A \vee B) &= 1, \\ \mathcal{A}(A \wedge (C \vee \neg C)) &= 1, \text{ y} \\ \mathcal{A}(B \vee (C \wedge \neg C)) &= 0.\end{aligned}$$

▲

La razón por la que $\mathcal{A}(A \wedge (C \vee \neg C)) = 1$ es que $\mathcal{A}(A) = 1$ y, independientemente del valor de verdad que asignemos a C , $(C \vee \neg C)$ tiene valor de verdad 1. Del mismo modo, $\mathcal{A}(B \vee (C \wedge \neg C)) = 0$ porque tanto B como $(C \wedge \neg C)$ tienen valor de verdad 0, independientemente del valor de verdad de C .

Sea $\mathcal{A}$ una asignación de $\mathcal{S}$ y sea F una fórmula. Si $\mathcal{A}(F) = 1$, entonces decimos que F se cumple bajo la asignación $\mathcal{A}$. De forma equivalente, decimos que A modela a F . Escribimos $A \models F$ para denotar este concepto.

Definición 1.7: Validez y tautología

Una fórmula es *válida* si se cumple para cualquier asignación. Utilizamos $\models F$ para indicarlo. Una fórmula válida se denomina *tautología*.

Ejemplo 1.5

La fórmula $(C \vee \neg C)$ del ejemplo anterior es una tautología.

▲

Definición 1.8: Satisfacibilidad

Una fórmula es satisfacible si se cumple bajo alguna condición.

Definición 1.9

Una fórmula es insatisfacible si no se cumple con ninguna asignación. Una fórmula insatisfacible se denomina contradicción.

Ejemplo 1.6

La fórmula $(C \wedge \neg C)$ es una contradicción. ▲

Supongamos que queremos determinar si una fórmula dada es válida o no. Este es un ejemplo *problema de decisión*. Un problema de decisión es cualquier problema que, dada una entrada determinada, plantee una pregunta que debe responderse con un "sí" o un "no". Dada una fórmula F como entrada, podemos preguntarnos "¿Es F satisfacible?", y nos referimos a esto como el *problema de satisfacibilidad*. En lógica proposicional, las tablas de verdad proporcionan un enfoque sistemático para resolver este tipo de problemas de decisión. Si todos los valores de verdad de F son 1, entonces F es válida. Si algún valor de verdad es 1, entonces F es satisfacible. De lo contrario, si ningún valor de verdad es 1, F es insatisfacible.

Ejemplo 1.7

Consideremos la fórmula $(A \wedge (A \rightarrow B)) \rightarrow B$. Para determinar si esta fórmula es satisfacible, calculamos la siguiente tabla de verdad.

A	B	$A \rightarrow B$	$A \wedge (A \rightarrow B)$	$(A \wedge (A \rightarrow B)) \rightarrow B$
0	0	1	0	1
0	1	1	0	1
1	0	0	0	1
1	1	1	1	1

Vemos que $(A \wedge (A \rightarrow B)) \rightarrow B$ tiene valor de verdad 1 bajo cualquier asignación. Por lo tanto, esta fórmula no solo es satisfacible, sino que además es válida. ▲

Ejemplo 1.8

Consideremos ahora la fórmula $((A \rightarrow B) \rightarrow A) \wedge \neg A$. Supongamos que queremos determinar si esta fórmula es satisfacible o no. De nuevo, elaboramos una tabla de verdad.

A	B	$(A \rightarrow B)$	$((A \rightarrow B) \rightarrow A)$	$\neg A$	$((A \rightarrow B) \rightarrow A) \wedge \neg A$
0	0	1	0	1	0
0	1	1	0	1	0
1	0	0	1	0	0
1	1	1	1	0	0

Esta fórmula es insatisfacible. Es una contradicción



En teoría, podemos determinar si una fórmula F es válida, satisfacible o insatisfacible consultando una tabla de verdad. Por desgracia, esto no siempre es un método eficiente. Si F contiene n fórmulas atómicas, hay que calcular 2^n filas en la tabla de verdad de F . Así pues, si F tiene, por ejemplo, 23 fórmulas atómicas, calcular una tabla de verdad no es factible. Uno de nuestros objetivos en este capítulo es encontrar métodos alternativos para resolver los problemas de validez y satisfacibilidad que eviten el uso de tablas de verdad. En términos más generales, nuestro objetivo es idear diversas formas de determinar si una fórmula dada es o no una consecuencia de un conjunto dado de fórmulas. Este es un problema central de cualquier lógica.

1.3 Consecuencia y equivalencia

A continuación, introducimos el concepto fundamental de consecuencia. En primer lugar, definimos qué significa que una fórmula sea consecuencia de otra. Más adelante en esta sección, definimos de forma similar que significa que una fórmula sea consecuencia de un conjunto de fórmulas.

Definición 1.10

La fórmula G es una *consecuencia* de la fórmula F si, para toda asignación $\mathcal{A}$, si $\mathcal{A} \models F$, entonces $\mathcal{A} \models G$. Lo denotamos como $F \models G$.

Ten en cuenta que el símbolo $\models$ se utiliza de diversas formas. Siempre hay una fórmula a la derecha de este símbolo. Cuando escribimos $\models F$, la interpretación de " $\models$ " depende de con qué rellenemos el espacio en blanco. El espacio en blanco puede rellenarse con una asignación $\mathcal{A}$, con una fórmula G o dejarse vacío (conjunto vacío). Las tres interpretaciones correspondientes a $\models$ son las siguientes:

- $\mathcal{A} \models F$ significa que $\mathcal{A}(F) = 1$. Esto se lee como " $\mathcal{A}$ modela F ".
- $G \models F$ significa que toda asignación que modela a G también modela F . Es decir, F es una consecuencia de G .
- $\models F$ significa que toda asignación satisface F . Es decir, F es una tautología.

Así pues, aunque $\models$ tiene múltiples interpretaciones, en contexto no resulta ambiguo. El concepto de consecuencia está estrechamente relacionado con el concepto de "implica" que se ha tratado en la sección 1.1. Una fórmula G es consecuencia de una fórmula F si y solo si " F implica G " es siempre verdadero. Reformulamos esto como la siguiente proposición.

Proposición 1.1: Consecuencia

Para cualquier fórmula F y G , G es una *consecuencia* de F si y solo si $F \rightarrow G$ es una tautología.

Demostración. Demostraremos que $F \rightarrow G$ no es tautología si y solo si G no es una consecuencia de F . Según la definición de "tautología", $F \rightarrow G$ no es una tautología si y solo si existe una asignación $\mathcal{A}$ tal que $\mathcal{A} \models \neg(F \rightarrow G)$. Según la definición de " $\rightarrow$ ", $\mathcal{A} \models \neg(F \rightarrow G)$ si y solo si $\mathcal{A} \models \neg(\neg F \vee G)$. Según la semántica de la lógica proposicional, $\mathcal{A} \models \neg(\neg F \vee G)$ si y solo si tanto $\mathcal{A} \models F$ como $\mathcal{A} \models \neg G$. Por último, según la definición de "consecuencia", existe una asignación $\mathcal{A}$ tal que $\mathcal{A} \models F$ y $\mathcal{A} \models \neg G$ si y solo si G no es una consecuencia de F . ■

Supongamos que queremos determinar si la fórmula G es o no una consecuencia de la fórmula F . A esto lo denominamos "probelma de la consecuencia". Según la porposición 1.1 esto puede reformularse como un problema de validez (puesto que G es consecuencia de F si y solo si $G \rightarrow F$ es válido). Estos problemas pueden resolverse calculando una tabla de verdad. Si los valores de verdad de $F \rightarrow G$ son todos 1, entonces G es una consecuencia de F . De lo contrario, no lo es. En particular, si F es una contradicción, entonces G es una consecuencia de F independientemente de G .

Ejemplo 1.9

Sean F y G fórmulas. cada una de las siguientes afirmaciones puede verificarse fácilmente calculando la tabla de verdad.

$$(F \wedge G) \models F$$

$$F \models (F \vee G)$$

$$(F \wedge \neg F) \models G$$

**Definición 1.11: Equivalencia**

Si G es una consecuencia de F y F es una consecuencia de G , entonces decimos que F y G son equivalentes. Lo expresamos como $F \equiv G$.

De la proposición 1.1 se deduce que dos fórmulas F y G son equivalentes si y solo si $F \leftrightarrow G$ es una tautología. Por lo tanto, podemos determinar si dos fórmulas F y G son equivalentes calculando una tabla de verdad. Cada una de las equivalencias de los siguientes ejemplos puede verificarse fácilmente de esta manera.

Ejemplo 1.10

Para todas las fórmulas F y G , $(F \wedge G) \equiv (G \wedge F)$ y $(F \vee G) \equiv (G \vee F)$



Ejemplo 1.11

Para cualquier fórmula F y cualquier tautología T , $(F \wedge T) \equiv F$ y $(F \vee T) \equiv T$. ▲

Ejemplo 1.12

Para cualquier fórmula F y cualquier contradicción $\perp$, $(F \wedge \perp) \equiv \perp$ y $(F \vee \perp) \equiv F$. ▲

Ejemplo 1.13 (Leyes distributivas)

Las dos equivalencias siguientes muestran las reglas de distributividad para $\wedge$ y $\vee$. Para todas las fórmulas F , G y H ,

$$(F \wedge (G \vee H)) \equiv ((F \wedge G) \vee (F \wedge H)) \text{ y}$$

$$(F \vee (G \wedge H)) \equiv ((F \vee G) \wedge (F \vee H)).$$

▲

Ejemplo 1.14 (Leyes de De Morgan)

Para todas las fórmulas F y G

$$\neg(F \wedge G) \equiv (\neg F \vee \neg G), \text{ y}$$

$$\neg(F \vee G) \equiv (\neg F \wedge \neg G).$$

▲

1.4 Demostraciones formales

Una lógica, por definición, cuenta con reglas para deducir la verdad de un enunciado a partir de la de otro. Estas reglas dan lugar a un sistema de demostración formal. En esta sección describimos uno de estos sistemas para la lógica proposicional.

Un sistema de demostración consiste en un conjunto de reglas básicas de deducción. Estas reglas nos permiten deducir fórmulas a partir de conjuntos de fórmulas. Puede requerirse de varios pasos para deducir una fórmula G dada a partir de un conjunto de fórmulas $\mathcal{F}$, donde cada "paso" es una aplicación de alguna de las reglas básicas. La lista de estos pasos constituye una demostración formal de G a partir de $\mathcal{F}$.

Nos interesa en particular la relación entre la noción de demostración formal y la noción de consecuencia. Queremos un sistema de demostración que sea *correcto*. Es decir, queremos que se cumpla la siguiente propiedad.

(Correctud) Si una fórmula G puede deducirse de un conjunto de fórmulas $\mathcal{F}$, entonces G es una consecuencia de $\mathcal{F}$.

Si un sistema de demostración es correcto, entonces constituye una alternativa a las tablas de verdad para determinar si una fórmula G es consecuencia de un conjunto de fórmulas $\mathcal{F}$. En esta sección

presentamos un sistema de demostración y demostramos que es correcto. Utilizamos este sistema en varios ejemplos. El sistema de demostración que presentamos está pensado para ser cómodo de usar. Para construir una demostración que deduzca G a partir de $\mathcal{F}$, uno puede plantearse la pregunta: ¿por qué G es consecuencia de $\mathcal{F}$? El objetivo es entonces traducir ese razonamiento a una demostración formal. Aunque este proceso es necesariamente puntilloso, el conjunto amplio pero coherente de reglas básicas que ofrecemos está pensado para facilitar la traducción del pensamiento a demostración formal.

El propósito de un sistema de demostración formal es hacer infalible el proceso de razonamiento. Idealmente, el sistema de demostración podría sustituir por completo al pensamiento. En lugar de pensar, podríamos simplemente seguir un conjunto de reglas de manera ciega. Si dos personas no están de acuerdo sobre si G es en verdad consecuencia de $\mathcal{F}$, no habría necesidad de debatir. Ambas partes podrían realizar un cómputo para ver si G es, en efecto, consecuencia de $\mathcal{F}$. La razón por la cual "no pensar" resulta ideal es que podríamos programar una computadora para realizar esta tarea. En la Sección 1.8 introducimos otro sistema de demostración conocido como *resolución*. La resolución es un sistema de demostración reducido que elimina el pensamiento de las demostraciones formales. Mientras que la resolución está pensada para la mecanización de las demostraciones, el sistema que describimos en esta sección está pensado para uso humano.

A continuación enumeramos las reglas básicas de nuestro sistema de demostración. La veracidad de muchas de estas reglas es evidente por sí misma. Por ejemplo, si F y G pueden deducirse de $\mathcal{F}$, entonces $F \wedge G$ también puede deducirse de $\mathcal{F}$. A esta regla la llamamos " $\wedge$ -Introducción." Utilizamos la siguiente notación: escribimos $\mathcal{F} \vdash G$ para abreviar " G puede deducirse de $\mathcal{F}$." Usando esta notación, $\wedge$ -Introducción se escribe como:

$$\text{si } \mathcal{F} \vdash F \text{ y } \mathcal{F} \vdash G \text{ entonces } \mathcal{F} \vdash (F \wedge G).$$

La Tabla 1.5 enumera esta regla y otras reglas básicas de deducción.

Hay muchas reglas. Obsérvese la organización de la lista. Comienza con un par de reglas bastante intuitivas: Suposición y Monotonidad. Le siguen varias reglas con nombres similares para los distintos símbolos de la lógica proposicional. Las cuatro reglas que cierran la lista reflejan las convenciones (C1) y (C2). Además de las reglas de la Tabla 1.5, contamos con las reglas de la Tabla 1.6 relativas a las definiciones de $\vee$, $\rightarrow$ y $\leftrightarrow$.

Nuestra lista de reglas es a la vez demasiado grande y demasiado pequeña. Es demasiado grande en el sentido de que algunas de estas reglas son redundantes. Por ejemplo, dado que $\vee$ puede expresarse en función de $\neg$ y $\wedge$, la regla $\vee$ -Simetría se sigue de $\wedge$ -Simetría (véase el Ejercicio 1.13). Podríamos eliminar estas reglas redundantes de nuestra lista. De hecho, ¡en realidad solo necesitamos tres reglas! Estas reglas deben formularse en el contexto adecuado y son el tema de la Sección 1.8. Nuestro interés actual no es la economía, sino la utilidad. Estas reglas nos permiten deducir fórmulas a partir de otras fórmulas. Cuantas más reglas tengamos a nuestra disposición, mejor. En este sentido, la lista anterior es demasiado pequeña. Como veremos, no hay fin a las reglas que pueden deducirse a partir de las enunciadas anteriormente.

Habiendo enumerado estas reglas de deducción, definimos ahora "demostración formal".

Tabla 1.5 Reglas básicas de deducción

Premisa	Conclusión	Nombre
G está en $\mathcal{F}$	$\mathcal{F} \vdash G$	Suposición
$\mathcal{F} \vdash G$ y $\mathcal{F} \subset \mathcal{F}'$	$\mathcal{F}' \vdash G$	Monotonicidad
$\mathcal{F} \vdash G$	$\mathcal{F} \vdash \neg\neg G$	Doble negación
$\mathcal{F} \vdash F, \mathcal{F} \vdash G$	$\mathcal{F} \vdash (F \wedge G)$	$\wedge$ -Introducción
$\mathcal{F} \vdash (F \wedge G)$	$\mathcal{F} \vdash F$	$\wedge$ -Eliminación
$\mathcal{F} \vdash (F \wedge G)$	$\mathcal{F} \vdash (G \wedge F)$	$\wedge$ -Simetría
$\mathcal{F} \vdash F$	$\mathcal{F} \vdash (F \vee G)$	$\vee$ -Introducción
$\mathcal{F} \vdash (F \vee G),$		
$\mathcal{F} \cup \{F\} \vdash H, \mathcal{F} \cup \{G\} \vdash H$	$\mathcal{F} \vdash H$	$\vee$ -Eliminación
$\mathcal{F} \vdash (F \vee G)$	$\mathcal{F} \vdash (G \vee F)$	$\vee$ -Simetría
$\mathcal{F} \cup \{F\} \vdash G$	$\mathcal{F} \vdash (F \rightarrow G)$	$\rightarrow$ -Introducción
$\mathcal{F} \vdash (F \rightarrow G), \mathcal{F} \vdash F$	$\mathcal{F} \vdash G$	$\rightarrow$ -Eliminación
$\mathcal{F} \vdash F$	$\mathcal{F} \vdash (F)$	$(,)$ -Introducción
$\mathcal{F} \vdash (F)$	$\mathcal{F} \vdash F$	$(,)$ -Eliminación
$\mathcal{F} \vdash ((F \wedge G) \wedge H)$	$\mathcal{F} \vdash (F \wedge G \wedge H)$	Regla de paréntesis para $\wedge$
$\mathcal{F} \vdash ((F \vee G) \vee H)$	$\mathcal{F} \vdash (F \vee G \vee H)$	Regla de paréntesis para $\vee$

Tabla 1.6 Más reglas de deducción

Reglas	Nombre
$\mathcal{F} \vdash (F \vee G)$ si y solo si $\mathcal{F} \vdash \neg(\neg F \wedge \neg G)$	$\vee$ -Definición
$\mathcal{F} \vdash (F \rightarrow G)$ si y solo si $\mathcal{F} \vdash (\neg F \vee G)$	$\rightarrow$ -Definición
$\mathcal{F} \vdash (F \leftrightarrow G)$ si y solo si tanto $\mathcal{F} \vdash (F \rightarrow G)$ como $\mathcal{F} \vdash (G \rightarrow F)$	$\leftrightarrow$ -Definición

Definición 1.12: Demostración formal

Una demostración formal en lógica proposicional es una sucesión finita de enunciados de la forma " $X \vdash Y$ " (donde X es un conjunto de fórmulas e Y es una fórmula) cada uno de los cuales se sigue de los enunciados anteriores mediante alguna de las reglas de la Tabla 1.5 o de la Tabla 1.6. Decimos que G puede deducirse de $\mathcal{F}$ si existe una demostración formal que concluye con el enunciado $\mathcal{F} \vdash G$.

Una demostración formal siempre puede escribirse en forma de dos columnas. La mejor manera de describir las demostraciones formales es dar un ejemplo.

Ejemplo 1.15

Sea $\mathcal{H} = \{(\neg A \vee B), (\neg A \vee C), (A \vee \neg D)\}$.

Deducimos la fórmula $D \rightarrow (A \wedge B \wedge C)$ a partir de $\mathcal{H}$.

Afirmación	Justificación
1. $\mathcal{H} \cup \{D\} \vdash D$	Suposición
2. $\mathcal{H} \cup \{D\} \vdash (A \vee \neg D)$	Suposición
3. $\mathcal{H} \cup \{D\} \vdash (\neg D \vee A)$	$\vee$ -Simetría aplicada a 2
4. $\mathcal{H} \cup \{D\} \vdash (D \rightarrow A)$	$\rightarrow$ -Definición aplicada a 3
5. $\mathcal{H} \cup \{D\} \vdash A$	$\rightarrow$ -Eliminación aplicada a 4 y 1
6. $\mathcal{H} \cup \{D\} \vdash (\neg A \vee B)$	Suposición
7. $\mathcal{H} \cup \{D\} \vdash (A \rightarrow B)$	$\rightarrow$ -Definición aplicada a 6
8. $\mathcal{H} \cup \{D\} \vdash B$	$\rightarrow$ -Eliminación aplicada a 7 y 5
9. $\mathcal{H} \cup \{D\} \vdash (\neg A \vee C)$	Suposición
10. $\mathcal{H} \cup \{D\} \vdash (A \rightarrow C)$	$\rightarrow$ -Definición aplicada a 9
11. $\mathcal{H} \cup \{D\} \vdash C$	$\rightarrow$ -Eliminación aplicada a 10 y 5
12. $\mathcal{H} \cup \{D\} \vdash (A \wedge B)$	$\wedge$ -Introducción aplicada a 5 y 8
13. $\mathcal{H} \cup \{D\} \vdash ((A \wedge B) \wedge C)$	$\wedge$ -Introducción aplicada a 12 y 11
14. $\mathcal{H} \cup \{D\} \vdash (A \wedge B \wedge C)$	Regla de paréntesis para $\wedge$ aplicada a 13
15. $\mathcal{H} \vdash (D \rightarrow (A \wedge B \wedge C))$	$\rightarrow$ -Introducción aplicada a 14
16. $\mathcal{H} \vdash D \rightarrow (A \wedge B \wedge C)$	(,)-Eliminación



A primera vista, la demostración anterior parece una manera complicada de mostrar un hecho que no es tan complicado. Sin embargo, al leer la demostración línea por línea, vemos que cada línea afirma una verdad sencilla y fácil de verificar. Las demostraciones pueden hacerse más concisas usando reglas

adicionales. Por ejemplo, obsérvese que en la demostración anterior introducimos repetidamente el símbolo $\rightarrow$ solo para eliminarlo después. En su lugar, podríamos haber demostrado por separado la siguiente regla.

Premisa: $\mathcal{F} \vdash (\neg F \vee G), \mathcal{F} \vdash F$

Conclusión: $\mathcal{F} \vdash G$

Afirmación	Justificación
1. $\mathcal{F} \vdash F$	Premisa
2. $\mathcal{F} \vdash (\neg F \vee G)$	Premisa
3. $\mathcal{F} \vdash (F \rightarrow G)$	$\rightarrow$ -Definición aplicada a 2
4. $\mathcal{F} \vdash G$	$\rightarrow$ -Eliminación aplicada a 3 y 1

La demostración del Ejemplo 1.15 repite esencialmente el argumento anterior tres veces. Podríamos considerar esta demostración de cuatro líneas como una subrutina. De haber establecido esta regla antes del Ejemplo 1.15, podríamos habernos referido a ella tres veces para hacer la demostración más concisa. A falta de un nombre mejor, bautizamos esta regla como " $\vee$ -Modus Ponens." "Modus ponens" es un nombre estándar para la regla que llamamos $\rightarrow$ -Eliminación. Como sugiere su nombre arcaico, esta regla lleva mucho tiempo entre nosotros. Se encuentra en lo que puede considerarse el origen de las demostraciones formales: los *Elementos* de Euclides. Los siguientes cinco ejemplos establecen otras reglas que facilitan la construcción de demostraciones.

Ejemplo 1.16 (Regla de la tautología)

Esta regla afirma que, para cualquier fórmula G , $(\neg G \vee G)$ puede deducirse de cualquier conjunto de fórmulas.

Premisa: Ninguna

Conclusión: $\mathcal{F} \vdash (\neg G \vee G)$

Afirmación	Justificación
1. $\mathcal{F} \cup \{G\} \vdash G$	Suposición
2. $\mathcal{F} \vdash (G \rightarrow G)$	$\rightarrow$ -Introducción aplicada a 1
3. $\mathcal{F} \vdash (\neg G \vee G)$	$\rightarrow$ -Definición aplicada a 2

▲

Ejemplo 1.17 (Regla de la contradicción)

Esta regla afirma que cualquier fórmula G puede deducirse de la contradicción $F \wedge \neg F$.

Premisa: $\mathcal{F} \vdash (F \wedge \neg F)$

Conclusión: $\mathcal{F} \vdash G$

Afirmación	Justificación
1. $\mathcal{F} \vdash (F \wedge \neg F)$	Premisa
2. $\mathcal{F} \vdash (\neg F \wedge F)$	$\wedge$ -Simetría aplicada a 1
3. $\mathcal{F} \vdash \neg F$	$\wedge$ -Eliminación aplicada a 2
4. $\mathcal{F} \vdash (\neg F \vee G)$	$\vee$ -Introducción aplicada a 3
5. $\mathcal{F} \vdash F$	$\wedge$ -Eliminación aplicada a 1
6. $\mathcal{F} \vdash G$	$\vee$ -Modus Ponens aplicada a 4 y 5

▲

Ejemplo 1.18 (Contrapositiva)

Esta es la regla de la lógica que afirma que si p implica q , entonces $\neg q$ implica $\neg p$.

Premisa: $\mathcal{F} \cup \{F\} \vdash G$

Conclusión: $\mathcal{F} \cup \{\neg G\} \vdash \neg F$

Afirmación	Justificación
1. $\mathcal{F} \cup \{F\} \vdash G$	Premisa
2. $\mathcal{F} \cup \{F\} \vdash \neg\neg G$	Doble negación aplicada a 1
3. $\mathcal{F} \vdash (F \rightarrow \neg\neg G)$	$\rightarrow$ -Introducción aplicada a 2
4. $\mathcal{F} \vdash (\neg F \vee \neg\neg G)$	$\rightarrow$ -Definición aplicada a 3
5. $\mathcal{F} \vdash (\neg\neg G \vee \neg F)$	$\vee$ -Simetría aplicada a 4
6. $\mathcal{F} \vdash (\neg G \rightarrow \neg F)$	$\rightarrow$ -Definición aplicada a 5
7. $\mathcal{F} \cup \{\neg G\} \vdash (\neg G \rightarrow \neg F)$	Monotonía aplicada a 6
8. $\mathcal{F} \cup \{\neg G\} \vdash \neg G$	Suposición
9. $\mathcal{F} \cup \{\neg G\} \vdash \neg F$	$\rightarrow$ -Eliminación aplicada a 7 y 8

▲

Ejemplo 1.19 (Demostración por casos)

Esta regla proporciona una manera útil de estructurar una demostración.

Premisa: $\mathcal{F} \cup \{F\} \vdash G, \mathcal{F} \cup \{\neg F\} \vdash G$

Conclusión: $\mathcal{F} \vdash G$

Afirmación	Justificación
1. $\mathcal{F} \cup \{F\} \vdash G$	Suposición
2. $\mathcal{F} \cup \{\neg F\} \vdash G$	Suposición
3. $\mathcal{F} \vdash (\neg F \vee F)$	Regla de la tautología
4. $\mathcal{F} \vdash G$	$\vee$ -Eliminación aplicada a 3, 2 y 1

▲

Ejemplo 1.20 (Demostración por contradicción)

Otra manera de estructurar una demostración es por contradicción. Como indica la siguiente demostración, la demostración por contradicción es (en este contexto) esencialmente la contrapositiva de la demostración por casos.

Premisa: $\mathcal{F} \cup \{F\} \vdash G$, $\mathcal{F} \cup \{F\} \vdash \neg G$

Conclusión: $\mathcal{F} \vdash \neg F$

Afirmación	Justificación
1. $\mathcal{F} \cup \{F\} \vdash G$	Premisa
2. $\mathcal{F} \cup \{\neg G\} \vdash \neg F$	Contrapositiva aplicada a 1
3. $\mathcal{F} \cup \{F\} \vdash \neg G$	Premisa
4. $\mathcal{F} \cup \{\neg \neg G\} \vdash \neg F$	Contrapositiva aplicada a 3
5. $\mathcal{F} \vdash \neg F$	Demostración por casos aplicada a 2 y 4

▲

Estas son ahora reglas establecidas que en adelante pueden usarse para justificar enunciados en demostraciones. Por "establecidas" entendemos que estas reglas son verdaderas, siempre que las reglas de la Tabla 1.5 lo sean. Debemos demostrar que este es el caso. Debemos demostrar que nuestro sistema de demostración es correcto: que si G puede deducirse de $\mathcal{F}$, entonces G es en efecto consecuencia de $\mathcal{F}$.

Teorema 1.1: Correctud

Si $\mathcal{F} \vdash G$, entonces $\mathcal{F} \models G$.

Demostración. Si $\mathcal{F} \vdash G$, entonces existe una demostración formal que concluye con $\mathcal{F} \vdash G$. Cada línea de la demostración contiene un enunciado de la forma $X \vdash Y$ justificado por alguna de las reglas de la Tabla 1.5 o de la Tabla 1.6. Queremos mostrar que, para cada línea de la demostración, si $X \vdash Y$, entonces $X \models Y$. Esto puede lograrse verificando cada regla de las tablas una por una. Ilustramos esto verificando tres reglas: Suposición, $\wedge$ -Eliminación y $\rightarrow$ -Introducción. La verificación

de las reglas restantes se deja como ejercicio.

Comenzamos con la primera regla de la Tabla 1.5: Suposición. La conclusión de esta regla es que $\mathcal{F} \vdash G$. Debemos mostrar, bajo la premisa de esta regla, que $\mathcal{F} \models G$. Pero esto es claro, puesto que la premisa establece que G está en $\mathcal{F}$ (si $\mathcal{A}$ modela $\mathcal{F}$, entonces $\mathcal{A}$ debe modelar G).

Consideremos ahora $\wedge$ -Eliminación. Esta regla afirma que si $\mathcal{F} \vdash (F \wedge G)$, entonces $\mathcal{F} \vdash F$. Debemos mostrar que si $\mathcal{F} \models (F \wedge G)$, entonces $\mathcal{F} \models F$. Es decir, debemos mostrar que F es consecuencia de $(F \wedge G)$. Esto se verifica mediante la siguiente tabla de verdad.

F	G	$(F \wedge G)$	$((F \wedge G) \rightarrow F)$
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	1

Consideremos ahora $\rightarrow$ -Introducción. Esta regla afirma que si $\mathcal{F} \cup \{F\} \vdash G$ entonces $\mathcal{F} \vdash (F \rightarrow G)$. Para verificar esto, debemos mostrar que si $\mathcal{F} \cup \{F\} \models G$ entonces $\mathcal{F} \models (F \rightarrow G)$. Suponiendo que $\mathcal{F} \cup \{F\} \models G$, queremos mostrar que, para cualquier asignación $\mathcal{A}$, si $\mathcal{A} \models \mathcal{F}$, entonces $\mathcal{A} \models (F \rightarrow G)$. Supongamos entonces que $\mathcal{A} \models \mathcal{F}$ y que $\mathcal{A}(F)$ está definido. Si $\mathcal{A}(F) = 0$, entonces $\mathcal{A} \models (F \rightarrow G)$ independientemente del valor de $\mathcal{A}(G)$. Si, por otro lado, $\mathcal{A}(F) = 1$, entonces $\mathcal{A} \models \mathcal{F} \cup \{F\}$. Por nuestra suposición, $\mathcal{A} \models G$. En cualquier caso, vemos que $\mathcal{A} \models (F \rightarrow G)$. Dado que $\mathcal{A}$ era una asignación arbitraria que modela $\mathcal{F}$, concluimos que $\mathcal{F} \models (F \rightarrow G)$, como se requería.

En esencia, debemos verificar que cada regla es verdadera al reemplazar $\vdash$ por $\models$. Para la mayoría de las reglas, como $\wedge$ -Eliminación, esto puede lograrse calculando una pequeña tabla de verdad. Para las últimas cuatro reglas de la Tabla 1.5, en realidad no hay nada que demostrar. Estas cuatro reglas se cumplen por las convenciones (C1) y (C2). Asimismo, cada regla de la Tabla 1.6 es correcta en virtud de las definiciones de $\forall$, $\rightarrow$ y $\leftrightarrow$. Dejamos la verificación de las reglas restantes de la Tabla 1.5 como Ejercicio 1.22. ■

Las demostraciones formales proporcionan un método para mostrar que una fórmula es consecuencia de otras fórmulas. Los siguientes corolarios establecen que las demostraciones formales también pueden mostrar que una fórmula es válida o insatisfacible.

Corolario 1.1

Si G puede deducirse del conjunto vacío, entonces G es una tautología.

Demostración. Si $\emptyset \vdash G$, entonces, por Monotonización, $\mathcal{F} \vdash G$ para todo conjunto de fórmulas $\mathcal{F}$. Por el Teorema 1.1, $\mathcal{F} \models G$ para todo conjunto de fórmulas $\mathcal{F}$. Se sigue que $\mathcal{A} \models G$ para cualquier asignación $\mathcal{A}$ y G es una tautología. ■

Corolario 1.2

Si $\neg G$ puede deducirse del conjunto vacío, entonces G es una contradicción.

Demostración. Esto es inmediato a partir del corolario anterior y de la definición de "contradicción." ■

Ejemplo 1.21

La siguiente demostración formal muestra que $((A \rightarrow B) \vee A)$ es una tautología.

Afirmación	Justificación
1. $\{\neg A\} \vdash \neg A$	Suposición
2. $\{\neg A\} \vdash (\neg A \vee B)$	$\vee$ -Introducción aplicada a 1
3. $\{\neg A\} \vdash (A \rightarrow B)$	$\rightarrow$ -Definición aplicada a 2
4. $\{\neg A\} \vdash ((A \rightarrow B) \vee A)$	$\vee$ -Introducción aplicada a 3
5. $\{A\} \vdash A$	Suposición
6. $\{A\} \vdash (A \vee (A \rightarrow B))$	$\vee$ -Introducción aplicada a 5
7. $\{A\} \vdash ((A \rightarrow B) \vee A)$	$\vee$ -Simetría aplicada a 6
8. $\emptyset \vdash ((A \rightarrow B) \vee A)$	Demostración por casos aplicada a 4 y 7

Las demostraciones formales también pueden mostrar que dos fórmulas son equivalentes. ▲

Definición 1.13

Las fórmulas F y G son *equivalentes de manera demostrable* si tanto $\{F\} \vdash G$ como $\{G\} \vdash F$.

Corolario 1.3

Si F y G son equivalentes de manera demostrable, entonces son equivalentes.

Demostración. Esto se sigue de inmediato del Teorema 1.1. ■

Consideremos ahora los recíprocos del Teorema 1.1 y de sus corolarios. El Teorema 1.1 afirma que si G puede deducirse formalmente de $\mathcal{F}$, entonces G es consecuencia de $\mathcal{F}$. ¿Es cierto lo contrario? ¿Podemos deducir de $\mathcal{F}$ toda consecuencia de $\mathcal{F}$? ¿Puede darse una demostración formal de toda tautología, como en el Ejemplo 1.21? Si dos fórmulas son equivalentes, ¿significa esto que podemos demostrar que son equivalentes? Afirmamos que la respuesta a cada una de estas preguntas es "sí."

Afirmamos que toda regla que sea verdadera en lógica proposicional, y son infinitas, puede deducirse a partir de las reglas de las Tablas 1.5 y 1.6. Esto no es evidente.

Ejemplo 1.22

Podría parecer que nuestra lista de reglas está incompleta. Por ejemplo, las fórmulas F y $\neg\neg F$ son claramente equivalentes. Así que si podemos deducir la fórmula $\neg\neg F$ a partir de un conjunto de fórmulas $\mathcal{F}$, deberíamos también poder deducir F a partir de $\mathcal{F}$. Sin embargo, esta no es una de nuestras reglas. Doble negación afirma que si $\mathcal{F} \vdash F$, entonces $\mathcal{F} \vdash \neg\neg F$. Mostramos ahora que el recíproco de Doble negación, aunque no está enunciado como regla, puede deducirse a partir de nuestras reglas.

Premisa: $\mathcal{F} \vdash \neg\neg F$

Conclusión: $\mathcal{F} \vdash F$

Afirmación	Justificación
1. $\mathcal{F} \vdash \neg\neg F$	Premisa
2. $\mathcal{F} \cup \{\neg F\} \vdash \neg\neg F$	Monotonicidad aplicada a 1
3. $\mathcal{F} \cup \{\neg F\} \vdash \neg F$	Suposición
4. $\mathcal{F} \cup \{\neg F\} \vdash (\neg F \wedge \neg\neg F)$	$\wedge$ -Introducción aplicada a 3 y 2
5. $\mathcal{F} \cup \{\neg F\} \vdash F$	Regla de la contradicción (Ejemplo 1.17) aplicada a 4
6. $\mathcal{F} \cup \{F\} \vdash F$	Suposición
7. $\mathcal{F} \vdash F$	Demostración por casos aplicada a 5 y 6

▲

Así pues, no solo F y $\neg\neg F$ son fórmulas equivalentes; podemos demostrar formalmente que son fórmulas equivalentes. Afirmamos que cada una de las equivalencias de la sección anterior son, de hecho, equivalentes de manera demostrable. En particular, mostramos que las reglas de Distributividad del Ejemplo 1.13 y las Leyes de De Morgan del Ejemplo 1.14 admiten deducciones formales.

Proposición 1.2: Leyes de De Morgan

Los pares de fórmulas equivalentes del Ejemplo 1.14 son, cada uno, equivalentes de manera demostrable.

Demostración. Demostramos esto para la segunda de las Leyes de De Morgan. Exhibimos demostraciones formales de cada una de las siguientes:

$$\{\neg(F \vee G)\} \vdash (\neg F \wedge \neg G), \text{ y}$$

$$\{(\neg F \wedge \neg G)\} \vdash \neg(F \vee G).$$

Afirmación	Justificación
1. $\{\neg(\neg F \wedge \neg G)\} \vdash (F \vee G)$	$\vee$ -Definición
2. $\{\neg(F \vee G)\} \vdash \neg\neg(\neg F \wedge \neg G)$	Contrapositiva aplicada a 1
3. $\{\neg(F \vee G)\} \vdash (\neg F \wedge \neg G)$	Doble negación aplicada a 2

Afirmación	Justificación
1. $\{(\neg F \wedge \neg G)\} \cup \{(F \vee G)\} \vdash (F \vee G)$	Suposición
2. $\{(\neg F \wedge \neg G)\} \cup \{(F \vee G)\} \vdash (\neg F \wedge \neg G)$	Suposición
3. $\{(\neg F \wedge \neg G)\} \cup \{(F \vee G)\} \vdash \neg F$	$\wedge$ -Eliminación aplicada a 2
4. $\{(\neg F \wedge \neg G)\} \cup \{(F \vee G)\} \vdash G$	$\vee$ -Eliminación aplicada a 1 y 3
5. $\{(\neg F \wedge \neg G)\} \cup \{(F \vee G)\} \vdash (\neg G \wedge \neg F)$	$\wedge$ -Simetría aplicada a 2
6. $\{(\neg F \wedge \neg G)\} \cup \{(F \vee G)\} \vdash \neg G$	$\wedge$ -Eliminación aplicada a 5
7. $\{(\neg F \wedge \neg G)\} \vdash \neg(F \vee G)$	Demostración por contradicción aplicada a 4 y 6

Hemos demostrado que $\neg(F \vee G)$ y $(\neg F \wedge \neg G)$ son equivalentes de manera demostrable. La verificación de la primera Ley de De Morgan se deja como Ejercicio 1.23. ■

Proposición 1.3: Distributividad de $\wedge$

Para cualesquiera fórmulas F , G y H , las fórmulas $(F \wedge (G \vee H))$ y $((F \wedge G) \vee (F \wedge H))$ son equivalentes de manera demostrable.

Demostración. Para demostrar esto, debemos deducir cada fórmula a partir de la otra. En lugar de dar demostraciones formales, esbozamos las deducciones y dejamos los detalles al lector. Primero mostramos que $(F \wedge G) \vee (F \wedge H)$ puede deducirse de $F \wedge (G \vee H)$.

Premisa: $\mathcal{F} \vdash F \wedge (G \vee H)$.

Conclusión: $\mathcal{F} \vdash (F \wedge G) \vee (F \wedge H)$

Esbozamos una demostración formal usando Demostración por casos. Suponiendo la premisa, mostramos que $(F \wedge G) \vee (F \wedge H)$ puede deducirse tanto de $\mathcal{F} \cup \{G\}$ como de $\mathcal{F} \cup \{\neg G\}$.

A partir de la premisa, vemos que $\mathcal{F} \cup \{G\} \vdash F$. Se sigue que $(F \wedge G)$ puede deducirse de $\mathcal{F} \cup \{G\}$. Obtenemos entonces $\mathcal{F} \cup \{G\} \vdash (F \wedge G) \vee (F \wedge H)$ por $\vee$ -Introducción.

A continuación mostramos que $\mathcal{F} \cup \{\neg G\} \vdash (F \wedge G) \vee (F \wedge H)$. A partir de la premisa vemos que tanto F como $(G \vee H)$ pueden deducirse de $\mathcal{F} \cup \{\neg G\}$. Dado que $\mathcal{F} \cup \{\neg G\} \vdash \neg G$, obtenemos $\mathcal{F} \cup \{\neg G\} \vdash H$ a partir de $(G \vee H)$ por $\vee$ -Modus Ponens. Se sigue que $\mathcal{F} \cup \{\neg G\} \vdash (F \wedge H)$. Finalmente, obtenemos $\mathcal{F} \cup \{\neg G\} \vdash (F \wedge G) \vee (F \wedge H)$ por $\vee$ -Introducción.

Debemos mostrar también que se cumple el recíproco.

Premisa: $\mathcal{F} \vdash (F \wedge G) \vee (F \wedge H)$

Conclusión: $\mathcal{F} \vdash F \wedge (G \vee H)$

Demostramos esto aplicando $\vee$ -Eliminación dos veces. Dado que $(G \vee H)$ puede deducirse tanto de $(F \wedge G)$ como de $(F \wedge H)$, obtenemos $\mathcal{F} \vdash (G \vee H)$ aplicando $\vee$ -Eliminación a la premisa. De la misma manera obtenemos $\mathcal{F} \vdash F$. La conclusión se sigue entonces por $\wedge$ -Introducción.

Estos argumentos pueden organizarse como demostraciones formales de dos columnas. Dejamos esto como Ejercicio 1.24. ■

Proposición 1.4: Distributividad de $\vee$

Para cualesquiera fórmulas F , G y H , las fórmulas $(F \vee (G \wedge H))$ y $((F \vee G) \wedge (F \vee H))$ son equivalentes de manera demostrable.

Demostración. Ejercicio 1.25. ■

Desde luego, no necesitamos demostraciones formales para verificar estas equivalencias. Podríamos usar tablas de verdad. En el caso de las reglas de Distributividad y de las Leyes de De Morgan, las tablas de verdad proporcionan un método de verificación más eficiente que las demostraciones formales. Por ahora, la importancia de las Proposiciones 1.2, 1.3 y 1.4 radica en que dan crédito a nuestra afirmación anterior de que podemos demostrar formalmente cualquier cosa que sea verdadera en lógica proposicional. Más adelante, estas proposiciones nos ayudarán a demostrar esa afirmación. Al comienzo de esta sección dijimos que nos interesaría la relación entre la noción de demostración formal y la noción de consecuencia. Demostramos en el Teorema 1.1 que si G puede demostrarse formalmente a partir de $\mathcal{F}$, entonces G es consecuencia de $\mathcal{F}$. Afirmamos, sin demostrarlo, que lo contrario también es cierto: si $\mathcal{F} \models G$ entonces $\mathcal{F} \vdash G$. Así, el símbolo $\models$ introducido en la sección anterior y el símbolo $\vdash$ introducido en la presente sección significan lo mismo en lógica proposicional. Este es el Teorema de completitud para la lógica proposicional, cuya demostración se dará al concluir este capítulo.

1.5 Demostración por inducción

Hay dos tipos de demostraciones que deben distinguirse. Hemos discutido y dado varios ejemplos de demostraciones formales. Este tipo de demostración surge de las reglas de la lógica. Se dice que tales demostraciones tienen lugar *dentro* de la lógica, y nos referimos a ellas como demostraciones internas. Las demostraciones formales tienen un alcance limitado. Solo pueden demostrar enunciados que puedan escribirse en la lógica. En cambio, podemos querer demostrar algo sobre la lógica misma. Podemos querer demostrar, por ejemplo, que toda sentencia de la lógica posee cierta propiedad. Tales enunciados que se refieren a la lógica misma generalmente no pueden ni plantearse ni demostrarse dentro de la lógica. Damos demostraciones externas para tales enunciados. Las demostraciones externas a veces se llaman metamatemáticas. Sin embargo, esta terminología contradice el hecho de que las demostraciones externas suelen ser de naturaleza más matemática que las demostraciones formales.

La inducción es un método de demostración externa que se usa repetidamente en este libro. Supongamos que queremos demostrar que cierta propiedad se cumple para toda fórmula de la lógica proposicional. Por ejemplo, en la siguiente sección mostramos que toda fórmula de la lógica proposicional es equivalente a alguna fórmula en forma normal conjuntiva. Definiremos "forma normal conjuntiva" más adelante. Nuestro interés actual es la pregunta de cómo podemos demostrar tal cosa para todas las fórmulas. Necesitamos una manera sistemática de verificar cada una de las fórmulas F . Hacemos esto por inducción sobre la complejidad de F . La inducción sobre la complejidad de F es análoga a la inducción matemática.

1.5.1. Inducción matemática

Recordemos que la inducción matemática es un método de demostración que nos permite demostrar algo para todos los números naturales. Por ejemplo, supongamos que queremos demostrar que, para todo número natural n , el número $11^n - 4^n$ es divisible entre 7. Usando inducción matemática, podemos hacer esto en dos pasos. Primero, mostramos que el enunciado es verdadero para $n = 1$. Esto es fácil. Segundo, mostramos que si el enunciado se cumple para $n = m$ para algún m , entonces también se cumple para $n = m + 1$. Este es el paso inductivo. En nuestro ejemplo, podemos hacer esto observando que

$$11^{m+1} - 4^{m+1} = 11^{m+1} - 11 \cdot 4^m + 7 \cdot 4^m = 11(11^m - 4^m) + 7 \cdot 4^m.$$

Se sigue que si $11^m - 4^m$ es divisible entre 7, entonces también lo es $11^{m+1} - 4^{m+1}$. Esto completa la demostración. Es como el efecto dominó. Es verdadero para $n = 1$ y, por lo tanto, según el segundo paso de la demostración, debe ser también verdadero para $n = 2$, y por lo tanto para $n = 3$, y $n = 4$, y así sucesivamente. Concluimos que, para todo número natural n , $11^n - 4^n$ es divisible entre 7.

Un ejemplo de inducción matemática más relevante para la lógica proposicional lo proporciona la demostración de la Proposición 1.5. Esta proposición es una generalización de las Leyes de De Morgan. Primero, introducimos algo de notación.

Notación. Sean $F_1, \dots, F_n$ fórmulas. Escribimos

$$\bigwedge_{i=1}^n F_i \quad \text{para abreviar} \quad F_1 \wedge F_2 \wedge \dots \wedge F_n, \text{ y}$$

$$\bigvee_{i=1}^n F_i \quad \text{para abreviar} \quad F_1 \vee F_2 \vee \dots \vee F_n.$$

Proposición 1.5

Sea $\{F_1, \dots, F_n\}$ un conjunto finito de fórmulas. Entonces

$$\neg \left(\bigwedge_{i=1}^n F_i \right) \equiv \bigvee_{i=1}^n \neg F_i \quad \text{y} \quad \neg \left(\bigvee_{i=1}^n F_i \right) \equiv \bigwedge_{i=1}^n \neg F_i.$$

Demostración. Mostramos que $\neg(\bigwedge_{i=1}^n F_i) \equiv \bigvee_{i=1}^n \neg F_i$ por inducción sobre n .

Primero, supongamos $n = 1$. Debemos mostrar que $\neg(\bigwedge_{i=1}^1 F_i) \equiv \bigvee_{i=1}^1 \neg F_i$. Por las definiciones de $\bigwedge$ y $\bigvee$, esto es lo mismo que $\neg(F_1) \equiv (\neg F_1)$, lo cual es verdadero por la convención (C1).

Nuestra hipótesis de inducción es que, para algún $m \geq 1$ y cualesquiera fórmulas $F_1, \dots, F_m$, tenemos

$$\neg\left(\bigwedge_{i=1}^m F_i\right) \equiv \bigvee_{i=1}^m \neg F_i.$$

Queremos mostrar que

$$\neg\left(\bigwedge_{i=1}^{m+1} F_i\right) \equiv \bigvee_{i=1}^{m+1} \neg F_i.$$

Por la definición de $\bigwedge$ tenemos

$$\neg\left(\bigwedge_{i=1}^{m+1} F_i\right) \equiv \neg\left(\left(\bigwedge_{i=1}^m F_i\right) \wedge F_{m+1}\right).$$

Por las Leyes de De Morgan obtenemos

$$\neg\left(\bigwedge_{i=1}^{m+1} F_i\right) \equiv \neg\left(\bigwedge_{i=1}^m F_i\right) \vee \neg F_{m+1}. \quad (1.1)$$

Por nuestra hipótesis de inducción,

$$\neg\left(\bigwedge_{i=1}^m F_i\right) \equiv \bigvee_{i=1}^m \neg F_i.$$

Sustituyendo esto en (1.1) obtenemos

$$\neg\left(\bigwedge_{i=1}^{m+1} F_i\right) \equiv \left(\bigvee_{i=1}^m \neg F_i\right) \vee \neg F_{m+1}.$$

Finalmente, por la definición de $\bigvee$ llegamos a

$$\neg\left(\bigwedge_{i=1}^{m+1} F_i\right) \equiv \bigvee_{i=1}^{m+1} \neg F_i.$$

Hemos mostrado que $\neg(\bigwedge_{i=1}^{m+1} F_i) \equiv \bigvee_{i=1}^{m+1} \neg F_i$, como se requería. Concluimos que $\neg(\bigwedge_{i=1}^n F_i) \equiv \bigvee_{i=1}^n \neg F_i$ para cualquier n .

La segunda equivalencia de la proposición se sigue de la primera. Dado que $\bigvee_{i=1}^{n+1} \neg F_i \equiv \neg(\bigwedge_{i=1}^{n+1} F_i)$ se cumple para cualesquiera fórmulas F_i , se cumple también al reemplazar cada F_i por $\neg F_i$:

$$\bigvee_{i=1}^{n+1} \neg F_i \equiv \neg\left(\bigwedge_{i=1}^{n+1} \neg F_i\right).$$

Dado que estas dos fórmulas son equivalentes, sus negaciones también lo son:

$$\neg\bigvee_{i=1}^{n+1} \neg F_i \equiv \neg\neg\left(\bigwedge_{i=1}^{n+1} \neg F_i\right).$$

Ahora bien, $\neg(\bigwedge_{i=1}^n F_i) \equiv (\bigvee_{i=1}^n \neg F_i)$ por doble negación. ■

De manera similar, podemos generalizar las reglas de distributividad como sigue.

Proposición 1.6

Sean $\{F_1, \dots, F_n\}$ y $\{G_1, \dots, G_m\}$ conjuntos finitos de fórmulas. Se cumplen las siguientes equivalencias:

$$\begin{aligned} \left(\bigwedge_{i=1}^n F_i \right) \vee \left(\bigwedge_{j=1}^m G_j \right) &\equiv \bigwedge_{i=1}^n \bigwedge_{j=1}^m (F_i \vee G_j) \\ \left(\bigvee_{i=1}^n F_i \right) \wedge \left(\bigvee_{j=1}^m G_j \right) &\equiv \bigvee_{i=1}^n \bigvee_{j=1}^m (F_i \wedge G_j) \end{aligned}$$

Demostración. Ejercicio 1.27. ■

Hay un paso no justificado en la demostración de la Proposición 1.5. En el paso marcado con (1.1), esencialmente dijimos que si $G \equiv G'$, entonces $(G \vee F) \equiv (G' \vee F)$. Aunque esta sustitución tiene sentido intuitivo, todavía no la hemos establecido como una regla que podamos usar. Validamos este paso en el Teorema 1.2. Demostramos este teorema por inducción sobre la complejidad de las fórmulas. Describimos ahora este método de demostración.

1.5.2. Inducción sobre la complejidad de las fórmulas

Supongamos que queremos mostrar que la propiedad P se cumple para toda fórmula F . Podemos hacer esto por inducción sobre la complejidad de F de la siguiente manera. Primero mostramos que toda fórmula atómica posee la propiedad P . Esto corresponde a verificar el caso $n = 1$ en la inducción matemática. El caso atómico es nuestra base de inducción. Suponemos entonces que la propiedad P se cumple para las fórmulas G y H . Esta es nuestra hipótesis de inducción. Nuestro objetivo es mostrar que la propiedad P se cumple necesariamente para $\neg G$, $G \wedge H$, $G \vee H$, $G \rightarrow H$ y $G \leftrightarrow H$. Si lo logramos, entonces podemos concluir legítimamente que P se cumple para todas las fórmulas. Esto completa la demostración.

Teorema 1.2: Teorema de sustitución

Supongamos que $F \equiv G$. Sea H una fórmula que contiene a F como subfórmula. Sea H' la fórmula obtenida al reemplazar alguna ocurrencia de F en H por G . Entonces $H \equiv H'$.

Demostración. Demostramos esto por inducción sobre la complejidad de H .

Primero supongamos que H es atómica. Entonces la única subfórmula de H es la propia H . Así, $F = H$. Se sigue que $H' = G$ y, dado que $F \equiv G$, tenemos $H \equiv H'$.

Nuestra hipótesis de inducción es que la conclusión del teorema se cumple para las fórmulas H_1 y H_2 ,

cada una de las cuales contiene una ocurrencia de F como subfórmula. Es decir, $H_1 \equiv H'_1$ y $H_2 \equiv H'_2$ siempre que H'_1 y H'_2 sean fórmulas obtenidas a partir de H_1 y H_2 al reemplazar una ocurrencia de F por G .

Supongamos que $H = \neg H_1$. Entonces $H' = \neg H'_1$. Dado que $H_1 \equiv H'_1$, tenemos $\neg H_1 \equiv \neg H'_1$. Se sigue que $H \equiv H'$, como se requería.

Supongamos que H es una de las siguientes fórmulas: $H_1 \wedge H_2$, $H_1 \vee H_2$, $H_1 \rightarrow H_2$ o $H_1 \leftrightarrow H_2$. Dado que F es subfórmula de H , F es subfórmula de H_1 , subfórmula de H_2 , o es H misma. Si $F = H$, entonces tenemos $H = F \equiv G = H'$ como en el caso atómico. Así que podemos suponer que la ocurrencia de F que ha de reemplazarse por G ocurre en H_1 o en H_2 . Sin pérdida de generalidad, podemos suponer que ocurre en H_1 .

Si $H = H_1 \wedge H_2$ entonces $H' = H'_1 \wedge H_2$. En este caso tenemos:

$H_1 \wedge H_2$ es verdadera si y solo si
tanto H_1 como H_2 son verdaderas si y solo si
tanto H'_1 como H_2 son verdaderas (dado que $H_1 \equiv H'_1$) si y solo si
 $H'_1 \wedge H_2$ es verdadera.

Es decir, $H_1 \wedge H_2 \equiv H'_1 \wedge H_2$. Dado que $H \equiv H_1 \wedge H_2$, tenemos $H \equiv H'$.

Si $H = H_1 \vee H_2$, entonces $H' = H'_1 \vee H_2$. Por la definición de $\vee$, tenemos $H \equiv \neg(\neg H_1 \wedge \neg H_2)$ y $H' \equiv \neg(\neg H'_1 \wedge \neg H_2)$. Se sigue de los casos anteriores (correspondientes a $\neg$ y $\wedge$) que $H \equiv H'$.

Si $H = H_1 \rightarrow H_2$, entonces $H' = H'_1 \rightarrow H_2$. Por la definición de $\rightarrow$, $H \equiv (\neg H_1 \vee H_2)$ y $H' \equiv (\neg H'_1 \vee H_2)$. Se sigue de los casos anteriores (correspondientes a $\neg$ y $\vee$) que $H \equiv H'$.

Si $H = H_1 \leftrightarrow H_2$, entonces $H' = H'_1 \leftrightarrow H_2$. Por la definición de $\leftrightarrow$, $H \equiv (H_1 \rightarrow H_2) \wedge (H_2 \rightarrow H_1)$ y $H' \equiv (H'_1 \rightarrow H_2) \wedge (H_2 \rightarrow H'_1)$. Se sigue de los casos anteriores (correspondientes a $\wedge$ y $\rightarrow$) que $H \equiv H'$.

Concluimos que, para cualquier fórmula H que contenga a F como subfórmula, $H \equiv H'$. ■

De hecho, este teorema sigue siendo cierto al reemplazar " $\equiv$ " por "equivalentes de manera demostrable."

Teorema 1.3

Supongamos que F y G son equivalentes de manera demostrable. Sea H una fórmula que contiene a F como subfórmula. Sea H' la fórmula obtenida al reemplazar alguna ocurrencia de F en H por G . Entonces H y H' son equivalentes de manera demostrable.

Demostración. La demostración es similar a la del Teorema 1.2. Se procede por inducción sobre la complejidad de H . La hipótesis de inducción es que tanto H_1 y H'_1 son equivalentes de manera demostrable, como H_2 y H'_2 son equivalentes de manera demostrable, donde H'_1 y H'_2 son fórmulas obtenidas a partir de H_1 y H_2 al reemplazar una ocurrencia de F por G . Queremos verificar, en cada uno de los cinco casos, que H y H' son equivalentes de manera demostrable. Para ello, recurrimos a las reglas de las Tablas 1.5 y 1.6 (mientras que en la demostración del Teorema 1.2 recurrimos a la semántica de la lógica proposicional). Dejamos los detalles de esta demostración como Ejercicio 1.28. ■

La palabra "inducción" indica que estamos razonando de un caso particular al caso general. Las demostraciones por inducción involucran dos pasos y concluyen que cierto enunciado se cumple, en general, para todos los números naturales o para todas las fórmulas. Estos dos pasos se llaman el "paso base" y el "paso inductivo." En la inducción matemática, el paso base es el paso en el que mostramos que el enunciado es verdadero para $n = 1$. Si estamos usando inducción sobre la complejidad de las fórmulas, entonces el paso base es el paso en el que verificamos que el enunciado se cumple para todas las fórmulas atómicas.

El paso inductivo de la inducción matemática es el paso en el que mostramos que, si el enunciado es verdadero para $n = m$, entonces también es verdadero para $n = m + 1$. El paso inductivo de la inducción sobre la complejidad de las fórmulas comprende cinco casos correspondientes a $\neg$, $\wedge$, $\vee$, $\rightarrow$ y $\leftrightarrow$. Obsérvese que, en la demostración del Teorema 1.2, los casos correspondientes a $\vee$, $\rightarrow$ y $\leftrightarrow$ se siguieron rápidamente de los casos relativos a $\neg$ y $\wedge$. Esto se debe a que $\vee$, $\rightarrow$ y $\leftrightarrow$ se definieron en función de $\neg$ y $\wedge$. Esto sugiere una forma alternativa para el paso inductivo, que describimos a continuación.

Supongamos que queremos mostrar que cierta propiedad P se cumple para todas las fórmulas de la lógica proposicional. Para hacer esto por inducción sobre la complejidad de las fórmulas, primero mostramos que P se cumple para todas las fórmulas atómicas (el paso base). Para el paso inductivo, en lugar de verificar los cinco casos anteriores, a veces podemos hacer solo tres casos. Primero mostramos que P se preserva bajo equivalencia. Es decir, mostramos que si $F \equiv G$ y G posee la propiedad P , entonces también la posee F . Si esto es cierto, entonces solo necesitamos considerar los casos correspondientes a $\neg$ y $\wedge$. Esto basta porque toda fórmula de la lógica proposicional es equivalente a una fórmula que usa solamente $\neg$ y $\wedge$ (y ni $\vee$, ni $\rightarrow$, ni $\leftrightarrow$). Mostramos esta versión del paso inductivo en la siguiente sección, donde demostramos que toda fórmula de la lógica proposicional es equivalente a una fórmula que está en forma normal conjuntiva.

1.6 Formas normales

En el Ejemplo 1.13 mostramos que la fórmula $((C \wedge D) \vee A) \wedge ((C \wedge D) \vee B) \wedge (E \vee \neg E)$ es equivalente a la fórmula $(A \wedge B) \vee (C \wedge D)$, que es una disyunción de dos conjunciones. En esta sección mostramos que no hay nada especial en $((C \wedge D) \vee A) \wedge ((C \wedge D) \vee B) \wedge (E \vee \neg E)$. Toda fórmula de la lógica proposicional es equivalente a una fórmula que es una disyunción de conjunciones. Comenzamos con algunas definiciones.

Definición 1.14: Literal

Un *literal* es una fórmula atómica o la negación de una fórmula atómica, y nos referimos a estos como positivos o negativos, respectivamente.

Ejemplo 1.23

Si A es una fórmula atómica, entonces A es un literal positivo y $\neg A$ es un literal negativo. ▲

Definición 1.15: Forma normal conjuntiva

Una fórmula F está en *forma normal conjuntiva* (FNC) si es una conjunción de disyunciones de literales. Es decir,

$$F = \bigwedge_{i=1}^n \left(\bigvee_{j=1}^m L_{i,j} \right)$$

donde cada $L_{i,j}$ es atómica o es la negación de una fórmula atómica.

Definición 1.16: Forma normal disyuntiva

Una fórmula F está en *forma normal disyuntiva* (FND) si es una disyunción de conjunciones de literales. Es decir,

$$F = \bigvee_{i=1}^n \left(\bigwedge_{j=1}^m L_{i,j} \right)$$

donde cada $L_{i,j}$ es atómica o es la negación de una fórmula atómica.

Ejemplo 1.24

$(A \vee B) \wedge (C \vee D) \wedge (\neg A \vee \neg B \vee \neg D)$ está en FNC,

$(\neg A \wedge B) \vee C \vee (B \wedge \neg C \wedge D)$ está en FND, y

$(A \vee B) \wedge ((A \wedge C) \vee (B \wedge D))$ no está ni en FNC ni en FND.

**Lema 1.1**

Sea F una fórmula en FNC y G una fórmula en FND. Entonces $\neg F$ es equivalente a una fórmula en FND y $\neg G$ es equivalente a una fórmula en FNC.

Demostración. Si F está en FNC, entonces F es la fórmula

$$\bigwedge_{i=1}^n \left(\bigvee_{j=1}^m L_{i,j} \right)$$

para ciertos literales $L_{i,j}$. La negación de esta fórmula,

$$\neg F = \neg \left(\bigwedge_{i=1}^n \left(\bigvee_{j=1}^m L_{i,j} \right) \right),$$

es equivalente, por la Proposición 1.5, a

$$\bigvee_{i=1}^n \neg \left(\bigvee_{j=1}^m L_{i,j} \right).$$

Asimismo, por la misma proposición, esto es equivalente a

$$\bigvee_{i=1}^n \left(\bigwedge_{j=1}^m \neg L_{i,j} \right).$$

Esta fórmula está en FND y es equivalente a $\neg F$.

De manera similar, usando la Proposición 1.5 dos veces, podemos demostrar que $\neg G$ es equivalente a una fórmula en FNC. ■

Teorema 1.4

Toda fórmula F es equivalente a alguna fórmula F_1 en FNC y a alguna fórmula F_2 en FND.

Demostración. Demostramos esto por inducción sobre la complejidad de F .

Primero supongamos que F es atómica. Entonces F ya está tanto en FNC como en FND. Así, podemos tomar $F_1 = F_2 = F$.

Nuestra hipótesis de inducción es que la conclusión del teorema se cumple para las fórmulas G y H . Es decir, suponemos que existen fórmulas H_1 y G_1 en FNC y H_2 y G_2 en FND tales que $H \equiv H_1 \equiv H_2$ y $G \equiv G_1 \equiv G_2$.

La propiedad de ser equivalente a fórmulas en FNC y en FND se preserva claramente bajo equivalencia. Si $F \equiv G$, entonces, por nuestra hipótesis de inducción, podemos simplemente tomar $F_1 = G_1$ y $F_2 = G_2$. Por lo tanto, basta verificar solo dos casos más, correspondientes a $\neg$ y $\wedge$.

Supongamos primero que F tiene la forma $\neg G$. Entonces $F \equiv \neg G_1 \equiv \neg G_2$. Dado que G_1 está en FNC, $\neg G_1$ es equivalente a una fórmula G_3 en FND por el Lema 1.1. De igual manera, $\neg G_2$ es equivalente a una fórmula G_4 en FNC. Así, podemos tomar $F_1 = G_4$ y $F_2 = G_3$.

Supongamos ahora que F tiene la forma $G \wedge H$. Entonces $F \equiv G_1 \wedge H_1$ por sustitución (Teorema 1.2). Dado que G_1 y H_1 están ambas en FNC, también lo está su conjunción.

Resta mostrar que $F = G \wedge H$ es equivalente a una fórmula en FND. De nuevo usando el Teorema 1.2, $F \equiv G_2 \wedge H_2$. Dado que cada una de estas fórmulas está en FND, pueden escribirse de la siguiente manera:

$$G_2 = \bigvee_i M_i \quad \text{y} \quad H_2 = \bigvee_j N_j$$

donde cada M_i y N_j es una conjunción de literales. Tenemos entonces

$$F \equiv \left(\bigvee_i M_i \right) \wedge \left(\bigvee_j N_j \right).$$

Usando la segunda equivalencia de la Proposición 1.6, tenemos

$$F \equiv \bigvee_i \bigvee_j (M_i \wedge N_j)$$

que es una disyunción de conjunciones de literales, como se requería. ■

Dada una fórmula F , el teorema anterior garantiza la existencia de una fórmula en FND que sea equivalente a F . Supongamos que queremos encontrar tal fórmula. Una manera de hacerlo es calcular una tabla de verdad para F . Por ejemplo, supongamos que F tiene la siguiente tabla de verdad.

A	B	F
0	0	1
0	1	0
1	0	1
1	1	0

Entonces F es verdadera bajo la asignación $\mathcal{A}$ si y solo si $\mathcal{A}$ corresponde a la fila 1 o a la fila 3 de la tabla. Esto conduce a una fórmula en FND. F es verdadera si y solo si, o bien A y B son ambas falsas (fila 1), O BIEN A es verdadera y B es falsa (fila 3). Así, F es equivalente a $(\neg A \wedge \neg B) \vee (A \wedge \neg B)$, que está en FND.

Del mismo modo, considerando las filas en las que F es falsa, podemos hallar una fórmula equivalente en FNC. F es verdadera si y solo si no estamos en la fila 2 Y no estamos en la fila 4. Es decir, F es verdadera si y solo si se cumple A o $\neg B$ (NO fila 2) Y se cumple $\neg A$ o $\neg B$ (NO fila 4). Así, F es equivalente a $(A \vee \neg B) \wedge (\neg A \vee \neg B)$, que está en FNC.

Esto, de hecho, proporciona una demostración alternativa del Teorema 1.4. Dada cualquier fórmula F , podemos usar una tabla de verdad para hallar fórmulas equivalentes en FNC y en FND. Una manera alternativa de hallar una fórmula en FNC equivalente a F la proporciona el siguiente algoritmo. Este algoritmo es a menudo, aunque no siempre, más eficiente que calcular una tabla de verdad.

Algoritmo FNC

Paso 1: Reemplazamos todas las subfórmulas de la forma $F \rightarrow G$ por $(\neg F \vee G)$ y todas las subfórmulas de la forma $F \leftrightarrow G$ por $(\neg F \vee G) \wedge (\neg G \vee F)$. Cuando ya no haya ocurrencias de $\rightarrow$ ni de $\leftrightarrow$, pasamos al Paso 2.

Paso 2: Eliminamos todas las dobles negaciones y aplicamos las Leyes de De Morgan donde sea posible. Es decir, reemplazamos todas las subfórmulas de la forma

$$\begin{aligned} &\neg\neg G \text{ por } G, \\ &\neg(G \wedge H) \text{ por } (\neg G \vee \neg H), \text{ y} \\ &\neg(G \vee H) \text{ por } (\neg G \wedge \neg H). \end{aligned}$$

Cuando ya no haya subfórmulas de estas formas, pasamos al Paso 3.

Paso 3: Aplicamos la regla de distributividad de $\vee$ donde sea posible. Es decir, reemplazamos todas las subfórmulas de la forma

$$(G \vee (H \wedge K)) \text{ o } ((H \wedge K) \vee G) \text{ por } ((G \vee H) \wedge (G \vee K)).$$

Si eliminamos de nuestra fórmula estas subfórmulas, entonces nos queda una fórmula en FNC. Si cambiamos el Paso 3 por la distributividad para $\wedge$, obtendríamos una fórmula en FND.

Ejemplo 1.25

Mostramos el Algoritmo FNC con

$$F = (A \vee B) \rightarrow (\neg B \wedge A).$$

En el Paso 1, eliminamos $\rightarrow$, reescribiendo la fórmula como

$$\neg(A \vee B) \vee (\neg B \wedge A).$$

En el Paso 2, aplicamos la Ley de De Morgan para obtener

$$(\neg A \wedge \neg B) \vee (\neg B \wedge A).$$

Pasando al Paso 3, vemos que la fórmula del Paso 2 está en FND. En particular, tiene la forma $(G \vee (H \wedge K))$ (tomando $G = (\neg A \wedge \neg B)$). Por distributividad, obtenemos

$$((\neg A \wedge \neg B) \vee \neg B) \wedge ((\neg A \wedge \neg B) \vee A).$$

Todavía tenemos dos $\vee$ que deben distribuirse:

$$(\neg A \vee \neg B) \wedge (\neg B \vee \neg B) \wedge (\neg A \vee A) \wedge (\neg B \vee A).$$

Ahora ya no hay subfórmulas de la forma $(G \vee (H \wedge K))$ o $((H \wedge K) \vee G)$, así que hemos terminado con el Paso 3. Vemos que hemos obtenido una fórmula en FNC, tal como se prometió.

Esta fórmula no está escrita de la mejor manera. Dado que $(\neg A \vee A)$ es una tautología, la fórmula anterior es equivalente a $(\neg A \vee \neg B) \wedge (\neg B) \wedge (\neg B \vee A)$, que es equivalente a $(A \vee \neg B) \wedge (\neg A \vee \neg B)$. Obsérvese que esta es la misma fórmula que obtuvimos a partir de la tabla de verdad tras la demostración del Teorema 1.4. ▲

Inspeccionando el Algoritmo FNC, vemos que el Teorema 1.4 puede fortalecerse. Este teorema afirma que, para cualquier fórmula F , existen fórmulas F_1 en FNC y F_2 en FND que son equivalentes a F . Afirmamos ahora que F_1 y F_2 son equivalentes de manera demostrable a F . Para verlo, consideremos el algoritmo paso a paso. En cada paso reemplazamos ciertas subfórmulas por fórmulas equivalentes. En cada caso podemos demostrar formalmente la equivalencia. Por conveniencia, usamos la notación $F \Vdash G$ para abreviar " F y G son equivalentes de manera demostrable."

Paso 1:

$F \rightarrow G \Vdash (\neg F \vee G)$ por $\rightarrow$ -Definición

$F \leftrightarrow G \Vdash (\neg F \vee G) \wedge (\neg G \vee F)$ por $\leftrightarrow$ -Definición y $\rightarrow$ -Definición.

Paso 2:

$\neg\neg G \Vdash G$ por Doble negación y el Ejemplo 1.22.

$\neg(G \wedge H) \Vdash (\neg G \vee \neg H)$ por la Proposición 1.2 (Leyes de De Morgan).

$\neg(G \vee H) \Vdash (\neg G \wedge \neg H)$ por la Proposición 1.2 (Leyes de De Morgan).

Paso 3:

$(G \vee (H \wedge K)) \Vdash ((G \vee H) \wedge (G \vee K))$ por la Proposición 1.4 (Distributividad de $\vee$).

$((H \wedge K) \vee G) \Vdash ((G \vee H) \wedge (G \vee K))$ por $\vee$ -Simetría y la Proposición 1.4.

Por el Teorema 1.3, el resultado F_1 de este algoritmo es equivalente de manera demostrable a F . De igual manera, F_2 y F son equivalentes de manera demostrable. Registramos este fortalecimiento del Teorema 1.4 de la siguiente manera.

Proposición 1.7

Para toda fórmula F existen fórmulas F_1 en FNC y F_2 en FND tales que F , F_1 y F_2 son equivalentes de manera demostrable.

1.7 Fórmulas de Horn

Una fórmula de Horn es un tipo de fórmula en FNC particularmente agradable. Existe un método rápido para determinar si una fórmula de Horn es satisfacible o no. Discutimos tanto este método como lo que se entiende por "rápido."

Definición 1.17: Fórmula de Horn

Una fórmula F es una *fórmula de Horn* si está en FNC y cada disyunción contiene a lo sumo un literal positivo.

Claramente, la conjunción de dos fórmulas de Horn es de nuevo una fórmula de Horn. Esto no es cierto para las disyunciones.

Ejemplo 1.26

La fórmula $A \wedge (\neg A \vee \neg B \vee C) \wedge (\neg B \vee D) \wedge (\neg C \vee \neg D)$ es una fórmula de Horn. La fórmula $A \vee B$ no es una fórmula de Horn. ▲

Una fórmula de Horn básica es una fórmula de Horn que no usa $\wedge$. Por ejemplo, $(\neg A \vee \neg B \vee C)$, A , y $(\neg B \vee \neg D)$ son fórmulas de Horn básicas. Toda fórmula de Horn es una conjunción de fórmulas de Horn básicas.

Hay tres tipos de fórmulas de Horn básicas: aquellas que no contienen ningún literal positivo (como $(\neg B \vee \neg D)$), aquellas que no contienen ningún literal negativo (como A), y aquellas que contienen

tanto un literal positivo como literales negativos (como $(\neg A \vee \neg B \vee C)$). Si una fórmula de Horn básica contiene tanto literales positivos como negativos, entonces puede escribirse como una implicación que involucra únicamente literales positivos. Por ejemplo, $(\neg A \vee \neg B \vee C)$ es equivalente a $(A \wedge B) \rightarrow C$. Si una fórmula de Horn básica no contiene ningún literal positivo, entonces puede escribirse como una implicación que involucra una contradicción. Por ejemplo, si $\perp$ es una contradicción, entonces $(\neg B \vee \neg D)$ es equivalente a $(B \wedge D) \rightarrow \perp$. En caso contrario, si una fórmula de Horn básica no contiene ningún literal negativo, entonces es una fórmula atómica. También podemos escribir esto como una implicación si lo deseamos. La fórmula atómica A es equivalente a $T \rightarrow A$, donde T es una tautología. De esta manera, toda fórmula de Horn básica puede escribirse como una implicación y toda fórmula de Horn puede escribirse como una conjunción de implicaciones.

Ejemplo 1.27

La fórmula de Horn del Ejemplo 1.26 puede escribirse de la siguiente manera:

$$(T \rightarrow A) \wedge ((A \wedge B) \rightarrow C) \wedge (B \rightarrow D) \wedge ((C \wedge D) \rightarrow \perp).$$

▲

Supongamos que se nos da una fórmula de Horn H y queremos decidir si es satisfacible o no. Nos referimos a este problema de decisión como el *problema de satisfacibilidad de Horn*. A diferencia de los otros problemas de decisión que hemos visto, existe un algoritmo eficiente para resolver el problema de satisfacibilidad de Horn. Hay tres pasos en este algoritmo, correspondientes a los tres tipos de fórmulas de Horn básicas. Suponemos que la fórmula de Horn se ha dado como una conjunción de implicaciones.

El algoritmo de Horn

Dada una fórmula de Horn H escrita como una conjunción de implicaciones, enumeramos las fórmulas atómicas que ocurren en H .

Paso 1: Marcamos cada fórmula atómica A de la lista que aparezca en una subfórmula de la forma $(T \rightarrow A)$.

Paso 2: Si existe una subfórmula de la forma $(A_1 \wedge A_2 \wedge \cdots \wedge A_m) \rightarrow C$ donde cada A_i ha sido marcada y C no ha sido marcada, marcamos C . Repetimos este paso hasta que no haya subfórmulas de esta forma y luego pasamos al Paso 3.

Paso 3: Consideramos las subfórmulas de la forma $(A_1 \wedge A_2 \wedge \cdots \wedge A_m) \rightarrow \perp$. Si existe tal subfórmula donde cada A_i ha sido marcada, concluimos "No, H no es satisfacible." En caso contrario, concluimos "Sí, H es satisfacible."

Ejemplo 1.28

Mostramos el algoritmo de Horn. Sea H la fórmula

$$(T \rightarrow A) \wedge (C \rightarrow D) \wedge ((A \wedge B) \rightarrow C) \wedge ((C \wedge D) \rightarrow \perp) \wedge (T \rightarrow B).$$

Las subfórmulas atómicas de H son A , B , C y D .

En el Paso 1 del algoritmo, dado que H tiene las subfórmulas $(T \rightarrow A)$ y $(T \rightarrow B)$, marcamos tanto A como B .

En el Paso 2, dado que H tiene la subfórmula $(A \wedge B) \rightarrow C$, marcamos C . Ahora que C ha sido marcada, también debemos marcar D por la subfórmula $(C \rightarrow D)$.

En el Paso 3, dado que H tiene la subfórmula $(C \wedge D) \rightarrow \perp$, el algoritmo concluye "No, H no es satisfacible."

Obsérvese que, para la fórmula de Horn del Ejemplo 1.27, el algoritmo de Horn arroja una conclusión distinta. ▲

Queremos mostrar que, para cualquier fórmula de Horn dada, el algoritmo de Horn funciona rápidamente. Primero mostramos que funciona.

Proposición 1.8

El algoritmo de Horn concluye "Sí, H es satisfacible" si y solo si H es satisfacible.

Demostración. Sea $\mathcal{S} = \{C_1, C_2, \dots, C_n\}$ el conjunto de fórmulas atómicas que ocurren en H . Al concluir el algoritmo, algunas de estas fórmulas atómicas han sido marcadas.

Supongamos que H es satisfacible. Entonces existe una asignación $\mathcal{A}$ de $\mathcal{S}$ tal que $\mathcal{A} \models H$. Para cada subfórmula de Horn básica B de H , $\mathcal{A}(B) = 1$. Si B tiene la forma $(T \rightarrow C_i)$, entonces $\mathcal{A}(C_i) = 1$. Si B tiene la forma $(C_1 \wedge C_2 \wedge \dots \wedge C_m) \rightarrow D$ donde cada $\mathcal{A}(C_i) = 1$, entonces $\mathcal{A}(D)$ también es igual a 1. Se sigue que $\mathcal{A}(C_i) = 1$ para cada C_i que haya sido marcada.

Supongamos, por contradicción, que el algoritmo concluye "No, H no es satisfacible." Esto solo ocurre si existe una subfórmula B de la forma $(A_1 \wedge A_2 \wedge \dots \wedge A_m) \rightarrow \perp$ donde cada A_i ha sido marcada. Dado que cada A_i ha sido marcada, $\mathcal{A}(A_i) = 1$ para cada A_i . Por la semántica de $\rightarrow$ (Tabla 1.4), tenemos $\mathcal{A}(B) = 0$, lo cual es una contradicción. Así, si H es satisfacible, entonces el algoritmo concluye "Sí, H es satisfacible."

Recíprocamente, supongamos que el algoritmo concluye "Sí, H es satisfacible." Sea $\mathcal{A}_0$ la asignación de $\mathcal{S}$ definida por $\mathcal{A}_0(C_i) = 1$ si y solo si C_i está marcada. Afirmamos que $\mathcal{A}_0 \models H$. Basta mostrar que $\mathcal{A}_0$ modela cada subfórmula de Horn básica de H .

Sea B una fórmula de Horn básica que es subfórmula de H . Si B tiene la forma $(T \rightarrow A)$, entonces A se marca en el Paso 1 del algoritmo y así $\mathcal{A}_0(B) = 1$. En caso contrario, B tiene la forma $(A_1 \wedge A_2 \wedge \dots \wedge A_n) \rightarrow G$ donde G es una fórmula atómica o una contradicción $\perp$. Si $\mathcal{A}_0(A_i) = 0$ para algún i , entonces $\mathcal{A}_0(B) = 1$. Supongamos entonces que $\mathcal{A}_0$ modela cada A_i . Entonces cada A_i ha sido marcada. Dado que el algoritmo concluyó "Sí," G no es $\perp$. Así, G es una fórmula atómica. Dado que cada A_i está marcada, G también está marcada (Paso 2 del algoritmo). Dado que $\mathcal{A}_0(G) = 1$, tenemos $\mathcal{A}_0(B) = 1$. ■

Así, el algoritmo de Horn funciona. Dada cualquier fórmula de Horn H , el algoritmo determina correctamente si H es satisfacible o no. Consideremos ahora la siguiente pregunta. ¿Cuántos pasos le toma al algoritmo de Horn llegar a una conclusión? La respuesta depende de la longitud de la entrada H . Supongamos que la fórmula H es una cadena de n símbolos, donde n es algún número natural grande. Afirmamos que el algoritmo de Horn concluye en menos de n^2 pasos.

Para verificar esta afirmación, contamos el número de pasos del algoritmo de Horn. Pero, ¿qué se entiende exactamente por un "paso"? Mirando el algoritmo, vemos que hay tres pasos llamados Paso 1, Paso 2 y Paso 3. No es esto lo que se quiere decir. Quizá tengamos que repetir el Paso 2 más de una vez, en cuyo caso se necesitarán más de tres pasos para llegar a una respuesta de "sí" o "no." Definimos con precisión qué constituye un "paso de un algoritmo" en el Capítulo 7. Por el momento, contemos el número de veces que debemos leer la entrada H .

Primero leemos la fórmula H símbolo por símbolo de izquierda a derecha y enumeramos todas sus subfórmulas atómicas. Dado que H contiene n símbolos, hay a lo sumo n fórmulas atómicas en nuestra lista. Luego, en el Paso 1, volvemos a leer H , esta vez buscando cualquier ocurrencia de la tautología T . Marcamos las fórmulas atómicas correspondientes. En el Paso 2, buscamos subfórmulas de la forma $(A_1 \wedge A_2 \wedge \cdots \wedge A_m) \rightarrow C$ donde cada A_i ha sido marcada. Si encontramos tal subfórmula donde C no ha sido marcada, entonces marcamos C . Habiendo marcado una nueva fórmula atómica, podemos haber creado nuevas subfórmulas de la forma $(A_1 \wedge A_2 \wedge \cdots \wedge A_m) \rightarrow C$ donde cada A_i ha sido marcada. Cada vez que marcamos una fórmula en el Paso 2, debemos volver a leer H . Dado que podemos marcar a lo sumo n fórmulas atómicas, debemos repetir el Paso 2 no más de n veces. Finalmente, en el Paso 3, debemos leer H una vez más (buscando $\perp$) para llegar a la conclusión. En total, debemos leer H a lo sumo $1 + 1 + n + 1 = n + 3$ veces para llegar a una conclusión. Dado que $n^2 > n + 3$ para $n > 2$, esto verifica nuestra afirmación.

Definición 1.18

Un algoritmo es de *tiempo polinomial* si existe un polinomio $p(x)$ tal que, dada una entrada de tamaño n , el algoritmo se detiene en menos de $p(n)$ pasos.

La clase de todos los problemas de decisión que pueden resolverse mediante algún algoritmo de tiempo polinomial se denota por P .

Si un algoritmo no es de tiempo polinomial, entonces, según cualquier criterio, no es rápido. La discusión anterior muestra que el algoritmo de Horn es de tiempo polinomial y, por lo tanto, el problema de satisfacibilidad de Horn está en P . En contraste, consideremos los siguientes problemas de decisión.

Problema de validez: Dada una fórmula F , ¿es F válida?

Problema de satisfacibilidad: Dada una fórmula F , ¿es F satisfacible?

Problema de consecuencia: Dadas las fórmulas F y G , ¿es G consecuencia de F ?

Problema de equivalencia: Dadas las fórmulas F y G , ¿son F y G equivalentes?

En cierto sentido, estos cuatro problemas son en realidad el mismo. Cualquier algoritmo que funcione para uno de estos problemas también funciona para todos ellos. Si tuviéramos un algoritmo para el problema de validez, por ejemplo, podríamos usarlo para resolver el problema de satisfacibilidad, dado que F es satisfacible si y solo si $\neg F$ no es válida. De manera similar, cualquier algoritmo para el problema de satisfacibilidad puede usarse para el problema de consecuencia, dado que G es consecuencia de F si y solo si $\neg(F \rightarrow G)$ no es satisfacible. Claramente, cualquier algoritmo para el problema de consecuencia puede usarse (dos veces) para resolver el problema de equivalencia.

Finalmente, dado un algoritmo que decide el problema de equivalencia, podemos verificar si F es equivalente a una tautología conocida T para resolver el problema de validez. En particular, si uno de estos cuatro problemas está en P , entonces los cuatro lo están.

Las tablas de verdad proporcionan un algoritmo para resolver cada uno de estos problemas. Para el problema de satisfacibilidad, primero calculamos una tabla de verdad para F y luego verificamos si su valor de verdad es alguna vez 1. Este algoritmo ciertamente funciona, pero, ¿cuántos pasos toma? Calcular la tabla de verdad no es un solo paso. De nuevo, contamos cuántas veces se nos exige leer la entrada F . Si F tiene n fórmulas atómicas, entonces la tabla de verdad de F tiene 2^n filas. Debemos consultar F para calcular cada una de estas filas. Así, debemos leer la entrada al menos 2^n veces. Esto es exponencial y no un polinomio. Dado cualquier polinomio $p(x)$, 2^n es mayor que $p(n)$ para valores de n suficientemente grandes. Así, este algoritmo no es de tiempo polinomial.

No se sabe si el problema de satisfacibilidad (y los otros tres problemas de decisión) está en P . No conocemos ningún algoritmo de tiempo polinomial para la satisfacibilidad, pero esto no significa que no exista alguno. Si alguien pudiera encontrar tal algoritmo, o demostrar que no existe tal algoritmo, esto respondería una de las preguntas abiertas más famosas de las matemáticas: la pregunta $P = NP$. Definiremos NP y discutiremos este problema en el Capítulo 7. Por ahora, simplemente señalamos que no presentamos un algoritmo eficiente para el problema de satisfacibilidad, y probablemente no exista tal algoritmo.

Sin embargo, presentamos un algoritmo que es una alternativa a las tablas de verdad para el problema de satisfacibilidad. Las demostraciones formales evitan las tablas de verdad, pero no siempre resuelven este problema de decisión. Dada una fórmula F , podemos usar demostraciones formales para mostrar que F es insatisfacible (demostrando que $\emptyset \vdash \neg F$), pero no podemos mostrar que F es satisfacible. De igual manera, las demostraciones formales pueden establecer que una fórmula es válida, o que una fórmula es consecuencia de otra, pero no pueden mostrar que una fórmula no es válida ni que no es consecuencia de otra. Si hallamos una demostración formal de $\{F\} \vdash G$, entonces podemos concluir legítimamente "sí, G es consecuencia de F ." Pero si G no es consecuencia de F , entonces buscaremos por siempre en vano una demostración y nunca llegaremos a una conclusión. En la siguiente sección presentamos la *resolución*, un refinamiento de las demostraciones formales que sí proporciona un algoritmo (aunque no de tiempo polinomial) para estos problemas de decisión.

1.8 Resolución

La resolución es un sistema de demostración formal que involucra un número mínimo de reglas. Una de las reglas es una variante de la regla del corte. Esta regla afirma que, a partir de las fórmulas $(F \rightarrow G)$ y $(G \rightarrow H)$, podemos deducir la fórmula $(F \rightarrow H)$. Otra regla es una variante de la regla de sustitución, enunciada de la siguiente manera.

Sea H una fórmula que contiene a F como subfórmula. Si $G \equiv F$, entonces podemos deducir H' a partir de H , donde H' es la fórmula obtenida al reemplazar alguna ocurrencia de F en H por G .

Es decir, consideramos el Teorema 1.2 como una regla de deducción. En realidad esto son muchas reglas en una, así que en cierto modo hacemos trampa para tener pocas reglas. En particular, para cualquier par de fórmulas equivalentes F y G , podemos deducir G a partir de F . Podría parecer

que esto frustra uno de nuestros propósitos: el problema de equivalencia. Sin embargo, la regla de sustitución puede relajarse un poco. El propósito principal de esta regla es llevar las fórmulas a FNC. La esencia de la resolución es que, una vez que las fórmulas están en FNC, solo necesitamos dos reglas para deducirlo todo. Esto proporcionará un algoritmo para el problema de equivalencia y los demás problemas de decisión de la sección anterior. También nos acerca un paso más a la demostración del Teorema de completitud para la lógica proposicional.

1.8.1. Cláusulas

Supongamos que F es una fórmula en FNC. Entonces F es una conjunción de disyunciones de literales. Nos referimos a una disyunción de literales como una *cláusula*. Por conveniencia, escribimos cada cláusula como un conjunto. Consideramos

$$L_1 \vee L_2 \vee \cdots \vee L_n \quad \text{como el conjunto} \quad \{L_1, L_2, \dots, L_n\}.$$

Cualquier fórmula que sea una disyunción de literales determina de manera única tal conjunto. Sin embargo, el conjunto no determina de manera única a la fórmula. Recordemos que dos conjuntos son iguales si y solo si contienen los mismos elementos. El orden y la repetición no importan. Por ejemplo, las fórmulas $(L_1 \vee L_2)$, $(L_2 \vee L_1)$, y $(L_1 \vee L_2 \vee L_2)$ dan lugar cada una al mismo conjunto $\{L_1, L_2\}$. Aunque estas fórmulas no son idénticas, son equivalentes.

Proposición 1.9

Sean C y D cláusulas. Si C y D son iguales al verse como conjuntos, entonces $C \equiv D$.

Demostración. Sea S el conjunto de literales que ocurren en C . Tanto C como D son equivalentes a la disyunción de los literales de S . ■

Si F está en FNC, entonces F es una conjunción de cláusulas y podemos escribir F como un conjunto de conjuntos. Consideramos a F como el conjunto $\{C_1, \dots, C_n\}$ donde las C_i son las cláusulas que ocurren en F (escritas como conjuntos). Por ejemplo, consideramos la fórmula $(A \vee B \vee \neg C) \wedge (C \vee D) \wedge \neg A \wedge (\neg B \vee \neg D)$ como el siguiente conjunto de cuatro cláusulas

$$\{\{A, B, \neg C\}, \{C, D\}, \{\neg A\}, \{\neg B, \neg D\}\}.$$

Proposición 1.10

Sean F y G dos fórmulas en FNC. Si F y G son iguales al verse como conjuntos, entonces $F \equiv G$.

Demostración. Sea C el conjunto de cláusulas que ocurren en F . Tanto F como G son equivalentes a la conjunción de las cláusulas de C . Esta proposición se sigue entonces de la Proposición 1.9. ■

A lo largo de esta sección, consideramos cualquier fórmula en FNC tanto como fórmula como conjunto de cláusulas. Si F y G son fórmulas en FNC, entonces su conjunción puede escribirse ya sea como la fórmula $F \wedge G$ o como el conjunto $F \cup G$. Por la proposición anterior, no hay ambigüedad en considerar a F tanto como conjunto como fórmula. Sin embargo, hacemos hincapié en que ver las fórmulas como conjuntos solo tiene sentido para fórmulas en FNC. En particular, no existe una contraparte agradable, en términos de teoría de conjuntos, para la disyunción o la negación. Las fórmulas $F \vee G$ y $\neg F$ no están en FNC y no pueden verse como conjuntos de cláusulas.

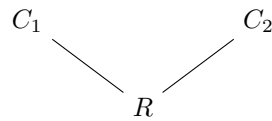
1.8.2. Resolventes

Dada una fórmula en FNC, la resolución usa repetidamente dos reglas para determinar si la fórmula es satisfacible o no. Una de estas reglas afirma que cualquier cláusula de F puede deducirse de F . La otra regla involucra el *resolvente* de dos cláusulas. Definimos ahora esta noción.

Definición 1.19: Resolvente

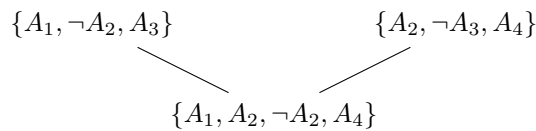
Sean C_1 y C_2 dos cláusulas. Supongamos que $A \in C_1$ y $\neg A \in C_2$ para alguna fórmula atómica A . Entonces la cláusula $R = (C_1 - \{A\}) \cup (C_2 - \{\neg A\})$ es un *resolvente* de C_1 y C_2 .

Representamos esta situación gráficamente mediante el siguiente diagrama:



Ejemplo 1.29

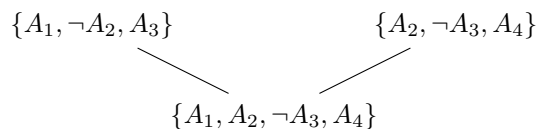
Sean $C_1 = \{A_1, \neg A_2, A_3\}$ y $C_2 = \{A_2, \neg A_3, A_4\}$. Dado que $A_3 \in C_1$ y $\neg A_3 \in C_2$, podemos hallar un resolvente.



▲

Ejemplo 1.30

El resolvente de dos cláusulas no es necesariamente único. En el ejemplo anterior, dado que $\neg A_2 \in C_1$ y $A_2 \in C_2$, también tenemos





Enumeramos ahora las tres reglas de deducción que se usan en la resolución.

- Sea G cualquier fórmula. Sea F la fórmula en FNC resultante de aplicar el Algoritmo FNC a G . Entonces F puede deducirse de G .
- Sea F una fórmula en FNC. Cualquier cláusula de F puede deducirse de F .
- Sea F una fórmula en FNC. Cualquier resolvente de dos cláusulas de F puede deducirse de F .

Sorprendentemente, estas tres reglas bastan para la lógica proposicional. La resolución es completa. Antes de demostrar este hecho, debemos verificar que estas reglas son correctas. Mostramos algo más fuerte. Mostramos que cada una de estas reglas puede deducirse usando demostraciones formales. En la primera regla, F puede deducirse de G por la Proposición 1.7. Si C es una cláusula de F , entonces podemos deducir C a partir de F usando $\wedge$ -Simetría y $\wedge$ -Eliminación.

Resta mostrar que R puede deducirse de F donde R es un resolvente de dos cláusulas de F . Obsérvese la similitud entre esto y la regla del corte. Sean C_1 y C_2 como en el Ejemplo 1.29. Entonces C_1 es equivalente a $(\neg A_1 \wedge A_2) \rightarrow A_3$ y C_2 es equivalente a $A_3 \rightarrow (A_2 \vee A_4)$. La regla del corte afirma que, a partir de estas fórmulas, podemos deducir la fórmula $(\neg A_1 \wedge A_2) \rightarrow (A_2 \vee A_4)$. Esta fórmula es equivalente al resolvente obtenido en el Ejemplo 1.29.

Proposición 1.11

Sean C_1 y C_2 cláusulas y sea R un resolvente de C_1 y C_2 . Entonces $\{C_1, C_2\} \vdash R$.

Demostración. Dado que C_1 y C_2 tienen un resolvente, debe existir una fórmula atómica A tal que A está en una de estas cláusulas y $\neg A$ está en la otra. Sin pérdida de generalidad, podemos suponer que A está en C_1 y $\neg A$ está en C_2 . Así, C_1 es equivalente a $(A \vee F)$ para alguna cláusula F y C_2 es equivalente a $(\neg A \vee G)$ para alguna cláusula G . La fórmula $(F \vee G)$ es un resolvente de C_1 y C_2 . Podemos suponer que R es este resolvente. Damos una demostración formal de $\{C_1, C_2\} \vdash R$.

Premisa: $\mathcal{F} \vdash (A \vee F)$ y $\mathcal{F} \vdash (\neg A \vee G)$

Conclusión: $\mathcal{F} \vdash (F \vee G)$

Afirmación	Justificación
1. $\mathcal{F} \vdash (A \vee F)$	Premisa
2. $\mathcal{F} \cup \{\neg A\} \vdash (A \vee F)$	Monotonicidad aplicada a 1
3. $\mathcal{F} \cup \{\neg A\} \vdash \neg A$	Suposición
4. $\mathcal{F} \cup \{\neg A\} \vdash F$	$\vee$ -Eliminación aplicada a 2 y 3
5. $\mathcal{F} \cup \{\neg A\} \vdash (F \vee G)$	$\vee$ -Introducción aplicada a 4
6. $\mathcal{F} \vdash (\neg A \vee G)$	Premisa
7. $\mathcal{F} \cup \{\neg \neg A\} \vdash (\neg A \vee G)$	Monotonicidad aplicada a 6
8. $\mathcal{F} \cup \{\neg \neg A\} \vdash \neg \neg A$	Suposición
9. $\mathcal{F} \cup \{\neg \neg A\} \vdash G$	$\vee$ -Eliminación aplicada a 7 y 8
10. $\mathcal{F} \cup \{\neg \neg A\} \vdash (G \vee F)$	$\vee$ -Introducción aplicada a 9
11. $\mathcal{F} \cup \{\neg \neg A\} \vdash (F \vee G)$	$\vee$ -Simetría aplicada a 10
12. $\mathcal{F} \vdash (F \vee G)$	Demostración por casos aplicada a 5 y 11

■

Así, cualquier cosa que pueda demostrarse usando resolución puede recibir una demostración formal. Se sigue entonces del Teorema 1.1 que la resolución es correcta. En particular, si R es el resolvente de dos cláusulas de una fórmula F en FNC, entonces R es consecuencia de F . Aparentemente, la resolución es un fragmento de nuestro sistema de demostración formal. Como mostramos ahora, la resolución es tan poderosa como las demostraciones formales.

1.8.3. Completitud de la resolución

Mostramos que la resolución puede usarse para determinar si una fórmula dada es satisfacible o no. Podemos suponer que la fórmula está en FNC. Dada cualquier fórmula F en FNC, sea $\text{Res}_0(F) = \{C \mid C \text{ es una cláusula de } F\}$. Para cada $n > 0$, sea

$$\text{Res}_n(F) = \text{Res}_{n-1}(F) \cup \{R \mid R \text{ es un resolvente de dos cláusulas de } \text{Res}_{n-1}(F)\}.$$

Dado que $\text{Res}_0(F) = F$ es un conjunto finito, solo hay finitas cláusulas que pueden deducirse de F usando resolventes. De hecho, solo hay finitas cláusulas que usan las mismas fórmulas atómicas que F . Así, eventualmente hallaremos algún m tal que $\text{Res}_m(F) = \text{Res}_{m+1}(F)$. Sea $\text{Res}^*(F)$ este $\text{Res}_m(F)$. Este es el conjunto de todas las cláusulas que pueden deducirse de F usando resolventes. Viéndolo como una fórmula, $\text{Res}^*(F)$ es la conjunción de todas las consecuencias de F que pueden deducirse mediante resolventes.

Proposición 1.12

Sea F una fórmula en FNC. Si $\emptyset \in \text{Res}^*(F)$, entonces F es insatisfacible.

Demostración. Si $\emptyset \in \text{Res}^*(F)$, entonces $\emptyset \in \text{Res}_n(F)$ para algún n . Dado que $\emptyset \notin \text{Res}_0(F)$ ($\emptyset$ no es una cláusula), debe existir algún m tal que $\emptyset \notin \text{Res}_m(F)$ y $\emptyset \in \text{Res}_{m+1}(F)$, en cuyo caso $\emptyset$ es el resolvente de dos cláusulas de $\text{Res}_m(F)$. Pero $\emptyset$ solo puede obtenerse como el resolvente de $\{A\}$ y $\{\neg A\}$ para alguna A atómica. Tanto $\{A\}$ como $\{\neg A\}$ deben estar en $\text{Res}_m(F)$. Por la proposición anterior, tanto A como $\neg A$ son consecuencias de F . Se sigue que $A \wedge \neg A$ es consecuencia de F y F es insatisfacible. ■

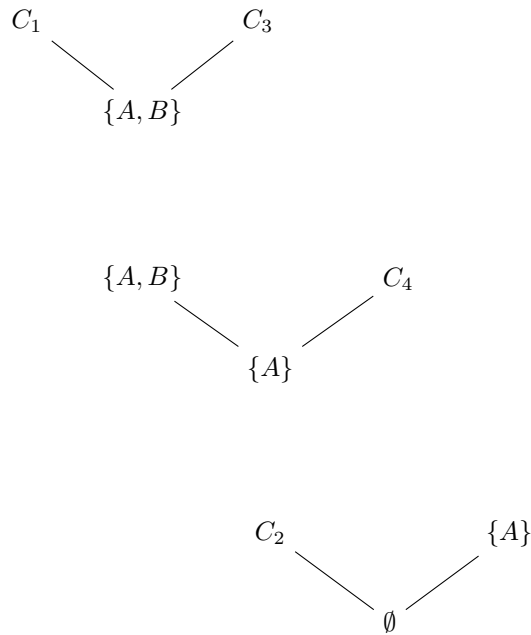
Ejemplo 1.31

Sea F la fórmula

$$\{\{A, B, \neg C\}, \{\neg A\}, \{A, B, C\}, \{A, \neg B\}\}.$$

Mostramos que F es insatisfacible usando resolución.

Sean C_1, C_2, C_3 y C_4 las cuatro cláusulas de F en el orden dado anteriormente.



Vemos que $\{A, B\} \in \text{Res}(F)$, $\{A\} \in \text{Res}_2(F)$, y $\emptyset \in \text{Res}_3(F)$. Por la Proposición 1.12, F es insatisfacible. Podemos organizar esto como una demostración de dos columnas de la siguiente manera.

Consecuencia de F	Justificación
C_1	Cláusula de F
C_3	Cláusula de F
$\{A, B\}$	Resolvente de C_1 y C_2
C_4	Cláusula de F
$\{A\}$	Resolvente de $\{A, B\}$ y C_4
C_2	Cláusula de F
$\emptyset$	Resolvente de $\{A\}$ y C_2

▲

Consideremos ahora el recíproco de la Proposición 1.12. Sea F una fórmula en FNC. Si F es insatisfacible, ¿debe entonces $\emptyset$ estar en $\text{Res}^*(F)$? Mostramos que la respuesta es "sí." La resolución es todo lo que necesitamos para mostrar insatisfacibilidad. Esto no es evidente de inmediato. Después de todo, para la columna de "Justificación" de estas demostraciones, solo tenemos dos opciones. O bien se da una cláusula, o bien es un resolvente de dos cláusulas deducidas previamente. Podría parecer que este método de demostración es demasiado restrictivo. Demostramos que no lo es.

Proposición 1.13

Sea F una fórmula en FNC. Si F es insatisfacible, entonces $\emptyset \in \text{Res}^*(F)$.

Demostración. Sea $F = \{C_1, \dots, C_k\}$. Suponemos que ninguna de las C_i es una tautología (en caso contrario, simplemente descartamos estas cláusulas y mostramos que $\emptyset$ puede deducirse de lo que resta). Demostraremos esta proposición por inducción sobre el número n de fórmulas atómicas que ocurren en F .

Sea $n = 1$. Sea A la única fórmula atómica que ocurre en F . Entonces solo hay tres cláusulas posibles en F . Cada C_i es $\{A\}$, $\{\neg A\}$, o $\{A, \neg A\}$. La última cláusula es una tautología y, por nuestra suposición anterior, no es una cláusula de F . Así, las únicas cláusulas en F son $\{A\}$ y $\{\neg A\}$. Hay tres posibilidades: $F = \{\{A\}\}$, $F = \{\{\neg A\}\}$, o $F = \{\{A\}, \{\neg A\}\}$. Las dos primeras son satisfacibles. Así, F debe ser $\{\{A\}, \{\neg A\}\}$. Claramente, $\emptyset \in \text{Res}^*(F)$.

Supongamos ahora que F tiene subfórmulas atómicas $A_1, \dots, A_{n+1}$. Supongamos además que $\emptyset \in \text{Res}^*(G)$ para cualquier fórmula insatisfacible G que use únicamente las fórmulas atómicas $A_1, \dots, A_n$. Definimos algunas fórmulas nuevas. Sea $\tilde{F}_0$ la conjunción de todas las C_i de F que no contienen a $\neg A_{n+1}$. Sea $\tilde{F}_1$ la conjunción de todas las C_i de F que no contienen a A_{n+1} . Estas son fórmulas en FNC. Afirmamos que, al verlas como conjuntos,

$$\tilde{F}_0 \cup \tilde{F}_1 = F.$$

Pues, supongamos que hay alguna cláusula C_i de F que no está en $\tilde{F}_0 \cup \tilde{F}_1$. Entonces C_i debe contener tanto a A_{n+1} como a $\neg A_{n+1}$. Pero entonces C_i es una tautología, lo cual contradice nuestra suposición

anterior. Así, $\tilde{F}_0 \cup \tilde{F}_1$ y F contienen las mismas cláusulas.

Sea $F_0 = \{C_i - \{A_{n+1}\} \mid C_i \in \tilde{F}_0\}$.

Sea $F_1 = \{C_i - \{\neg A_{n+1}\} \mid C_i \in \tilde{F}_1\}$.

Es decir, F_0 se forma eliminando A_{n+1} de cada cláusula de $\tilde{F}_0$ en la que aparece. De igual manera, F_1 se obtiene eliminando $\neg A_{n+1}$ de cada cláusula de $\tilde{F}_1$.

Afirmamos que, si reemplazamos A_{n+1} en F por una contradicción, entonces la fórmula resultante es equivalente a F_0 . Y si reemplazamos A_{n+1} en F por una tautología, entonces la fórmula resultante es equivalente a F_1 . Damos un ejemplo para ilustrar esto, pero dejamos al lector la verificación de este hecho.

Ejemplo 1.32

Supongamos que $n = 2$, de modo que A_{n+1} es A_3 .

Sea $F = \{\{A_1, A_3\}, \{A_2\}, \{\neg A_1, \neg A_2, A_3\}, \{\neg A_2, \neg A_3\}\}$.

Entonces $\tilde{F}_0 = \{\{A_1, A_3\}, \{A_2\}, \{\neg A_1, \neg A_2, A_3\}\}$

y $\tilde{F}_1 = \{\{A_2\}, \{\neg A_2, \neg A_3\}\}$.

Así, $F_0 = \{\{A_1\}, \{A_2\}, \{\neg A_1, \neg A_2\}\}$

y $F_1 = \{\{A_2\}, \{\neg A_2\}\}$.

Ahora bien, F es la fórmula $(A_1 \vee A_3) \wedge (A_2) \wedge (\neg A_1 \vee \neg A_2 \vee A_3) \wedge (\neg A_2 \vee \neg A_3)$.

Si sabemos que A_3 tiene valor de verdad 0, esto se convierte en $(A_1 \vee 0) \wedge (A_2) \wedge (\neg A_1 \vee \neg A_2 \vee 0) \wedge (1)$, que es equivalente a F_0 .

Si sabemos que A_3 tiene valor de verdad 1, entonces F se reduce a $(1) \wedge (A_2) \wedge (1) \wedge (\neg A_2 \vee 0)$, que es equivalente a F_1 . ▲

Dado que A_{n+1} debe tener valor de verdad 0 o 1, se sigue que $F \equiv F_0 \vee F_1$. Dado que F es insatisfacible, F_0 y F_1 son cada una insatisfacible. Las fórmulas F_0 y F_1 solo usan las fórmulas atómicas $A_1, \dots, A_n$. Por nuestra hipótesis de inducción, $\emptyset \in \text{Res}^*(F_0)$ y $\emptyset \in \text{Res}^*(F_1)$. (Obsérvese que $\emptyset$ puede deducirse fácilmente tanto de F_0 como de F_1 en nuestro ejemplo.)

Ahora bien, F_0 se formó a partir de $\tilde{F}_0$ eliminando A_{n+1} de cada cláusula. Dado que podemos deducir $\emptyset$ a partir de F_0 , podemos deducir $\emptyset$ o $\{A_{n+1}\}$ a partir de $\tilde{F}_0$ (reinstaurando $\{A_{n+1}\}$ en cada cláusula de F_0). De igual manera, podemos deducir $\emptyset$ o $\{\neg A_{n+1}\}$ a partir de $\tilde{F}_1$. Si podemos deducir $\{A_{n+1}\}$ de F_0 y $\{\neg A_{n+1}\}$ de F_1 , entonces podemos deducir $\emptyset$ de $\tilde{F}_0 \cup \tilde{F}_1$. Dado que $F = \tilde{F}_0 \cup \tilde{F}_1$, concluimos que $\emptyset \in \text{Res}^*(F)$. ■

Esto proporciona un algoritmo para el problema de satisfacibilidad. Dada cualquier fórmula G , primero hallamos una fórmula F en FNC que sea equivalente a G (usando el Algoritmo FNC). Luego calculamos el conjunto finito $\text{Res}^*(F)$. Si $\emptyset \in \text{Res}^*(F)$, el algoritmo concluye "No, G no es satisfacible." En caso contrario, concluye "Sí, G es satisfacible." Por las Proposiciones 1.12 y 1.13, este algoritmo funciona. Este algoritmo no es necesariamente rápido. Como mencionamos anteriormente, no se conoce ningún algoritmo de tiempo polinomial para este problema de decisión. Sin embargo, en ciertos casos, este algoritmo puede llegar rápidamente a una conclusión. Si F es insatisfacible, no necesariamente tenemos que calcular todo $\text{Res}^*(F)$. En cuanto aparece $\emptyset$, sabemos que no es satisfacible. Si F es satisfacible, por otro lado, entonces las tablas de verdad pueden llegar rápidamente a una conclusión. Solo necesitamos calcular la tabla de verdad hasta hallar un valor de verdad de 1. Resumimos los principales resultados de esta sección en el siguiente teorema. Este teorema es una

versión finita del Teorema de completitud para la lógica proposicional.

Teorema 1.5

Sean F y G fórmulas de la lógica proposicional. Sea H la fórmula en FNC obtenida al aplicar el Algoritmo FNC a la fórmula $F \wedge \neg G$. Las siguientes afirmaciones son equivalentes:

1. $F \models G$
2. $\{F\} \vdash G$
3. $\emptyset \in \text{Res}^*(H)$

Demostración. (2) implica (1) por el Teorema 1.1.

(1) implica (3) por la Proposición 1.13.

Debemos mostrar que (3) implica (2). Por la Proposición 1.7, tenemos $\{F \wedge \neg G\} \vdash H$. Por $\wedge$ -Introducción, $\{F, \neg G\} \vdash F \wedge \neg G$. Se sigue que $\{F, \neg G\} \vdash H$.

Dado que $\emptyset \in \text{Res}^*(H)$, debe existir una fórmula atómica A tal que tanto $\{A\}$ como $\{\neg A\}$ están en $\text{Res}^*(H)$. Se sigue de la Proposición 1.11 que tanto $\{H\} \vdash A$ como $\{H\} \vdash \neg A$. Por lo tanto, tanto

$$\{F, \neg G\} \vdash A \quad \text{como} \quad \{F, \neg G\} \vdash \neg A.$$

Por demostración por contradicción, tenemos $\{F\} \vdash \neg\neg G$. Finalmente, $\{F\} \vdash G$ por Doble negación. ■

1.9 Completitud y compacidad

La completitud y la compacidad son dos propiedades que una lógica puede o no poseer. Concluimos nuestro estudio de la lógica proposicional mostrando que esta lógica posee, en efecto, cada una de estas propiedades.

Una lógica es un lenguaje formal que cuenta con reglas para deducir la verdad de un enunciado a partir de la de otro. Si una sentencia G puede deducirse de un conjunto de sentencias $\mathcal{F}$ usando estas reglas, entonces escribimos $\mathcal{F} \vdash G$. La notación $\mathcal{F} \models G$, por otro lado, significa que siempre que cada sentencia de $\mathcal{F}$ sea verdadera, G también lo es. Si $\mathcal{F} \vdash G$, entonces $\mathcal{F} \models G$. Lo contrario, sin embargo, no es necesariamente cierto. Dicho de otra manera, $\mathcal{F} \models G$ significa que $\mathcal{F}$ implica G y $\mathcal{F} \vdash G$ significa que podemos demostrar que $\mathcal{F}$ implica G usando las reglas de la lógica. Pero que algo sea verdadero no significa que podamos demostrarlo. Quizá las reglas de la lógica son demasiado débiles para demostrar todo (o el poder expresivo de la lógica es demasiado fuerte). Si podemos demostrar todo lo que es verdadero (es decir, si $\mathcal{F} \models G$ sí implica $\mathcal{F} \vdash G$), entonces decimos que la lógica es *completa*.

(Completitud:) $\mathcal{F} \models G$ si y solo si $\mathcal{F} \vdash G$.

En la Sección 1.4 definimos la notación $\mathcal{F} \vdash G$ para la lógica proposicional enumerando un conjunto de reglas. Sin embargo, la completitud debe entenderse no como un enunciado sobre estas reglas

específicas, sino como un enunciado sobre la lógica misma. La completitud afirma la existencia de una lista de reglas que nos permite deducir toda consecuencia a partir de cualquier conjunto de fórmulas de la lógica. Para demostrar esto necesitamos exhibir tal lista de reglas. Mostramos que las reglas de las Tablas 1.5 y 1.6, junto con las reglas de la resolución, bastan para la lógica proposicional. Como veremos en el Capítulo 9, la lógica de segundo orden no tiene completitud. No podemos dar una lista agradable de reglas que nos permita deducir toda consecuencia a partir de cualquier conjunto de sentencias de segundo orden.

Para demostrar que la lógica proposicional tiene completitud, debemos pasar de conjuntos finitos a conjuntos infinitos de fórmulas. Si F es finito, entonces $F \models G$ si y solo si $F \vdash G$ por el Teorema 1.5. Supongamos ahora que F es infinito. Si F es un conjunto de fórmulas en FNC, entonces puede verse como un conjunto de cláusulas. El conjunto $\text{Res}_n(F)$ se define como se hizo para conjuntos finitos de cláusulas. Sea $\text{Res}^*(F)$ la unión de todos los conjuntos $\text{Res}_n(F)$ (para $n \in \mathbb{N}$). De nuevo, $\text{Res}^*(F)$ es el conjunto de todas las cláusulas que pueden deducirse de F usando resolución. Si F es infinito, entonces $\text{Res}^*(F)$ es infinito y no puede verse como una fórmula. Tal conjunto infinito de cláusulas es satisfacible si y solo si existe una asignación que modela cada cláusula del conjunto. Para demostrar que la lógica proposicional tiene completitud, basta demostrar lo siguiente.

Proposición 1.14

Sea F un conjunto de fórmulas en FNC. Entonces $\emptyset \in \text{Res}^*(F)$ si y solo si F es insatisfacible.

Demostración. Para F finito, esto es una reformulación de las Proposiciones 1.12 y 1.13. Recordemos las demostraciones de estos dos enunciados. Para la Proposición 1.13, supusimos que F era insatisfacible, y demostramos que $\emptyset \in \text{Res}^*(F)$ por inducción sobre el número de fórmulas atómicas que ocurren en F . Pero la inducción matemática solo demuestra que algo es verdadero para todo n finito. Así, el método que usamos para demostrar la Proposición 1.13 no funciona si F involucra infinitas fórmulas atómicas.

Consideremos la otra dirección de la Proposición 1.14. Supongamos que $\emptyset \in \text{Res}^*(F)$. Entonces $\emptyset \in \text{Res}_n(F)$ para algún n . Es decir, podemos deducir $\emptyset$ a partir de F en un número finito de pasos. Por lo tanto, podemos deducir $\emptyset$ a partir de algún subconjunto finito F' de F . Por la Proposición 1.12, F' es insatisfacible. Dado que F' es un subconjunto de F , F también debe ser insatisfacible. Así, una dirección de la Proposición 1.14 se sigue de los resultados de la sección anterior. Podemos deducir el caso infinito a partir del caso finito observando que, si $\emptyset$ puede deducirse de F , entonces puede deducirse de algún subconjunto finito de F . Para demostrar la otra dirección de la Proposición 1.14 necesitamos una idea análoga. Necesitamos mostrar que si F es insatisfacible, entonces algún subconjunto finito de F es insatisfacible. Esto se conoce como *compacidad*. Completamos esta demostración más adelante en esta sección, una vez establecida la compacidad de la lógica proposicional (Teorema 1.6). ■

Compacidad: F es insatisfacible si y solo si algún subconjunto finito de F es insatisfacible.

Dicho de otra manera, la compacidad afirma que F es satisfacible si y solo si todo subconjunto finito de F es satisfacible. Al igual que con la completitud, una dirección de la compacidad siempre se cumple.

Si F es satisfacible, entonces todo subconjunto finito de F también debe ser satisfacible. Pero el hecho de que todo subconjunto finito de un conjunto sea satisfacible no significa necesariamente que el conjunto mismo lo sea. Consideremos, por ejemplo, el siguiente conjunto de oraciones en español.

F_0 = "Hay finitos objetos en el universo."
 F_1 = "Hay al menos un objeto en el universo."
 F_2 = "Hay al menos dos objetos en el universo."
 F_3 = "Hay al menos tres objetos en el universo."
 $\dots$
 F_n = "Hay al menos n objetos en el universo."
 $\dots$

Tomadas en conjunto, estas oraciones son contradictorias. Si hay más de n objetos para cada n , entonces no puede haber finitos objetos, como afirma F_0 . Sin embargo, si tomamos solo finitas de las afirmaciones anteriores, no hay ningún problema. Cualquier conjunto finito de estas oraciones es satisfacible, pero la colección completa no lo es. Cualquier lógica que pueda expresar estas oraciones no tiene compacidad.

Demostremos que la lógica proposicional sí tiene compacidad en el Teorema 1.6. Primero, demostramos el siguiente lema. Este lema puede no parecer relevante por el momento, pero es la clave para demostrar el Teorema 1.6.

Lema 1.2

Sea X un conjunto infinito de cadenas binarias finitas. Existe una cadena binaria infinita $\bar{w}$ tal que cualquier prefijo de $\bar{w}$ es también prefijo de infinitas $\bar{x}$ en X .

Demostración. Una cadena binaria es una sucesión de ceros y unos, como 1011. Las cadenas 1, 10, 101 y 1011 son los prefijos de 1011. Tenemos un conjunto infinito X de tales cadenas de longitud finita. Queremos construir una cadena infinita $\bar{w}$ de ceros y unos tal que cada prefijo de $\bar{w}$ sea también prefijo de infinitas cadenas de X .

Construimos $\bar{w}$ paso a paso, de izquierda a derecha. En cada paso haremos dos cosas. En el paso n -ésimo, no solo decidimos cuál debe ser el n -ésimo dígito de $\bar{w}$, sino que también eliminamos de X las cadenas que no nos convienen.

Para determinar cuál debe ser el primer dígito de $\bar{w}$, observamos los primeros dígitos de todas las cadenas de X . Por supuesto, hay infinitas cadenas y no podemos observar todos estos dígitos a la vez, pero supongamos que somos, de alguna manera, omniscientes. Hay dos posibilidades. O bien vemos infinitos unos, o no los vemos. Si infinitas cadenas de X comienzan con 1, entonces hacemos que el primer dígito de $\bar{w}$ sea 1 y eliminamos todas las cadenas de X que comienzan con 0 (todavía nos quedan infinitas). En caso contrario, si solo finitas cadenas de X comienzan con 1, las eliminamos y hacemos que el primer dígito de $\bar{w}$ sea 0.

Supongamos ahora que hemos determinado los primeros n dígitos de $\bar{w}$. Supongamos también que hemos eliminado de X todas las sucesiones que no comienzan con estos mismos n dígitos y que nos queda un subconjunto infinito X' de X . Para determinar la entrada $(n+1)$ -ésima de $\bar{w}$ observamos los dígitos $(n+1)$ -ésimos de todas las cadenas de X' . Dado que X' es infinito, X' debe tener infinitas

cadenas de longitud $n + 1$ o mayor. Así, de nuevo hay dos posibilidades. Si infinitas cadenas de X' tienen un 1 en la posición $(n + 1)$ -ésima, entonces hacemos que el dígito $(n + 1)$ -ésimo de $\bar{w}$ sea 1. En caso contrario, hacemos que el dígito $(n + 1)$ -ésimo sea 0. En cualquier caso, eliminamos todas las cadenas de X' que no comparten las mismas primeras $n + 1$ entradas que $\bar{w}$. Todavía nos queda un subconjunto infinito de X .

Continuando este procedimiento, obtenemos una sucesión infinita $\bar{w}$ tal que los primeros n dígitos de $\bar{w}$ coinciden con los primeros n dígitos de infinitas sucesiones de X . En realidad no hemos dado una manera práctica de construir $\bar{w}$, pero hemos demostrado que tal cadena existe. ■

Estamos listos ahora para demostrar que la lógica proposicional tiene compacidad.

Teorema 1.6: Compacidad de la lógica proposicional

Un conjunto de sentencias de la lógica proposicional es satisfacible si y solo si todo subconjunto finito es satisfacible.

Demostración. Como señalamos anteriormente, solo una dirección de esto requiere demostración. Supongamos que $F = \{F_1, F_2, \dots\}$ es un conjunto de fórmulas y que todo subconjunto finito de F es satisfacible. Sean $A_1, A_2, A_3, \dots$ una lista sin repetición de las fórmulas atómicas que ocurren en F_1 , seguidas de las fórmulas atómicas que ocurren en F_2 (pero no en F_1), y así sucesivamente.

Dado que todo subconjunto finito de F es satisfacible, para cada n existe una asignación $\mathcal{A}_n$ tal que $\mathcal{A}_n \models \bigwedge_{i=1}^n F_i$. Así, cada F_i de F se cumple bajo todas, salvo a lo sumo finitas, de estas asignaciones. Podemos suponer que $\mathcal{A}_n$ está definida únicamente sobre las fórmulas atómicas que ocurren en $F_1, \dots, F_n$. Para cada n , los valores de verdad que $\mathcal{A}_n$ asigna a $A_1, A_2, \dots$ forman una sucesión finita de ceros y unos. Así, $X = \{\mathcal{A}_n \mid n = 1, 2, \dots\}$ es un conjunto infinito de sucesiones binarias finitas. Por el lema anterior, existe una sucesión binaria infinita $\bar{w}$ tal que todo prefijo de $\bar{w}$ es prefijo de infinitas sucesiones de X .

Definimos una asignación $\mathcal{A}$ sobre todas las $\mathcal{A}_n$ de la siguiente manera: sea $\mathcal{A}(A_n)$ el n -ésimo dígito de $\bar{w}$. Debemos mostrar que toda fórmula F de F se cumple bajo $\mathcal{A}$. Esto se sigue del hecho de que F se cumple bajo todas, salvo a lo sumo finitas, de las asignaciones de X . Sea m tal que F no contiene ninguna fórmula atómica posterior a A_m en nuestra lista. Entonces existe una $\mathcal{A}_n$ en X tal que $\mathcal{A}_n \models F$ y las primeras m entradas de $\mathcal{A}_n$ son las mismas que las de $\mathcal{A}$. Se sigue que $\mathcal{A}$ también modela F . ■

La Proposición 1.14 se sigue de la compacidad. Ahora podemos demostrar que la lógica proposicional tiene completitud. Podríamos dar una demostración similar a la del Teorema 1.5 usando la Proposición 1.14 en lugar de las Proposiciones 1.12 y 1.13. Sin embargo, la compacidad produce una demostración más directa.

Teorema 1.7: Completitud de la lógica proposicional

Para cualquier sentencia G y conjunto de sentencias $\mathcal{F}$, $\mathcal{F} \models G$ si y solo si $\mathcal{F} \vdash G$.

Demostración. Por el Teorema 1.1, si $\mathcal{F} \vdash G$, entonces $\mathcal{F} \models G$.

Recíprocamente, supongamos que $\mathcal{F} \models G$. Entonces $\mathcal{F} \cup \{\neg G\}$ es insatisfacible. Por compacidad, algún subconjunto finito de $\mathcal{F} \cup \{\neg G\}$ es insatisfacible. Así, existe un $\mathcal{F}_0 \subset \mathcal{F}$ finito tal que $\mathcal{F}_0 \cup \{\neg G\}$ es insatisfacible y, equivalentemente, $\mathcal{F}_0 \models G$. Dado que $\mathcal{F}_0$ es finito, podemos aplicar el Teorema 1.5 para obtener $\mathcal{F}_0 \vdash G$. Finalmente, $\mathcal{F} \vdash G$ por Monotonía. ■

Ejercicios

Ejercicio 1.1 Muestre que $\neg$ y $\vee$ pueden tomarse como símbolos primitivos en la lógica proposicional. Es decir, muestre que cada uno de los símbolos $\wedge$, $\rightarrow$ y $\leftrightarrow$ puede definirse en términos de $\neg$ y $\vee$.

Ejercicio 1.2 Muestre que $\neg$ y $\rightarrow$ pueden tomarse como símbolos primitivos en la lógica proposicional. Es decir, muestre que cada uno de los símbolos $\wedge$, $\vee$ y $\leftrightarrow$ puede definirse en términos de $\neg$ y $\rightarrow$.

Ejercicio 1.3 Halle las tablas de verdad para cada una de las siguientes fórmulas. Indique si cada una es una tautología, una contradicción, o ninguna de las dos.

- (a) $(\neg A \rightarrow B) \vee ((A \wedge \neg C) \leftrightarrow B)$
- (b) $(A \rightarrow B) \wedge (A \rightarrow \neg B)$
- (c) $(A \rightarrow (B \vee C)) \vee (C \rightarrow \neg A)$
- (d) $((A \rightarrow B) \wedge C) \vee (A \wedge D)$

Ejercicio 1.4 En cada uno de los siguientes casos, determine si las dos fórmulas son equivalentes.

- (a) $(A \wedge B) \vee C$ y $(A \rightarrow \neg B) \rightarrow C$
- (b) $((A \rightarrow B) \rightarrow B) \rightarrow B$ y $(A \rightarrow B)$
- (c) $((A \rightarrow B) \rightarrow A) \rightarrow A$ y $(C \rightarrow D) \vee C$
- (d) $A \leftrightarrow ((\neg A \wedge B) \vee (A \wedge \neg B))$ y $\neg B$

Ejercicio 1.5 Muestre que los siguientes enunciados son equivalentes.

- 1. $F \models G$,
- 2. $\models F \rightarrow G$,
- 3. $F \wedge \neg G$ es insatisfacible, y
- 4. $F \equiv F \wedge G$.

Ejercicio 1.6 Muestre que los siguientes enunciados son equivalentes.

1. $F \equiv G$,
2. $\models F \leftrightarrow G$, y
3. $(F \wedge \neg G) \vee (\neg F \wedge G)$ es insatisfacible.

Ejercicio 1.7

- (a) Halle una fórmula F en FNC que tenga la siguiente tabla de verdad.

A	B	C	F
0	0	0	0
1	0	0	1
0	1	0	1
0	0	1	1
1	1	0	0
1	0	1	0
0	1	1	0
1	1	1	1

- (b) Halle una fórmula en FND que tenga la tabla de verdad anterior.

Ejercicio 1.8 Halle fórmulas en FNC equivalentes a cada una de las siguientes.

- (a) $(A \leftrightarrow B) \leftrightarrow C$
- (b) $(A \rightarrow (B \vee C)) \vee (C \rightarrow \neg A)$
- (c) $(\neg A \wedge \neg B \wedge C) \vee (\neg A \wedge \neg C) \vee (B \wedge C) \vee A$.

Ejercicio 1.9 La *regla del corte* afirma que, a partir de las fórmulas $(F \rightarrow G)$ y $(G \rightarrow H)$, podemos deducir la fórmula $(F \rightarrow H)$. Verifique esta regla dando una demostración formal.

Ejercicio 1.10

- (a) Sea $\leftrightarrow$ -Simetría la siguiente regla:

Premisa: $\mathcal{F} \vdash (F \leftrightarrow G)$

Conclusión: $\mathcal{F} \vdash (G \leftrightarrow F)$

Verifique esta regla dando una demostración formal.

(b) Dé una demostración formal de que $\{(F \leftrightarrow G)\} \vdash (\neg F \leftrightarrow \neg G)$.

Ejercicio 1.11 Dé demostraciones formales de que las fórmulas $(F \wedge (F \vee G))$ y $(F \vee (F \wedge G))$ son equivalentes de manera demostrable.

Ejercicio 1.12 Si $F \rightarrow G$ es consecuencia de F , entonces también lo es $\neg G \rightarrow \neg F$. Nos referimos a esta regla como $\rightarrow$ -Contrapositiva. Verifique esta regla dando una demostración formal.

Ejercicio 1.13 Muestre que $\vee$ -Simetría se sigue de las demás reglas de las Tablas 1.5 y 1.6.

Ejercicio 1.14 Muestre que $\rightarrow$ -Eliminación se sigue de las demás reglas de las Tablas 1.5 y 1.6.

Ejercicio 1.15 Muestre que Doble negación se sigue de Suposición, Monotonidad y Demostración por casos.

Ejercicio 1.16 Supongamos que eliminamos de la Tabla 1.5 las siguientes cuatro reglas: $\vee$ -Eliminación, $\vee$ -Simetría, $\rightarrow$ -Introducción y $\rightarrow$ -Eliminación, y las reemplazamos por las Leyes de De Morgan, la Distributividad de $\vee$, la regla del corte (Ejercicio 1.9), y el recíproco de Doble negación (si $\mathcal{F} \vdash \neg\neg F$ entonces $\mathcal{F} \vdash F$). Muestre que el conjunto de reglas resultante es completo.

Ejercicio 1.17 Use resolución para verificar cada uno de los siguientes enunciados:

- (a) $\neg A$ es consecuencia de $(A \rightarrow B) \wedge (A \rightarrow \neg B)$
- (b) $(\neg A \wedge \neg B \wedge C) \vee (\neg A \wedge \neg C) \vee (B \wedge C) \vee A$ es una tautología
- (c) $((A \rightarrow B) \wedge (A \rightarrow \neg B)) \rightarrow \neg A$ es una tautología.

Ejercicio 1.18 Para cada fórmula del Ejercicio 1.3, halle una fórmula equivalente en FNC.

Ejercicio 1.19 Para cada fórmula del Ejercicio 1.3, verifique su respuesta a ese problema usando resolución.

Ejercicio 1.20 Determine si las siguientes fórmulas de Horn son satisfacibles o no. Si es satisfacible, halle una asignación que modele la fórmula.

- (a) $(T \rightarrow A_1) \wedge (T \rightarrow A_2) \wedge (A_1 \wedge A_2 \wedge A_3 \rightarrow A_4) \wedge (A_1 \wedge A_2 \wedge A_4 \rightarrow A_5) \wedge (A_1 \wedge A_2 \wedge A_3 \wedge A_4 \rightarrow A_6) \wedge (A_5 \wedge A_6 \rightarrow A_7) \wedge (A_2 \rightarrow A_3) \wedge (A_7 \rightarrow \perp)$
- (b) $(T \rightarrow A_1) \wedge (T \rightarrow A_2) \wedge (A_1 \wedge A_2 \wedge A_4 \rightarrow A_3) \wedge (A_1 \wedge A_5 \wedge A_6) \wedge (A_2 \wedge A_7 \rightarrow A_5) \wedge (A_1 \wedge A_3 \wedge A_5 \rightarrow A_7) \wedge (A_2 \rightarrow A_4) \wedge (A_4 \rightarrow A_8) \wedge (A_2 \wedge A_3 \wedge A_4 \rightarrow A_9) \wedge (A_3 \wedge A_9 \rightarrow A_6) \wedge (A_6 \wedge A_7 \rightarrow A_8) \wedge (A_7 \wedge A_8 \wedge A_9 \rightarrow \perp)$

Ejercicio 1.21 Considere la siguiente fórmula en FND.

$$(A_1 \wedge B_1) \vee (A_2 \wedge B_2) \vee \cdots \vee (A_n \wedge B_n)$$

Dada esta fórmula como entrada, ¿cuántos pasos le tomará al Algoritmo FNC detenerse y producir una fórmula en FNC? ¿Es este algoritmo de tiempo polinomial?

Ejercicio 1.22 Complete la demostración del Teorema 1.1.

Ejercicio 1.23 Complete la demostración de la Proposición 1.2.

Ejercicio 1.24 Demuestre la Proposición 1.3 dando dos demostraciones formales.

Ejercicio 1.25 Demuestre la Proposición 1.4.

Ejercicio 1.26 ¿Qué tiene de incorrecto la siguiente afirmación? ¿Por qué la "demostración falsa" dada no es una demostración?

Afirmación: $(A \rightarrow B) \vee C$ no es subfórmula de ninguna fórmula.

Demostración [Falsa] Sea F cualquier fórmula. Mostramos que $(A \rightarrow B) \vee C$ no es subfórmula de F por inducción sobre la complejidad de F .

Si F es atómica, entonces claramente $(A \rightarrow B) \vee C$ no es una subfórmula.

Sean F_1 y F_2 dos fórmulas. Nuestra hipótesis de inducción es que ni F_1 ni F_2 tienen a $(A \rightarrow B) \vee C$ como subfórmula.

Supongamos que F es $\neg F_1$. Si $(A \rightarrow B) \vee C$ es subfórmula de F , entonces $(A \rightarrow B) \vee C$ es subfórmula de F_1 o es F misma. No es subfórmula de F_1 por nuestra hipótesis de inducción. Además, dado que $(A \rightarrow B) \vee C$ no contiene el símbolo $\neg$, no puede ser F .

Supongamos que F es $F_1 \wedge F_2$. Si $(A \rightarrow B) \vee C$ es subfórmula de F , entonces, dado que $(A \rightarrow B) \vee C$ no contiene el símbolo $\wedge$, debe ser subfórmula de F_1 o de F_2 . Pero, por nuestra hipótesis de inducción, este no es el caso.

Se sigue que $(A \rightarrow B) \vee C$ no es subfórmula de ninguna fórmula.

Ejercicio 1.27 Demuestre la Proposición 1.6 por inducción matemática. Es decir, dadas las fórmulas $\{F_1, \dots, F_n\}$ y $\{G_1, \dots, G_m\}$, demuestre cada uno de los siguientes enunciados por inducción sobre n .

$$(a) (\bigvee_{i=1}^n F_i) \vee (\bigvee_{j=1}^m G_j) \equiv \bigvee_{i=1}^n \bigvee_{j=1}^m (F_i \vee G_j)$$

$$(b) (\bigwedge_{i=1}^n F_i) \wedge (\bigwedge_{j=1}^m G_j) \equiv \bigwedge_{i=1}^n \bigwedge_{j=1}^m (F_i \wedge G_j).$$

Ejercicio 1.28 Demuestre el Teorema 1.3 por inducción sobre la complejidad de H .

Ejercicio 1.29 Sean $\mathcal{F}$ y $\mathcal{G}$ conjuntos de fórmulas. Decimos que $\mathcal{F}$ es equivalente a $\mathcal{G}$, denotado $\mathcal{F} \equiv \mathcal{G}$, si para toda asignación $\mathcal{A}$, $\mathcal{A} \models \mathcal{F}$ si y solo si $\mathcal{A} \models \mathcal{G}$.

(a) Muestre que lo siguiente es verdadero:

Para cualesquiera $\mathcal{F}$ y $\mathcal{G}$, $\mathcal{F} \equiv \mathcal{G}$ si y solo si tanto
 $\mathcal{F} \vdash G$ para cada $G \in \mathcal{G}$ como
 $\mathcal{G} \vdash F$ para cada $F \in \mathcal{F}$.

(b) Demuestre que lo siguiente no es verdadero:

Para cualesquiera $\mathcal{F}$ y $\mathcal{G}$, $\mathcal{F} \equiv \mathcal{G}$ si y solo si tanto
para cada $G \in \mathcal{G}$ existe $F \in \mathcal{F}$ tal que $G \models F$, como
para cada $F \in \mathcal{F}$ existe $G \in \mathcal{G}$ tal que $F \models G$.

Ejercicio 1.30 Si una contradicción puede deducirse de un conjunto de sentencias, entonces se dice que el conjunto de sentencias es *inconsistente*. En caso contrario, el conjunto de sentencias es *consistente*. Sea $\mathcal{F}$ un conjunto de sentencias. Muestre que $\mathcal{F}$ es consistente si y solo si es satisfacible.

Ejercicio 1.31 Supongamos que $\mathcal{F}$ es un conjunto inconsistente de sentencias (según se definió en el Ejercicio 1.30). Para cada $G \in \mathcal{F}$, sea $\mathcal{F}_G$ el conjunto obtenido al eliminar G de $\mathcal{F}$.

(a) Demuestre que, para cualquier $G \in \mathcal{F}$, $\mathcal{F}_G \vdash \neg G$ usando el resultado del Ejercicio 1.30.

(b) Demuestre que, para cualquier $G \in \mathcal{F}$, $\mathcal{F}_G \vdash \neg G$ esbozando una demostración formal.

Ejercicio 1.32 Un conjunto de sentencias $\mathcal{F}$ se dice *cerrado bajo conjunción* si para cualesquiera F y G en $\mathcal{F}$, $F \wedge G$ también está en $\mathcal{F}$. Supongamos que $\mathcal{F}$ está cerrado bajo conjunción y es inconsistente (según se definió en el Ejercicio 1.30). Demuestre que, para cualquier $G \in \mathcal{F}$, existe $F \in \mathcal{F}$ tal que $\{F\} \vdash \neg G$.

Ejercicio 1.33 Llamamos a un conjunto de sentencias *minimal insatisfacible* si es insatisfacible, pero todo subconjunto propio es satisfacible.

(a) Muestre que existen conjuntos minimales insatisfacibles de sentencias de tamaño n para cualquier n .

(b) Muestre que todo conjunto insatisfacible de sentencias tiene un subconjunto minimal insatisfacible.

Ejercicio 1.34 [Teorema de interpolación de Craig] Supongamos que $\models (F \rightarrow G)$ y que F no es una contradicción y G no es una tautología. Muestre que existe una fórmula H tal que toda fórmula atómica de H está tanto en F como en G , y $\models (F \rightarrow H)$ y $\models (H \rightarrow G)$.

Ejercicio 1.35 [Teorema de definibilidad de Beth] Sea H una subfórmula de F . Sean $A_1, \dots, A_m$ las subfórmulas atómicas de F que no ocurren en H . Supongamos que, para cualquier fórmula H' , la fórmula $H \leftrightarrow H'$ es consecuencia de la fórmula $F \wedge F'$, donde F' es la fórmula obtenida al reemplazar cada ocurrencia de H en F por H' . Supongamos también que $m \geq 1$. Muestre que existe una fórmula G que no tiene subfórmulas atómicas distintas de $A_1, \dots, A_m$ tal que $\models F \rightarrow (H \leftrightarrow G)$.

2 Estructuras y lógica de primer orden

2.1 El lenguaje de la lógica de primer orden

La lógica de primer orden es un lenguaje más rico que la lógica proposicional. Su léxico contiene no solo los símbolos $\wedge, \vee, \neg, \rightarrow$ y $\leftrightarrow$ (y los paréntesis) de la lógica proposicional, sino también los símbolos $\exists$ y $\forall$ para "existe" y "para todo", junto con diversos símbolos para representar variables, constantes, funciones y relaciones. Estos símbolos se agrupan en cinco categorías.

- **Variables.** Las letras minúsculas del final del alfabeto ($\dots, x, y, z$) se usan para denotar variables. Las variables representan elementos arbitrarios de un conjunto subyacente. Esto, de hecho, es a lo que se refiere "primer orden." Las variables que representan conjuntos de elementos se llaman de *segundo orden*. La lógica de segundo orden, tratada en el Capítulo 9, se distingue por la inclusión de tales variables.
- **Constantes.** Las letras minúsculas del principio del alfabeto ($a, b, c, \dots$) se usan habitualmente para denotar constantes. Una constante representa un elemento específico de un conjunto subyacente.
- **Funciones.** Las letras minúsculas f, g y h se usan comúnmente para denotar funciones. Los argumentos pueden enumerarse entre paréntesis después del símbolo de función, como $f(x_1, x_2, \dots, x_n)$. La lógica de primer orden cuenta con símbolos para funciones de cualquier número de variables. Si f es una función de una, dos o tres variables, entonces se llama unaria, binaria o ternaria, respectivamente. En general, una función de n variables se llama n -aria y a n se le llama la *aridad* de la función.
- **Relaciones.** Las letras mayúsculas, especialmente P, Q, R y S , se usan para denotar relaciones. Al igual que las funciones, cada relación tiene una aridad asociada.

Disponemos de una cantidad infinita de cada uno de estos cuatro tipos de símbolos. Dado que solo hay finitas letras, se usan subíndices para lograr esta infinitud. Por ejemplo, $x_1, x_2, x_3, \dots$ se usan a menudo para denotar variables. Por supuesto, podemos usar cualquier símbolo que queramos en lógica de primer orden. Asignar las letras del alfabeto de la manera anterior es una convención conveniente. Si abre este libro en una página al azar y ve " $R(a, x, y)$," puede suponer con seguridad que R es una relación ternaria, x y y son variables, y a es una constante. Sin embargo, en ocasiones podemos usar símbolos que aún no hemos mencionado. Podemos usar el símbolo $\heartsuit$ si así lo deseamos. Sin embargo, si lo hacemos, debemos indicar qué representa este símbolo, ya sea una constante, una variable, una función de 23 variables, una relación ternaria, o lo que sea.

- **Símbolos fijos.** Los símbolos fijos son $\wedge, \vee, \neg, \rightarrow, \leftrightarrow, (,), \exists$ y $\forall$.

Por "fijos" entendemos que estos símbolos siempre se interpretan de la misma manera. Si busca en la página 211 de este libro y ve el símbolo $\wedge$, significa lo mismo que en la página 8. Significa "y." Lo

mismo ocurre con cada uno de los símbolos fijos. En cambio, la interpretación del símbolo de función f depende del contexto. Podemos usar este símbolo para representar cualquier función que elijamos. Los símbolos fijos $\exists$ y $\forall$, llamados *cuantificadores*, hacen que el lenguaje de la lógica de primer orden sea mucho más expresivo que la lógica proposicional. Se llaman el cuantificador existencial y el cuantificador universal, respectivamente. En cualquier fórmula de primer orden, cada cuantificador va inmediatamente seguido de una variable. Leemos $\exists x$ como "existe x tal que" y $\forall x$ como "para todo x ."

A continuación se muestra un ejemplo de una sentencia de la lógica de primer orden: $\forall y \exists x R(f(x), y)$. Esta sentencia dice que para todo y existe x tal que la relación R se cumple para el par ordenado $(f(x), y)$. Aquí f es una función unaria y R es una relación binaria. Que esta sentencia sea verdadera o no depende del contexto. Si la relación R es la igualdad, entonces esta sentencia es verdadera si y solo si la función f es sobreyectiva.

Debido a la ubicuidad de la igualdad en las matemáticas, agregamos a nuestra lista de símbolos fijos el símbolo $=$ para "igual." Seguimos refiriéndonos a esto como "lógica de primer orden," aunque a menudo se le llama "lógica de primer orden con igualdad." La inclusión de la igualdad permite que los cuantificadores realmente cuantifiquen. Por ejemplo, la sentencia

$$\exists x_1 \exists x_2 \exists x_3 (\neg(x_1 = x_2) \wedge \neg(x_1 = x_3) \wedge \neg(x_2 = x_3))$$

dice que existen al menos tres elementos distintos. Del mismo modo, podemos escribir sentencias que digan que existen al menos siete elementos, o menos de 23 elementos, o exactamente 45 elementos. Hemos enumerado ya por completo los símbolos de la lógica de primer orden. Nuestro siguiente objetivo es definir la sintaxis y la semántica. Es decir, necesitamos decir qué cadenas de estos símbolos son admisibles como fórmulas y también cómo interpretar las fórmulas.

2.2 La sintaxis de la lógica de primer orden

La definición de fórmula en lógica de primer orden es análoga a la definición de fórmula en lógica proposicional. Primero definimos las fórmulas atómicas y luego damos reglas para construir fórmulas más complejas. Usamos letras romanas mayúsculas como F , G y H para denotar fórmulas en lógica proposicional. En lógica de primer orden, reservamos estas letras para otros usos y en su lugar usamos letras griegas minúsculas como φ , ψ y θ para denotar fórmulas.

Antes de definir las fórmulas, debemos definir el término *término*. Los términos se definen inductivamente mediante las siguientes dos reglas.

(T1) Toda variable y toda constante es un término.

(T2) Si f es una función m -aria y $t_1, \dots, t_m$ son términos, entonces $f(t_1, \dots, t_m)$ también es un término.

Definición 2.1: Fórmula atómica

Una *fórmula atómica* es una fórmula que tiene la forma $t_1 = t_2$ o $R(t_1, \dots, t_n)$ donde R es una relación n -aria y $t_1, \dots, t_n$ son términos.

Al igual que en la lógica proposicional, consideramos primitivos algunos de los símbolos fijos. Los demás símbolos se definen en función de los símbolos primitivos. Consideramos primitivos a $\neg$, $\wedge$ y $\exists$. Toda fórmula de la lógica de primer orden se construye a partir de fórmulas atómicas mediante la aplicación repetida de tres reglas. Cada regla corresponde a un símbolo primitivo.

- (R1) Si φ es una fórmula, entonces también lo es $\neg\varphi$.
- (R2) Si φ y ψ son fórmulas, entonces también lo es $\varphi \wedge \psi$.
- (R3) Si φ es una fórmula, entonces también lo es $\exists x\varphi$ para cualquier variable x .

Obsérvese que (R1) y (R2) también eran reglas de la lógica proposicional, y solo la regla (R3) es nueva.

Definición 2.2

Una cadena de símbolos es una *fórmula* de la lógica de primer orden si y solo si se construye a partir de fórmulas atómicas mediante la aplicación repetida de las reglas (R1), (R2) y (R3).

Las definiciones de $\forall$, $\rightarrow$ y $\leftrightarrow$ son las mismas que en lógica proposicional. Definimos $\forall x\varphi$ como $\neg\exists x\neg\varphi$. Para cualquier fórmula φ , las dos fórmulas $\forall x\varphi$ y $\neg\exists x\neg\varphi$ son intercambiables. Así, de (R1) y (R3) obtenemos lo siguiente: si φ es una fórmula, entonces también lo es $\forall x\varphi$ para cualquier variable x .

Ejemplo 2.1

$\forall yP(x, y) \vee \exists yQ(x, y)$ es una fórmula de la lógica de primer orden, y $x(Q\forall P)y\exists$ ($\forall$ no lo es). ▲

En la siguiente sección discutimos la semántica de la lógica de primer orden. Para ello necesitamos conocer el orden de las operaciones de los símbolos. Los paréntesis dictan el orden de las operaciones en cualquier fórmula. En ausencia de paréntesis, usamos la siguiente regla: $\neg$, $\exists$ y $\forall$ tienen prioridad sobre $\wedge$, $\vee$, $\rightarrow$ y $\leftrightarrow$.

Ejemplo 2.2

$\exists xP(x, y) \vee Q(x, y)$ significa $(\exists xP(x, y)) \vee (Q(x, y))$ y $\forall yP(x, y) \rightarrow Q(x, y)$ significa $(\forall yP(x, y)) \rightarrow (Q(x, y))$. ▲

También usamos la siguiente convención: el orden en que se consideran $\neg$, $\exists$ y $\forall$ está determinado por el orden en que aparecen enumerados. Empleamos de nuevo las convenciones (C1) y (C2) de la Sección 1.1. Estas nos permiten omitir los paréntesis que no son necesarios.

Ejemplo 2.3

Escribimos $\neg\exists x\forall y\exists zR(x, y, z)$ en lugar de $\neg(\exists x(\forall y(\exists z(R(x, y, z))))$. ▲

Habiendo definido las fórmulas, definimos a continuación la noción de subfórmula.

Definición 2.3

Sea φ una fórmula de la lógica de primer orden. Definimos inductivamente qué significa que θ sea una subfórmula de φ de la siguiente manera:

- Si φ es atómica, entonces θ es una subfórmula de φ si y solo si $\theta = \varphi$.
- Si φ tiene la forma $\neg\psi$, entonces θ es una subfórmula de φ si y solo si $\theta = \varphi$ o θ es una subfórmula de ψ .
- Si φ tiene la forma $\psi_1 \wedge \psi_2$, entonces θ es una subfórmula de φ si y solo si $\theta = \varphi$, o θ es una subfórmula de ψ_1 , o θ es una subfórmula de ψ_2 .
- Si φ tiene la forma $\exists x\psi$, entonces θ es una subfórmula de φ si y solo si $\theta = \varphi$ o θ es una subfórmula de ψ .

Ejemplo 2.4

Sea φ la fórmula $\exists x\forall yP(x, y) \vee \forall x\exists yQ(x, y)$ donde P y Q son relaciones binarias. Las subfórmulas de φ son $\exists x\forall yP(x, y)$, $\forall yP(x, y)$, $P(x, y)$, $\forall x\exists yQ(x, y)$, $\exists yQ(x, y)$, $Q(x, y)$ y φ misma. Obsérvese que la fórmula $P(x, y) \vee \forall x\exists yQ(x, y)$, que ocurre como parte de φ , no es una subfórmula de φ . ▲

Las variables libres de una fórmula φ son aquellas variables que ocurren en φ y que no están cuantificadas. Por ejemplo, en la fórmula $\forall yR(x, y)$, x es una variable libre, pero y no lo es, ya que está cuantificada por $\forall$. Para cualquier fórmula de primer orden φ , sea $\text{libre}(\varphi)$ el conjunto de variables libres de φ . Podemos definir $\text{libre}(\varphi)$ inductivamente de la siguiente manera:

- Si φ es atómica, entonces $\text{libre}(\varphi)$ es el conjunto de todas las variables que ocurren en φ ,
 si $\varphi = \neg\psi$, entonces $\text{libre}(\varphi) = \text{libre}(\psi)$,
 si $\varphi = \psi \wedge \theta$, entonces $\text{libre}(\varphi) = \text{libre}(\psi) \cup \text{libre}(\theta)$, y
 si $\varphi = \exists x\psi$, entonces $\text{libre}(\varphi) = \text{libre}(\psi) - \{x\}$.

Definición 2.4: Sentencia

Una *sentencia* de la lógica de primer orden es una fórmula que no tiene variables libres.

Ejemplo 2.5

$\exists x\forall yP(x, y) \vee \forall x\exists yQ(x, y)$ es una sentencia de la lógica de primer orden, mientras que $\exists xP(x, y) \vee \forall xQ(x, y)$ es una fórmula pero no una sentencia (y es una variable libre). ▲

Ejemplo 2.6

Sea φ la fórmula $\forall y\exists x f(x) = y$. Entonces φ es una sentencia, ya que ambas variables que ocurren en φ están cuantificadas. Las fórmulas $f(x) = y$ y $\exists x f(x) = y$ son ambas subfórmulas de φ . Ninguna de estas subfórmulas es una sentencia. ▲

En contraste con las variables libres de una fórmula φ , las variables ligadas de φ son aquellas variables que sí tienen cuantificadores. Para cualquier fórmula de primer orden φ , $\text{lig}(\varphi)$ denota el conjunto de variables ligadas que ocurren en φ . Nuevamente, esta noción puede definirse con precisión por inducción.

Si φ es atómica, entonces $\text{lig}(\varphi) = \emptyset$,
 si $\varphi = \neg\psi$, entonces $\text{lig}(\varphi) = \text{lig}(\psi)$,
 si $\varphi = \psi \wedge \theta$, entonces $\text{lig}(\varphi) = \text{lig}(\psi) \cup \text{lig}(\theta)$, y
 si $\varphi = \exists x\psi$, entonces $\text{lig}(\varphi) = \text{lig}(\psi) \cup \{x\}$.

Toda variable que ocurre en φ está en $\text{libre}(\varphi)$ o en $\text{lig}(\varphi)$. Como muestra el siguiente ejemplo, estos dos conjuntos no son necesariamente disjuntos. Una variable puede tener tanto ocurrencias libres como ligadas dentro de la misma fórmula.

Ejemplo 2.7

Considere la fórmula $\exists x(R(x, y) \wedge \exists yR(y, x))$. La variable y ocurre libre en $R(x, y)$ y ligada en $\exists yR(y, x)$. La variable x ocurre únicamente como variable ligada. Así, si ψ denota esta fórmula, $\text{libre}(\psi) = \{y\}$ y $\text{lig}(\psi) = \{x, y\}$. ▲

Notación. Escribimos $\varphi(x_1, x_2, \dots, x_n)$ para denotar una fórmula que tiene variables libres $x_1, x_2, \dots, x_n$. Escribimos $\varphi(t_1, t_2, \dots, t_n)$ para denotar la fórmula obtenida al reemplazar cada ocurrencia libre de x_i en φ por el término t_i . Al usar esta notación, siempre debe suponerse que cada t_i no contiene ninguna de las variables en $\text{lig}(\varphi)$. (Por ejemplo, si $\varphi(x)$ es $\exists y\neg(x = y)$, entonces no permitimos la sustitución $\varphi(y)$.)

La presencia de variables libres distingue las fórmulas de las sentencias. Esta distinción no existía en la lógica proposicional. La noción de verdad se define solo para las sentencias. No tiene sentido preguntar si la fórmula $y = x + 1$ es verdadera o no. Pero sí podemos preguntar si $\forall y\exists x(y = x + 1)$ o $c_1 = c_2 + 1$ son verdaderas o no. La respuesta, como ya hemos indicado, depende del contexto.

2.3 Semántica y estructuras

Al igual que en la lógica proposicional, la semántica de $\wedge$ y $\neg$ puede describirse diciendo que $\wedge$ se comporta como "y" y $\neg$ se comporta como "negación." Del mismo modo, la semántica de los cuantificadores $\exists$ y $\forall$ puede inferirse de las frases "existe" y "para todo." Sin embargo, debemos ser más precisos al definir la semántica de una lógica. El objetivo de esta sección es definir formalmente la semántica de la lógica de primer orden. Primero, describimos intuitivamente la semántica con algunos ejemplos.

Considere la sentencia de primer orden

$$\forall y\exists x f(x) = y.$$

Esta sentencia dice que para todo y existe x tal que $f(x) = y$. Para determinar si esta sentencia es verdadera o no, necesitamos un contexto. Depende de qué representen las variables y qué sea la función f . Por ejemplo, supongamos que las variables son números reales y f está definida por la regla $f(x) = x^2$. Entonces la sentencia anterior es falsa, ya que no existe x tal que $f(x) = -1$. Si la

función f está definida por $f(x) = x^3$ (o si las variables representan números complejos), entonces la sentencia es verdadera.

Considere ahora $\forall x \forall y (R(x, y) \rightarrow \exists z (\neg(z = x) \wedge \neg(z = y) \wedge (R(x, z) \wedge R(z, y))))$. Esta sentencia dice que, para cualesquiera x y y , si $R(x, y)$ se cumple, entonces existe algún z distinto de x y de y tal que tanto $R(x, z)$ como $R(z, y)$ se cumplen. Supongamos de nuevo que las variables representan números reales. Si la relación $R(x, y)$ significa $x < y$, entonces la sentencia anterior es verdadera, ya que entre cualesquiera dos números reales existen otros números reales. Es decir, los números reales son densos. Sin embargo, si las variables representan enteros (o si R significa $\leq$), entonces esta sentencia es falsa. Así pues, que una sentencia sea verdadera o no depende de dos cosas: nuestro conjunto subyacente y nuestra interpretación de los símbolos de función, constante y relación. Esta observación nos lleva al concepto central de este capítulo. Una *estructura* consiste en un conjunto subyacente junto con una interpretación de diversas funciones, constantes y relaciones. El papel de las estructuras en la lógica de primer orden es análogo al papel que desempeñan las asignaciones en la lógica proposicional. Dada cualquier sentencia φ y cualquier estructura M , definimos qué significa que M modele a φ . Intuitivamente, esto significa que la sentencia φ es verdadera respecto de M . Como en la lógica proposicional, escribimos $M \models \varphi$ para denotar este concepto. La definición formal de este concepto se dará más adelante en esta sección.

Las estructuras surgen de manera natural en muchas ramas de las matemáticas. Por ejemplo, un espacio vectorial es una estructura. Los grupos, anillos y campos del álgebra abstracta también proporcionan ejemplos de estructuras. En teoría de grafos, los grafos pueden verse como estructuras de primer orden (discutiremos esto en detalle en la Sección 2.4). Los números reales proporcionan ejemplos de estructuras que deberían resultar familiares a cualquier lector. Los números reales no forman una sola estructura, sino muchas. Recordemos que una estructura tiene dos componentes: un conjunto subyacente y una interpretación de ciertas funciones, constantes y relaciones. Cuando nos referimos a "los números reales," solo estamos especificando el conjunto subyacente y no los símbolos que han de interpretarse. Podríamos querer considerar los reales con las funciones de suma y multiplicación. Esa es una estructura. Otra estructura son los reales con la relación $\leq$ y la constante 0. Dependiendo de qué aspecto de los números reales queramos investigar, podemos elegir diversas funciones, constantes y relaciones sobre los reales. A las funciones, constantes y relaciones que elegimos considerar se le llama el *vocabulario* de la estructura. Cada elección de un vocabulario determina una estructura distinta que tiene a los números reales como conjunto subyacente.

Definición 2.5: Vocabulario

Un *vocabulario* es un conjunto de símbolos de función, de relación y de constante.

Definición 2.6: $\mathcal{V}$ -estructura

Sea $\mathcal{V}$ un vocabulario. Una $\mathcal{V}$ -estructura consiste en un conjunto subyacente no vacío U junto con una interpretación de $\mathcal{V}$. Una interpretación de $\mathcal{V}$ asigna:

- un elemento de U a cada constante de $\mathcal{V}$,
- una función de U^n en U a cada función n -aria de $\mathcal{V}$, y
- un subconjunto de U^n a cada relación n -aria de $\mathcal{V}$.

Decimos que M es una *estructura* si es una $\mathcal{V}$ -estructura para algún vocabulario $\mathcal{V}$.

Presentamos las estructuras enumerando el conjunto subyacente, o universo, seguido de los símbolos de función, relación y constante que interpreta.

Ejemplo 2.8

Sea $\mathcal{V} = \{f, R, c\}$ donde f es una función unaria, R es una relación binaria y c es una constante. Entonces $M = (\mathbb{Z} \mid f, R, c)$ denota una $\mathcal{V}$ -estructura. El universo de M es el conjunto de los enteros $\mathbb{Z}$. Para completar la descripción de M , debemos decir cómo se interpretan los símbolos de $\mathcal{V}$. Podemos decir, por ejemplo, que M interpreta $f(x)$ como x^2 , $R(x, y)$ como $x < y$, y la constante c como 3. Esto describe por completo la estructura M . ▲

Ejemplo 2.9

Sea $\mathcal{V} = \{P, R\}$ donde P es una relación unaria y R es una relación binaria. Entonces $M = (\mathbb{N} \mid P, R)$ denota una $\mathcal{V}$ -estructura. El universo de M es el conjunto de los números naturales $\mathbb{N}$. Para completar la descripción de M , debemos decir cómo se interpretan los símbolos de $\mathcal{V}$. Podemos decir, por ejemplo, que M interpreta

$$P(x) \text{ como " } x \text{ es un número par," y}$$

$$R(x, y) \text{ como " } x + 1 = y."$$

Esta información describe por completo la estructura M . ▲

Ejemplo 2.10

$R = (\mathbb{R} \mid +, \cdot, 0, 1)$ denota una estructura en el vocabulario $\{+, \cdot, 0, 1\}$ donde $+$ y $\cdot$ son funciones binarias y 0 y 1 son constantes. El universo de R es el conjunto de los números reales $\mathbb{R}$. Para completar la descripción de R , debemos decir cómo se interpretan los símbolos. Podemos simplemente decir que R interpreta los símbolos "de la manera usual." Esto significa que R interpreta $+$ como la suma, $\cdot$ como el producto, 0 como el 0 y 1 como el 1. Esto describe por completo la estructura R . ▲

Definición 2.7

Sea $\mathcal{V}$ un vocabulario. Una $\mathcal{V}$ -fórmula es una fórmula en la que toda función, relación y constante pertenece a $\mathcal{V}$. Una $\mathcal{V}$ -sentencia es una $\mathcal{V}$ -fórmula que es una sentencia.

Si M es una $\mathcal{V}$ -estructura, entonces cada $\mathcal{V}$ -sentencia φ es verdadera o falsa en M . Si φ es verdadera en M , decimos que M modela φ y escribimos $M \models \varphi$. Las estructuras en lógica de primer orden desempeñan un papel análogo al de las asignaciones en lógica proposicional. Pero mientras que, en lógica proposicional, solo había finitas asignaciones posibles para una sentencia, no hay fin a la cantidad de estructuras que pueden o no modelar una sentencia dada de la lógica de primer orden. Intuitivamente, $M \models \varphi$ significa que la sentencia φ es verdadera respecto de M . Debemos definir este concepto con precisión. Antes de hacerlo, consideramos un ejemplo más.

Ejemplo 2.11

Consideremos de nuevo la estructura R del Ejemplo 2.10. El vocabulario de esta estructura es $\{+, \cdot, 0, 1\}$, que denotamos por $\mathcal{V}_{ar}$ (el vocabulario de la aritmética).

Consideremos la $\mathcal{V}_{ar}$ -sentencia $\forall x \exists y (1 + x \cdot x = y)$. Esta sentencia dice que para cualquier x existe y que es igual a $x^2 + 1$. Esto es verdadero en R . Si tomamos cualquier número real, lo elevamos al cuadrado y le sumamos uno, el resultado es otro número real. Así, $R \models \forall x \exists y (1 + x \cdot x = y)$.

Consideremos ahora la $\mathcal{V}_{ar}$ -sentencia $\forall y \exists x (1 + x \cdot x = y)$. Esta sentencia afirma que para todo y existe x tal que $1 + x^2 = y$. Esta sentencia no es verdadera en R . Si tomamos, por ejemplo, $y = -2$, entonces no existe tal x . Así, la estructura R no modela la sentencia $\forall y \exists x (1 + x \cdot x = y)$. ▲

Sea M una $\mathcal{V}$ -estructura y sea φ una $\mathcal{V}$ -sentencia. Definimos ahora formalmente qué significa que M modele a φ . Primero definimos este concepto para sentencias φ que no contienen las abreviaturas $\vee$, $\rightarrow$, $\leftrightarrow$ ni $\forall$. Definimos $M \models \varphi$ por inducción sobre el número total de ocurrencias de los símbolos $\wedge$, $\neg$ y $\exists$. Si φ tiene cero ocurrencias de estos símbolos, entonces φ es atómica.

- Si φ es atómica, entonces φ tiene la forma $t_1 = t_2$ o $R(t_1, \dots, t_m)$ donde $t_1, \dots, t_m$ son términos y R es una relación de $\mathcal{V}$. Dado que φ es una sentencia, φ no contiene variables, así que cada t_i se interpreta como algún elemento a_i del universo U de M . En este caso,

$M \models t_1 = t_2$ si y solo si a_1 y a_2 son el mismo elemento de U , y

$M \models R(t_1, \dots, t_m)$ si y solo si la tupla $(a_1, \dots, a_m)$ está en el subconjunto de U^m asignado a la relación R .

Supongamos ahora que φ contiene $m + 1$ ocurrencias de $\wedge$, $\neg$ y $\exists$. Supongamos que $M \models \psi$ se ha definido para cualquier sentencia ψ que contenga a lo sumo m ocurrencias de estos símbolos. Dado que las fórmulas de primer orden se construyen a partir de fórmulas atómicas usando las reglas (R1), (R2) y (R3), hay tres posibilidades para φ .

- Si φ tiene la forma $\neg\psi$, entonces $M \models \varphi$ si y solo si M no modela ψ .
- Si φ tiene la forma $\psi \wedge \theta$, entonces $M \models \varphi$ si y solo si tanto $M \models \psi$ como $M \models \theta$.

La tercera posibilidad es que φ tenga la forma $\exists x\psi$. Si x no es una variable libre de ψ , entonces $M \models \varphi$ si y solo si $M \models \psi$. En caso contrario, sea $\psi(x)$ una fórmula que tiene a x como variable libre. Antes de definir $M \models \varphi$ en este caso, introducimos la noción de *expansión*.

Definición 2.8: Expansión

Sea $\mathcal{V}$ un vocabulario. Una *expansión* de $\mathcal{V}$ es un vocabulario que contiene a $\mathcal{V}$ como subconjunto.

Definición 2.9

Sea M una $\mathcal{V}$ -estructura. Una estructura M' es una *expansión* de M si M' tiene el mismo universo que M e interpreta los símbolos de $\mathcal{V}$ de la misma manera que M .

Si M' es una expansión de M , entonces, invirtiendo nuestro punto de vista, decimos que M es un *reducto* de M' . Si M' es una expansión de M , entonces el vocabulario de M' es necesariamente una expansión del vocabulario de M .

Ejemplo 2.12

La estructura $M' = (\mathbb{R} \mid +, -, \cdot, <, 0, 1)$ es una expansión de $M = (\mathbb{R} \mid +, \cdot, <, 0)$, donde cada una de estas estructuras interpreta los símbolos de la manera usual (véase el Ejemplo 2.10). ▲

Ejemplo 2.13

Cualquier estructura es (trivialmente) una expansión de sí misma. ▲

Nuestro interés inmediato es la expansión de una $\mathcal{V}$ -estructura M que se obtiene al agregar al vocabulario una nueva constante por cada elemento del universo U_M de M . Sea $\mathcal{V}(M)$ el vocabulario $\mathcal{V} \cup \{c_m \mid m \in U_M\}$ donde cada c_m es una constante. Sea M_C la única expansión de M a una $\mathcal{V}(M)$ -estructura que interpreta cada c_m como el elemento m .

- Si φ tiene la forma $\exists x\psi(x)$, entonces $M \models \varphi$ si y solo si $M_C \models \psi(c)$ para alguna constante c de $\mathcal{V}(M)$.

Hemos definido ahora $M \models \varphi$ para cualquier sentencia φ que no use $\vee$, $\rightarrow$, $\leftrightarrow$ ni $\forall$. Dado que cada uno de estos símbolos se define en función de $\neg$, $\wedge$ y $\exists$, la definición de $M \models \varphi$ puede extenderse a todas las sentencias de manera natural. Supongamos que, para alguna sentencia φ' , se ha definido $M \models \varphi'$. Supongamos además que φ tiene una subfórmula de la forma $\neg(\neg\psi \wedge \neg\theta)$. Sea φ' la sentencia obtenida al reemplazar una ocurrencia de la subfórmula $\neg(\neg\psi \wedge \neg\theta)$ en φ por $(\psi \vee \theta)$. Definimos que $M \models \varphi$ signifique lo mismo que $M \models \varphi'$. Del mismo modo, si φ' se obtiene de φ reemplazando una subfórmula de la forma $(\psi \rightarrow \theta)$ por $(\neg\psi \vee \theta)$, $(\psi \leftrightarrow \theta)$ por $(\psi \rightarrow \theta) \wedge (\theta \rightarrow \psi)$, o $\neg\exists x\neg\psi(x)$ por $\forall x\psi(x)$, entonces, por definición, $M \models \varphi$ si y solo si $M \models \varphi'$.

Hemos definido ahora qué significa que una $\mathcal{V}$ -estructura M sea un modelo de una $\mathcal{V}$ -sentencia φ .

Extendemos además la definición para que se aplique a todas las $\mathcal{V}(M)$ -sentencias. Recordemos que $\mathcal{V}(M)$ es la expansión de $\mathcal{V}$ obtenida al agregar una nueva constante por cada elemento del universo de M . Existe una expansión natural de M a una $\mathcal{V}(M)$ -estructura denotada por M_C . Para cualquier $\mathcal{V}(M)$ -sentencia φ , definimos $M \models \varphi$ como $M_C \models \varphi$. Para cualquier $\mathcal{V}$ -estructura M , nos referimos a las constantes de $\mathcal{V}(M)$ que no están en $\mathcal{V}$ como *parámetros*.

Ejemplo 2.14

Sea $\mathcal{V}_<$ el vocabulario que consiste en una única relación binaria $<$. Sea $R_<$ la $\mathcal{V}_<$ -estructura cuyo conjunto subyacente es $\mathbb{R}$ y que interpreta $<$ de la manera usual. Entonces $R_<$ modela

$$\begin{aligned} & \forall x \exists y \exists z ((y < x) \wedge (x < z)), \\ & \forall x \forall y ((x < y) \rightarrow \exists z ((x < z) \wedge (z < y))) \\ & (3 < 5) \wedge (-2 < 0), \quad y \\ & \neg \exists x ((x < -2) \wedge (5 < x)). \end{aligned}$$

Las dos primeras son $\mathcal{V}_<$ -sentencias. Las otras dos son $\mathcal{V}_<(R_<)$ -sentencias que no son $\mathcal{V}_<$ -sentencias. Consideramos a -2 , 0 , 3 y 5 como parámetros. ▲

Obsérvese que no hemos definido el concepto de "modelar" para fórmulas que no son sentencias. Por convención, cuando se dice que una estructura M modela una fórmula $\varphi(x_1, \dots, x_n)$, lo que se quiere decir es que M modela la sentencia $\forall x_1 \dots \forall x_n \varphi(x_1, \dots, x_n)$. Por supuesto, la fórmula $\varphi(x_1, \dots, x_n)$ puede ser verdadera para algunos valores de $x_1, \dots, x_n$ y no para otros. El conjunto de n -tuplas para las cuales la fórmula se cumple se llama el *conjunto definido* por φ .

Definición 2.10

Sea $\varphi(x_1, \dots, x_n)$ una $\mathcal{V}$ -fórmula. Sea M una $\mathcal{V}$ -estructura con conjunto subyacente U_M . El conjunto de todas las n -tuplas $(b_1, \dots, b_n) \in (U_M)^n$ para las cuales $M \models \varphi(b_1, \dots, b_n)$ se denota por $\varphi(M)$. El conjunto $\varphi(M)$ se llama un *subconjunto $\mathcal{V}$ -definible* de M (aunque en realidad es un subconjunto de $(U_M)^n$).

Típicamente, la mayoría de los subconjuntos del universo de una estructura no son definibles (como veremos en la Sección 2.5). Los subconjuntos definibles son subconjuntos especiales y desempeñan un papel central en la teoría de modelos (Capítulos 4 a 6). Los subconjuntos $\mathcal{V}$ -definibles son los subconjuntos que el vocabulario $\mathcal{V}$ es capaz de describir. Para efectos de la teoría de modelos, la noción de estructura de primer orden puede definirse sin hacer referencia a la sintaxis de la lógica de primer orden. En general, una "estructura" puede definirse como un conjunto junto con subconjuntos especiales que tienen nombres. Una estructura de primer orden es una estructura que tiene nombres para aquellos conjuntos que son definibles mediante una fórmula de primer orden (véanse los Ejercicios 2.11 y 2.12).

Ejemplo 2.15

Sean $\mathcal{V}_<$ y $R_<$ como en el ejemplo anterior. Consideremos las $\mathcal{V}_<(R_<)$ -fórmulas

$$(x < y) \vee (x > y) \vee (x = y), \quad y \\ (\neg(x < 3) \wedge \neg(x = 3) \wedge (5 < x)) \vee (x = 5) \vee (x < -2).$$

Sea $\varphi(x, y)$ la primera fórmula y sea $\psi(x)$ la segunda fórmula. Entonces $R_< \models \varphi(x, y)$. Con esto queremos decir que $R_<$ modela la sentencia $\forall x \forall y \varphi(x, y)$. Se sigue que el conjunto definido por $\varphi(x, y)$ es todo $\mathbb{R}^2$. En cambio, la fórmula $\psi(x)$ no se cumple para todo x en $\mathbb{R}$. Así, $R_<$ no modela esta fórmula. El conjunto $\psi(R_<)$ definido por $\psi(x)$ es $(-\infty, -2) \cup (3, 5]$. Obsérvese que $R_<$ tampoco modela la fórmula $\neg\psi(x)$. El conjunto $\neg\psi(R_<)$ es $[-2, 3] \cup (5, \infty)$, el complemento de $\psi(R_<)$ en $\mathbb{R}$.

▲

Si M modela a φ , decimos que φ se cumple en M , o simplemente, que φ es verdadera en M . Una sentencia puede ser verdadera en una estructura y no en otra. Si una $\mathcal{V}$ -sentencia φ se cumple en toda $\mathcal{V}$ -estructura, entonces es *válida* (o una *tautología*). Si la sentencia φ se cumple en alguna estructura, entonces es *satisfacible*. En caso contrario, si no hay ninguna estructura en la que φ sea verdadera, entonces φ es *insatisfacible* (o una *contradicción*).

Usamos la misma terminología que en lógica proposicional. Damos las definiciones análogas de consecuencia y equivalencia. Para $\mathcal{V}$ -sentencias θ y φ , " θ es consecuencia de φ " significa que, para toda $\mathcal{V}$ -estructura M , si $M \models \varphi$ entonces $M \models \theta$. Y " θ es equivalente a φ " significa que θ y φ son consecuencias la una de la otra. De nuevo, usamos la siguiente notación:

$$\begin{aligned} \models \varphi &\text{ significa que } \varphi \text{ es una tautología,} \\ \varphi \models \psi &\text{ significa que } \psi \text{ es consecuencia de } \varphi, \text{ y} \\ \varphi \equiv \theta &\text{ significa que } \varphi \text{ y } \theta \text{ son equivalentes.} \end{aligned}$$

La definición de satisfacibilidad puede extenderse para aplicarse a todas las fórmulas de la lógica de primer orden (no solo a las sentencias). La fórmula $\varphi(x_1, \dots, x_m)$ es satisfacible si y solo si la sentencia $\forall x_1 \dots \forall x_m \varphi(x_1, \dots, x_m)$ es satisfacible. Por lo tanto, las nociones de insatisfacibilidad, tautología y consecuencia también se aplican a las fórmulas, además de a las sentencias (véase el Ejercicio 2.6). Uno de nuestros objetivos principales es resolver los siguientes problemas de decisión.

Problema de validez: Dada una fórmula φ , ¿es φ válida?

Problema de satisfacibilidad: Dada una fórmula φ , ¿es φ satisfacible?

Problema de consecuencia: Dadas las fórmulas φ y ψ , ¿es ψ consecuencia de φ ?

Problema de equivalencia: Dadas las fórmulas φ y ψ , ¿son φ y ψ equivalentes?

Estos son, en cierto sentido, variaciones del mismo problema. Por esta razón nos enfocamos en uno solo de ellos: el problema de satisfacibilidad. Si pudiéramos resolver este problema, entonces también podríamos resolver el problema de validez (preguntando si $\neg\varphi$ es insatisfacible), el problema de consecuencia (preguntando si $\varphi \wedge \neg\psi$ es insatisfacible), y el problema de equivalencia (preguntando si φ y ψ son consecuencias la una de la otra).

La pregunta de si una fórmula dada es satisfacible o no concierne a la sintaxis de la fórmula, más que a la semántica. Por ejemplo, consideremos la fórmula $(y + 1) < y$. Si interpretamos el vocabulario

$\{+, <, 1\}$ de la manera usual, entonces esta fórmula no puede satisfacerse. El resultado de sumarle uno a un número no puede ser menor que el número. Bajo una interpretación diferente, sin embargo, esta fórmula sí es satisfacible (supongamos que interpretamos $<$ como "distinto de"). Por la misma razón, $2 + 2 = 4$ no es una tautología. Como ejemplo de una fórmula que no es satisfacible, consideremos $\forall x R(x, y) \rightarrow \exists x \neg R(x, y)$. Esta fórmula es insatisfacible en virtud de su estructura. Tiene la forma " p implica no p ." Sin importar cómo se interprete la relación binaria R , la fórmula es contradictoria. El problema de satisfacibilidad para la lógica de primer orden es decididamente más difícil que el problema correspondiente para la lógica proposicional. En lógica proposicional podíamos, en teoría, calcular una tabla de verdad para determinar si una fórmula es satisfacible o no. En lógica de primer orden, tendríamos que revisar cada estructura para hacer esto. No tenemos una manera sistemática de hacerlo. Así, por ahora, no tenemos manera de demostrar que una fórmula de primer orden es insatisfacible. Mostrar que una fórmula es satisfacible, sin embargo, puede ser fácil. Solo necesitamos hallar una estructura en la que sea verdadera.

Ejemplo 2.16

Sea φ la sentencia $\forall x \exists y R(x, y) \wedge \exists y \forall x \neg R(x, y)$. Para mostrar que esto es satisfacible, debemos hallar una estructura M que modele φ .

Sea $M = (\mathbb{N} \mid R)$ donde $\mathbb{N}$ denota los números naturales y la relación binaria R se interpreta como la relación sucesor. Es decir, $R(x, y)$ se cumple si y solo si $y = x + 1$ (y es el sucesor de x).

Bajo esta interpretación, φ dice que todo elemento tiene un sucesor y existe un elemento que no tiene predecesor. Esto es verdadero en M . Todo número natural tiene un sucesor, pero 0 no tiene predecesor.

Así, $M \models \varphi$ y φ es satisfacible.

También es fácil ver que φ no es una tautología. Solo necesitamos hallar una estructura que modele $\neg \varphi$. Consideremos, por ejemplo, la estructura $N = (\mathbb{Z} \mid R)$ donde $\mathbb{Z}$ denota el conjunto de los enteros y la relación binaria R se interpreta como la relación sucesor. Esta estructura no modela φ , ya que todo entero tiene un predecesor. ▲

Ejemplo 2.17

Sea $\mathcal{V}_E$ el vocabulario $\{E\}$ que consiste en una relación binaria. Sea M una $\mathcal{V}_E$ -estructura. La relación E es una relación de equivalencia sobre M si y solo si M modela las tres sentencias

$$\begin{aligned} & \forall x E(x, x) \\ & \forall x \forall y (E(x, y) \rightarrow E(y, x)) \\ & \forall x \forall y \forall z ((E(x, y) \wedge E(y, z)) \rightarrow E(x, z)). \end{aligned}$$

La primera sentencia, llamémosla φ_1 , dice que E es reflexiva; la segunda sentencia, φ_2 , dice que E es simétrica; y la tercera sentencia φ_3 dice que E es transitiva.

Ya hemos visto relaciones de equivalencia antes. "Equivalencia" fue el nombre que dimos a la relación $\equiv$ entre fórmulas de la lógica proposicional. Es fácil ver que esta relación merece el nombre que le dimos. Claramente satisface las tres condiciones de una relación de equivalencia. Es decir, la $\mathcal{V}_E$ -estructura $(U \mid E)$ modela cada φ_i , donde U es el conjunto de todas las fórmulas de la lógica proposicional y E se interpreta como $\equiv$.

Podemos mostrar que estas tres sentencias no son redundantes, es decir, que las tres son necesarias para definir la noción de relación de equivalencia. Para ello, mostramos que ninguna de estas sentencias es consecuencia de las otras dos. Por ejemplo, para mostrar que φ_2 no es consecuencia de φ_1 y φ_3 , debemos hallar una estructura que sea modelo de $\varphi_1 \wedge \varphi_3 \wedge \neg\varphi_2$. Es decir, debemos exhibir una $\mathcal{V}_E$ -estructura donde E sea reflexiva y transitiva, pero no simétrica. La $\mathcal{V}_E$ -estructura $(\mathbb{R} \mid E)$ donde E se interpreta como $\leq$ sobre los números reales es una estructura así. Del mismo modo, podemos mostrar que φ_1 no es consecuencia de φ_2 y φ_3 , y que φ_3 no es consecuencia de φ_1 y φ_2 . Dejamos esto como Ejercicio 2.4. ▲

En estos ejemplos hemos podido mostrar que ciertas fórmulas son satisfacibles exhibiendo estructuras en las que se cumplen. Usando esta misma idea, podemos mostrar que una fórmula dada no es una tautología, que una fórmula no es consecuencia de otra, y que dos fórmulas dadas no son equivalentes. Sin embargo, por ahora no tenemos manera de mostrar que una fórmula es insatisfacible, o una tautología, o que una fórmula es consecuencia de otra. Este es el tema del Capítulo 3, donde definimos tanto las demostraciones formales como la resolución para la lógica de primer orden.

2.4 Ejemplos de estructuras

Examinemos ahora algunas estructuras específicas. Consideramos cuatro tipos de estructuras que uno encuentra en matemáticas y en ciencias de la computación: sistemas numéricos, órdenes lineales, bases de datos y grafos.

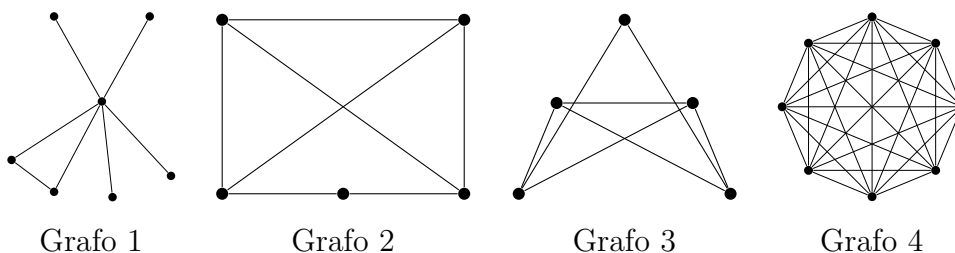
2.4.1. Grafos

La teoría de grafos proporciona ejemplos de estructuras matemáticas que son a la vez accesibles y versátiles.

Definición 2.11: Grafo

Un *grafo* es un conjunto de puntos, llamados *vértices*, y líneas, llamadas *aristas*, de modo que toda arista comienza en un vértice y termina en un vértice. Se dice que dos vértices son *adyacentes* si están conectados por una arista.

Los siguientes son ejemplos de grafos:



En lugar de dar una imagen, podemos describir un grafo enumerando sus vértices y aristas. Los siguientes datos describen por completo un grafo.

Vértices: a, b, c, d, e

Aristas: $ab, ad, ae, bc, cd, ce, de$

Este grafo tiene cinco vértices (a, b, c, d y e) y siete aristas (entre los vértices a y b , a y d , y así sucesivamente). Obsérvese que tanto el Grafo 2 como el Grafo 3 se ajustan a esta descripción. Consideramos al Grafo 2 y al Grafo 3 como dos representaciones del mismo grafo.

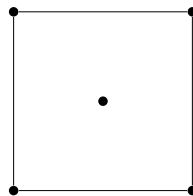
Podemos ver cualquier grafo como una estructura G de la siguiente manera. El conjunto subyacente U de G es el conjunto de vértices. El vocabulario $\mathcal{V}_G$ de G consiste en una única relación binaria R . La estructura G interpreta R como la relación de adyacencia. Es decir, para elementos a y b de U , $G \models R(a, b)$ si y solo si el grafo tiene una arista entre los vértices a y b .

Cada uno de los grafos anteriores modela cada una de las siguientes dos $\mathcal{V}_G$ -sentencias.

$$\begin{aligned} &\forall x \neg R(x, x) \\ &\forall x \forall y (R(x, y) \leftrightarrow R(y, x)) \end{aligned}$$

La primera de estas sentencias dice que la relación binaria R no es reflexiva (ningún vértice es adyacente a sí mismo). La segunda sentencia dice que R es simétrica. De ahora en adelante, cuando hablemos de un *grafo*, queremos decir una $\mathcal{V}_G$ -estructura que modela las dos sentencias anteriores. Nuestra noción de "grafo" se describe con más precisión, en términos de la teoría de grafos, como un "grafo no dirigido sin aristas múltiples ni lazos."

Los Grafos 1 a 4 también modelan la sentencia $\forall x \exists y R(x, y)$, que afirma que cada vértice es adyacente a algún otro vértice. Sin embargo, esto no es cierto para todos los grafos. Por ejemplo, considere el siguiente grafo:



Grafo 5

El vértice en el centro del cuadrado no es adyacente a ningún vértice. Por lo tanto, este grafo modela $\exists x \forall y \neg R(x, y)$, que es equivalente a la negación de $\forall x \exists y R(x, y)$. Cualquier grafo que contenga más de un vértice y modele esta negación no puede ser *conexo*. Definimos ahora esta terminología.

Definición 2.12

Para cualesquiera vértices a y b de un grafo, un *camino* de a a b es una sucesión de vértices que comienza con a y termina con b , tal que cada vértice, salvo a , es adyacente al vértice anterior en la sucesión.

Definición 2.13: Conexa

Un grafo es *conexo* si para cualesquiera dos vértices a y b de G , existe un camino de a a b .

Cada uno de los Grafos 1 a 4 es conexo. Dado que cada uno tiene más de un vértice, cada uno modela $\forall x \exists y R(x, y)$. Por otro lado, ninguno de estos grafos modela $\exists x \forall y R(x, y)$. Esta sentencia afirma que existe un vértice adyacente a todo vértice. Dado que ningún vértice es adyacente a sí mismo, ningún grafo modela esta sentencia (es decir, la negación de $\exists x \forall y R(x, y)$ es consecuencia de $\forall x \neg R(x, x)$). Sin embargo, el Grafo 1 contiene un vértice que es adyacente a todo vértice distinto de sí mismo. Esto puede expresarse en lógica de primer orden de la siguiente manera:

$$\exists x \forall y (\neg(x = y) \rightarrow R(x, y)).$$

El Grafo 4 también modela esta sentencia.

Para distinguir el Grafo 1 del Grafo 4, podemos decir que el Grafo 1 contiene un *único* vértice que es adyacente a todo vértice distinto de sí mismo. Esto puede expresarse como una sentencia de la lógica de primer orden. Para simplificar esta sentencia, sea $\varphi(x)$ la fórmula $\forall y (\neg(x = y) \rightarrow R(x, y))$. Para cualquier grafo G y cualquier vértice a de G , $G \models \varphi(a)$ si y solo si a es adyacente a todo vértice de G distinto de a mismo. La siguiente sentencia dice que existe un único elemento así:

$$\exists y \varphi(y) \wedge \forall z (\varphi(z) \rightarrow (z = y)).$$

Esta sentencia distingue al Grafo 1 de los Grafos 2 a 4.

El Grafo 4, por otro lado, se caracteriza por la siguiente sentencia, que dice que $\varphi(x)$ se cumple para todo vértice x .

$$\forall x \forall y (\neg(x = y) \rightarrow R(x, y)).$$

A cualquier grafo que modele esta sentencia se le llama una *clica* (o *grafo completo*). A la clica que tiene n vértices se le llama la n -clica y se denota por K_n . Así, el Grafo 4 es la 8-clica K_8 . Obsérvese que, cuando se especifica n , usamos el artículo definido al referirnos a la n -clica. Esto se debe a que dos n -clicas cualesquiera son esencialmente la misma. Más precisamente, son isomorfas.

Definición 2.14: Isomorfismo de grafos

Se dice que los grafos G_1 y G_2 son *isomorfos* si existe una correspondencia biunívoca f del conjunto de vértices de G_1 sobre el conjunto de vértices de G_2 tal que, para cualesquiera vértices a y b de G_1 , a y b son adyacentes en G_1 si y solo si $f(a)$ y $f(b)$ son adyacentes en G_2 . A tal función f se le llama un *isomorfismo*.

Los grafos isomorfos son esencialmente el mismo.

Ejemplo 2.18

Considere los siguientes dos grafos.

Grafo G :

Vértices: a, b, c, d

Aristas: ab, bc, cd, ad .

Grafo H :

Vértices: w, x, y, z

Aristas: wx, wy, xz, yz .

La función f definida por

$$f(a) = w, \quad f(b) = x, \quad f(c) = z, \quad \text{y} \quad f(d) = y$$

es un isomorfismo de G sobre H . Ambos grafos pueden representarse como cuadrados. La única diferencia entre G y H son las letras usadas para representar los vértices. ▲

Hemos exhibido una $\mathcal{V}_G$ -sentencia que distingue al Grafo 1 del Grafo 4. Podemos hacer algo mucho mejor que esto. Existe una $\mathcal{V}_G$ -sentencia que distingue al Grafo 1 de todos los grafos que no son isomorfos al Grafo 1. Es decir, existe una $\mathcal{V}_G$ -sentencia φ_G tal que, para cualquier grafo H , $H \models \varphi_G$ si y solo si H es isomorfo a G . Demostramos esto en la Sección 2.6 como Proposición 2.17. En este sentido, la lógica de primer orden es un lenguaje poderoso para describir grafos finitos.

En otro sentido, sin embargo, la lógica de primer orden no es un lenguaje poderoso. Propiedades básicas de la teoría de grafos no pueden expresarse usando lógica de primer orden. Por ejemplo, no existe ninguna sentencia de primer orden que diga que un grafo tiene un número par de vértices. Tampoco puede la lógica de primer orden decir que un grafo es conexo. Recordemos que la sentencia $\forall x \exists y R(x, y)$ se cumple en cualquier grafo conexo que tenga más de un vértice. Sin embargo, el hecho de que esta sentencia se cumpla en una estructura no significa que sea conexa. No existe una $\mathcal{V}_G$ -sentencia φ tal que $G \models \varphi$ si y solo si G es un grafo conexo. Estas y otras limitaciones de la lógica de primer orden se discuten en la Sección 4.7.

2.4.2. Bases de datos relacionales

Las bases de datos relacionales proporcionan ejemplos concretos de estructuras. Cualquier colección de datos puede verse como una base de datos, ya sea un directorio telefónico, un catálogo de discos, o un árbol genealógico. Una base de datos relacional se presenta como un conjunto de tablas. Por ejemplo, las tres tablas siguientes forman una base de datos relacional (Tablas 2.1 a 2.3).

Describimos ahora una estructura D que representa esta base de datos relacional. El conjunto subyacente de D consiste en todos los elementos que aparecen como entrada en alguna columna de una tabla. Así, este conjunto contiene 13 nombres y cuatro fechas.

El vocabulario $\mathcal{V}$ de D consiste en una relación n -aria por cada tabla, donde n es el número de columnas de la tabla. Es decir, el vocabulario contiene una relación unaria F y relaciones binarias P y B correspondientes a las tablas de Mujeres, Padres y Cumpleaños.

La $\mathcal{V}$ -estructura D interpreta estas relaciones como filas de las tablas. Por ejemplo, $D \models B(a, b)$ si y solo si " ab " es una fila de la tabla de Cumpleaños. Esto describe por completo la $\mathcal{V}$ -estructura D . Por ejemplo, vemos que

$$\begin{aligned} D \models F(\text{Dot}), \quad D \models P(\text{Zelda}, \text{Max}), \quad \text{y} \\ D \models \neg B(\text{Zelda}, \text{28 de julio}). \end{aligned}$$

Además de B , F y P , podemos definir fórmulas de primer orden que expresan otras relaciones diversas

Tabla 2.1 Tabla de padres

Padre/Madre	Hijo/Hija
Ray	Ken
Ray	Sue
Sue	Tim
Dot	Jim
Bob	Jim
Bob	Liz
Jim	Tim
Sue	Sam
Jim	Sam
Zelda	Max
Sam	Max

Tabla 2.2 Tabla de mujeres

Mujeres
Dot
Zelda
Liz
Sue

Tabla 2.3 Tabla de cumpleaños

Persona	Cumpleaños
Ann	5 de agosto
Leo	8 de agosto
Max	28 de julio
Sam	1 de agosto
Sue	24 de julio

en D . Por ejemplo, la fórmula $\neg F(x)$ dice que x es hombre. La fórmula $\exists z(P(x, z) \wedge P(z, y))$ dice que x es abuelo o abuela de y . La conjunción de esta fórmula con $F(x)$ dice que x es abuela de y . La fórmula $\exists y B(x, y)$ dice que x es una fecha, y la negación de esta fórmula dice que x es una persona. La fórmula $\exists z(B(x, z) \wedge B(y, z))$ afirma que x y y cumplen años el mismo día. No hay fin a las relaciones que pueden definirse (véase el Ejercicio 2.2).

Volvemos a este ejemplo al final del Capítulo 3, donde discutimos Prolog. Prolog es un lenguaje de programación basado en la lógica de Horn de primer orden que puede usarse para presentar y consultar cualquier base de datos relacional.

2.4.3. Órdenes lineales

A continuación, examinamos algunas estructuras en el vocabulario $\mathcal{V}_<$ que consiste únicamente en la relación binaria $<$. En lugar de usar la notación " $< (x, y)$," usamos la más familiar " $x < y$ " para expresar que la relación binaria $<$ se cumple para el par ordenado (x, y) . Como indica nuestra elección de símbolos, cada una de las estructuras que consideramos interpreta $<$ como "menor que."

Consideramos cuatro $\mathcal{V}_<$ -estructuras denotadas por $\mathbb{N}_<$, $\mathbb{Z}_<$, $\mathbb{Q}_<$ y $\mathbb{R}_<$. Para definir cada estructura, debemos indicar cuál es el conjunto subyacente y cómo se interpretan los símbolos. Los conjuntos subyacentes de las estructuras anteriores son, en orden, los números naturales, los enteros, los números racionales y los números reales. Cada una de estas estructuras interpreta $<$ de la manera usual. Podemos presentar estas estructuras de forma más concisa como sigue:

- $\mathbb{N}_< = (\mathbb{N} \mid <)$
- $\mathbb{Z}_< = (\mathbb{Z} \mid <)$
- $\mathbb{Q}_< = (\mathbb{Q} \mid <)$
- $\mathbb{R}_< = (\mathbb{R} \mid <)$.

Estas cuatro estructuras tienen mucho en común. Todas son $\mathcal{V}_<$ -estructuras y cada una de ellas modela las siguientes $\mathcal{V}_<$ -sentencias:

$$\begin{aligned} & \forall x \forall y ((x < y) \rightarrow \neg(y < x)) \\ & \forall x (\neg(x < x)) \\ & \forall x \forall y ((x < y) \vee (y < x) \vee (x = y)) \\ & \forall x \forall y \forall z (((x < y) \wedge (y < z)) \rightarrow (x < z)). \end{aligned}$$

Tomadas en conjunto, estas sentencias dicen que $<$ ordena linealmente al conjunto subyacente. Cada una de las cuatro estructuras modela cada una de estas cuatro sentencias. Sin embargo, esto no es cierto para todas las $\mathcal{V}_<$ -sentencias. Sea φ la sentencia $\forall x \exists y (y < x)$, que dice que no hay un elemento más pequeño. Claramente, $\mathbb{R}_<$, $\mathbb{Q}_<$ y $\mathbb{Z}_<$ son modelos de φ . Sin embargo, $\mathbb{N}_<$ sí tiene un elemento más pequeño, a saber, 1. Así, $\mathbb{N}_<$ no modela φ ; más bien, $\mathbb{N}_<$ modela la sentencia $\exists x \forall y \neg(y < x)$, que afirma que existe un elemento más pequeño. Llamemos θ a esta sentencia. Obsérvese que θ es equivalente a $\neg\varphi$. La sentencia θ distingue a $\mathbb{N}_<$ de los otros tres modelos.

A continuación, busquemos una sentencia de primer orden que distinga a $\mathbb{Z}_<$ de las otras tres estructuras. Observemos que $\mathbb{Z}_<$ no tiene un elemento más pequeño y no es densa. Un conjunto linealmente ordenado es *denso* si entre cualesquiera dos elementos hay otro elemento. Esta propiedad puede expresarse en lógica de primer orden mediante la siguiente $\mathcal{V}_<$ -sentencia

$$\forall x \forall y ((x < y) \rightarrow \exists z ((x < z) \wedge (z < y))).$$

Llamemos δ a esta sentencia. Tanto $\mathbb{Q}_<$ como $\mathbb{R}_<$ modelan δ . Entre cualesquiera dos números racionales a y b existen infinitos números racionales $[(a+b)/2]$, por ejemplo. Lo mismo es cierto para los números reales. Sin embargo, los enteros no son densos. Entre 1 y 2 no hay otros enteros. Así, $\mathbb{Z}_< \models \neg\delta$. La $\mathcal{V}_<$ -sentencia $\varphi \wedge \neg\delta$ distingue a $\mathbb{Z}_<$ de las otras tres estructuras.

Supongamos ahora que queremos distinguir entre $\mathbb{Q}_<$ y $\mathbb{R}_<$. Podríamos usar el hecho de que $\mathbb{R}_<$ es más grande que $\mathbb{Q}_<$. (En la siguiente sección discutimos en detalle el tamaño de una estructura y mostramos que, en cierto sentido preciso, hay más números reales que números racionales.) Otra característica distintiva es la completitud del orden. Un orden lineal es *completo en el orden* si no puede partirse en dos intervalos abiertos. El conjunto de los números racionales, por ejemplo, es la unión de los intervalos $(-\infty, \sqrt{2})$ y $(\sqrt{2}, \infty)$. Los paréntesis "(" y ")" indican que los intervalos no contienen los extremos (esto es lo que queremos decir con "abierto"). Dado que $\sqrt{2}$ no es un número racional, todo número racional está en uno de estos dos intervalos. Así, $\mathbb{Q}_<$ no tiene completitud en el orden. La estructura $\mathbb{R}_<$, por otro lado, sí tiene completitud en el orden. Esta es una característica distintiva de los números reales.

Sin embargo, si intentamos hallar una $\mathcal{V}_<$ -sentencia que distinga a $\mathbb{R}_<$ de $\mathbb{Q}_<$, fracasaremos. Para toda $\mathcal{V}_<$ -sentencia φ , $\mathbb{R}_< \models \varphi$ si y solo si $\mathbb{Q}_< \models \varphi$. Damos una demostración elemental de esto en la Sección 5.2. Nuestro lenguaje de primer orden es demasiado débil para expresar cualquier diferencia entre estas estructuras. Señalamos que $\mathbb{R}_<$ es completa en el orden mientras que $\mathbb{Q}_<$ no lo es, pero no podemos expresar esto mediante una sentencia de primer orden (inténtelo). Más bien, la completitud del orden es un concepto de segundo orden. En lógica de segundo orden podemos expresar cosas como "no existen dos subconjuntos tales que..." También señalamos que $\mathbb{R}$ es más grande que $\mathbb{Q}$, pero, como veremos en el Capítulo 4, la lógica de primer orden no puede distinguir entre un número infinito y otro. Tanto $\mathbb{Q}_<$ como $\mathbb{R}_<$ son infinitas, y eso es todo lo que la lógica de primer orden puede decir. Desde el punto de vista de las $\mathcal{V}_<$ -sentencias, las estructuras $\mathbb{R}_<$ y $\mathbb{Q}_<$ son idénticas.

2.4.4. Sistemas numéricos

Aunque la lógica de primer orden no puede distinguir entre las $\mathcal{V}_<$ -estructuras $\mathbb{Q}_<$ y $\mathbb{R}_<$, sí puede distinguir entre los números reales y los números racionales en vocabularios distintos de $\mathcal{V}_<$. Consideremos el vocabulario de la aritmética $\{+, \cdot, 0, 1\}$, con funciones binarias $+$ y $\cdot$, y constantes 0 y 1. Sea $\mathcal{V}_{ar}$ este vocabulario, y consideremos las siguientes $\mathcal{V}_{ar}$ -estructuras:

- $A = (\mathbb{Z} \mid +, \cdot, 0, 1)$,
- $Q = (\mathbb{Q} \mid +, \cdot, 0, 1)$,
- $R = (\mathbb{R} \mid +, \cdot, 0, 1)$,

- $C = (\mathbb{C} \mid +, \cdot, 0, 1)$.

Los conjuntos subyacentes de estas estructuras son, en orden, los enteros, los números racionales, los números reales y los números complejos. Cada una de estas estructuras interpreta los símbolos de $\mathcal{V}_{ar}$ de la manera usual.

En lugar de usar la notación formal " $+(x, y) = z$," usamos la más convencional " $x + y = z$." Del mismo modo, escribimos $x \cdot y$ en lugar de $\cdot(x, y)$. Dejamos que 2 abrevie $(1 + 1)$, x^2 abrevie $x \cdot x$, y así sucesivamente. Cualquier polinomio que tenga números naturales como coeficientes es un $\mathcal{V}_{ar}$ -término. Ecuaciones como (por ejemplo)

$$x^5 - 9x + 3 = 0$$

son $\mathcal{V}_{ar}$ -fórmulas. De nuevo, 3 y x^5 no son símbolos de $\mathcal{V}_{ar}$; son abreviaturas de los $\mathcal{V}_{ar}$ -términos $(1 + (1 + 1))$ y $x \cdot (x \cdot (x \cdot (x \cdot x)))$, respectivamente.

Todavía no podemos expresar la completitud del orden en este vocabulario, pero sí podemos distinguir entre las estructuras R y Q . La $\mathcal{V}_{ar}$ -sentencia $\exists x(x^2 = 1 + 1)$ afirma la existencia de $\sqrt{2}$. Se sigue que R modela esta sentencia y Q no. Del mismo modo, la ecuación $2x + 3 = 0$ tiene solución en Q pero no en A . Así, $Q \models \exists x(2x + 3 = 0)$, mientras que $A \models \neg \exists x(2x + 3 = 0)$.

Para progresar de $\mathbb{N}$ a $\mathbb{Z}$, a Q , agregamos soluciones para cada vez más polinomios. Llegamos al final del camino con los números complejos C . Los números complejos se obtienen agregando a los reales la solución $i = \sqrt{-1}$ de la ecuación $x^2 + 1 = 0$. La $\mathcal{V}_{ar}$ -sentencia $\exists x(x^2 + 1 = 0)$ distingue a C de las demás estructuras de nuestra lista. El conjunto $\mathbb{C}$ consiste en todos los números de la forma $a + bi$ donde a y b son ambos números reales. El Teorema Fundamental del Álgebra establece que, para cualquier polinomio no constante $P(x)$ con coeficientes en $\mathbb{C}$, la ecuación $P(x) = 0$ tiene solución en $\mathbb{C}$ (esto es cierto incluso para polinomios de más de una variable). Así, no hay necesidad de extender a un sistema numérico más grande. En virtud de agregar una solución de $x^2 + 1 = 0$ a $\mathbb{R}$, hemos agregado una solución para todo polinomio.

Los nombres de estos sistemas numéricos reflejan sesgos históricos. Los números de conteo $1, 2, 3, \dots$ son los números "naturales" a considerar en matemáticas. Los números negativos no son naturales, la raíz cuadrada de 2 es irracional, y la raíz cuadrada de -1 es imaginaria. Los nombres sugieren que las cosas se vuelven más complicadas a medida que avanzamos de los números "naturales" a los números "complejos." Desde el punto de vista de la lógica de primer orden, sin embargo, esto está al revés. La estructura C es la más simple. La estructura R no es simple como C , pero sí tiene muchas propiedades deseables. Discutiremos las propiedades de estas dos estructuras en el Capítulo 5. La estructura A no es tan agradable. La " A " representa "aritmética," lo cual suena bastante elemental. Sin embargo, desde el punto de vista de la lógica de primer orden, A es la más compleja. Investigamos la estructura A en el Capítulo 8.

2.5 El tamaño de una estructura

Para cualquier conjunto U , $|U|$ denota el número de elementos de U . Para una $\mathcal{V}$ -estructura M , $|M|$ significa $|U_M|$, el número de elementos del conjunto subyacente U_M de M . Nos referimos a $|M|$ como el *tamaño* de M . Por ejemplo, si M_2 es el Grafo 2 de la Sección 2.4.1, entonces $|M_2| = 5$. Si el conjunto subyacente de M es infinito, podríamos simplemente escribir $|M| = \infty$ y no decir más, pero

esto simplifica excesivamente la situación. Implica que dos conjuntos infinitos cualesquiera tienen el mismo tamaño. Este no es el caso. Para explicar esto, necesitamos decir con precisión qué queremos decir con "el mismo tamaño."

Sean A y B dos conjuntos finitos. Imagine cada conjunto como una caja de pelotas de ping-pong. Imagine meter la mano izquierda en la caja A y la mano derecha en la caja B , y sacar una pelota de cada una. Repita este proceso. Meta las manos en las cajas y simultáneamente saque una pelota de cada una, y de nuevo, y de nuevo. Eventualmente, una de las cajas se vacía. Si la caja B se vacía primero, entonces concluimos que la caja A debía contener al menos tantas pelotas como la caja B al principio. Es decir, $|B| \leq |A|$. Dado que A y B son finitos, esto es elemental. Para conjuntos infinitos tomamos esta idea como la definición de " $|B|$ es menor o igual que $|A|$."

Definición 2.15

Sean A y B conjuntos. Definimos " $|B| \leq |A|$ " de la siguiente manera: $|B| \leq |A|$ si existe una función inyectiva f de B en A .

La función de esta definición desempeña el mismo papel que nuestras manos derecha e izquierda en la discusión anterior. La definición exige que f sea inyectiva y tenga dominio B . Dado cualquier elemento b de B , la función "elige" un elemento $f(b)$ de A . Si existe tal función, concluimos que $|B| \leq |A|$.

Ejemplo 2.19

Sea P el conjunto de todos los números naturales primos y sea E el conjunto de todos los números naturales pares. Sea $f : P \rightarrow E$ definida por $f(p) = 2p$. Esta función es inyectiva. Concluimos que $|P| \leq |E|$. Es decir, hay al menos tantos números pares como números primos. ▲

Ejemplo 2.20

Recordemos que $\mathbb{N} \times \mathbb{N}$ denota el conjunto de todos los pares ordenados (m, n) de números naturales. Sea $f : \mathbb{N} \rightarrow \mathbb{N} \times \mathbb{N}$ definida por $f(n) = (n, 1)$ para todo $n \in \mathbb{N}$. Esta función es inyectiva. Concluimos que $|\mathbb{N}| \leq |\mathbb{N} \times \mathbb{N}|$. Al lector no debería sorprenderle este hecho. Menos evidente es que lo contrario también es cierto. Considere la función $g : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ definida por $g(m, n) = 2^m 3^n$. Esta también es una función inyectiva. Así, no solo $|\mathbb{N}|$ es menor o igual que $|\mathbb{N} \times \mathbb{N}|$, sino que también $|\mathbb{N} \times \mathbb{N}|$ es menor o igual que $|\mathbb{N}|$. Naturalmente, concluimos que estos dos conjuntos tienen el mismo tamaño. ▲

Definición 2.16

Sean A y B conjuntos. Decimos que A y B tienen el *mismo tamaño* y escribimos $|A| = |B|$ si tanto $|B| \leq |A|$ como $|A| \leq |B|$. Escribimos $|A| < |B|$ si $|A| \leq |B|$ y no es el caso que $|A| = |B|$.

Así pues, para mostrar que dos conjuntos A y B tienen el mismo tamaño, debemos exhibir una función inyectiva de A a B y una función inyectiva de B a A . Basta con mostrar que existe una

función f de A a B (o de B a A) que sea tanto inyectiva como sobreyectiva (dado que f^{-1} también es inyectiva y sobreyectiva). A tal función se le llama una *correspondencia biunívoca* o *biyección*.

Ejemplo 2.21

Sea $\mathbb{N}$ el conjunto de los números naturales y sea de nuevo E el conjunto de los números naturales pares. En cierto sentido hay "más" números naturales que números pares (dado que $E \subset \mathbb{N}$). Sin embargo, estos dos conjuntos tienen el mismo tamaño. Esto lo atestigua la función $f(x) = 2x$, que define una biyección de $\mathbb{N}$ sobre E . ▲

Ejemplo 2.22

Sea $\mathbb{R}$ los números reales y sea I el intervalo $(0, 1)$, el conjunto de todos los reales entre 0 y 1. La función $f : \mathbb{R} \rightarrow I$ definida por $f(x) = (2/\pi) \arctan x$ es una biyección de $\mathbb{R}$ sobre I . Así, $|\mathbb{R}| = |I|$. ▲

Si los conjuntos A y B pueden ponerse en correspondencia biunívoca entre sí, entonces deben tener el mismo tamaño. El siguiente teorema afirma que el recíproco también es cierto. Si $|A| \leq |B|$ y $|B| \leq |A|$, entonces debe existir una biyección entre A y B . Esto proporciona una definición alternativa de "mismo tamaño."

Teorema 2.1

Los conjuntos A y B tienen el mismo tamaño si y solo si existe una biyección de A sobre B .

Demostración. Solo una dirección requiere demostración. Como ya señalamos, si existe una biyección entre A y B , entonces A y B deben tener el mismo tamaño. Demostramos ahora lo contrario: si $|A| = |B|$, entonces tal biyección necesariamente existe.

Supongamos que A y B tienen el mismo tamaño. Por la definición de "mismo tamaño," existen funciones inyectivas $f : A \rightarrow B$ y $g : B \rightarrow A$. Nuestro objetivo es exhibir una biyección $h : A \rightarrow B$. Antes de definir h , definimos algunas sucesiones.

Dado cualquier $a \in A$, definimos una sucesión (posiblemente finita) s_a de la siguiente manera. Sea $a_1 = a$. Supongamos ahora que $a_m \in A$ se ha definido para algún $m \in \mathbb{N}$. Tomemos $b_m \in B$ tal que $g(b_m) = a_m$. Si no existe tal b_m , entonces la sucesión termina. En caso contrario, si b_m sí existe, la sucesión continúa. Tomemos $a_{m+1} \in A$ tal que $f(a_{m+1}) = b_m$. De nuevo, si no existe tal a_{m+1} , la sucesión termina. Obsérvese que la sucesión alterna entre elementos de A y elementos de B . La sucesión s_a puede representarse de la siguiente manera:

$$a_1 \xleftarrow{g} b_1 \xleftarrow{f} a_2 \xleftarrow{g} b_2 \xleftarrow{f} a_3 \xleftarrow{g} b_3 \dots$$

Hay tres posibilidades para la sucesión s_a . O bien termina en algún elemento $a_i \in A$, o bien termina en algún elemento $b_i \in B$, o bien nunca termina. Estas tres posibilidades dividen al conjunto A en tres subconjuntos.

- Sea A_A el conjunto de todos los $a \in A$ tales que s_a termina en A .

- Sea A_B el conjunto de todos los $a \in A$ tales que s_a termina en B .
- Sea A_N el conjunto de todos los $a \in A$ tales que s_a nunca termina.

De manera similar, podemos definir sucesiones s_b que comienzan con $b \in B$ y dividen a B de la siguiente manera:

- Sea B_A el conjunto de todos los $b \in B$ tales que s_b termina en A .
- Sea B_B el conjunto de todos los $b \in B$ tales que s_b termina en B .
- Sea B_N el conjunto de todos los $b \in B$ tales que s_b nunca termina.

La función f , restringida a A_A , es una biyección $f : A_A \rightarrow B_A$. Sabemos que f es inyectiva. Para ver que es sobreyectiva, tomemos cualquier $b \in B_A$. Dado que la sucesión s_b termina en A , debe existir $a \in A_A$ tal que $f(a) = b$. (En caso contrario, s_b sería la sucesión de un solo elemento b). Del mismo modo, g , restringida a B_B , forma una biyección $g : B_B \rightarrow A_B$. Finalmente, A_N y B_N están en correspondencia biunívoca tanto por g como por f . Una biyección $h : A \rightarrow B$ puede definirse ahora uniendo estas tres partes.

$$h(a) = \begin{cases} f(a), & a \in A_A \\ g^{-1}(a), & a \in A_B \\ f(a), & a \in A_N \end{cases}$$

■

Para conjuntos finitos, el Teorema 2.1 es elemental. Para determinar cuántas pelotas de ping-pong hay en una caja dada, ponemos las pelotas de ping-pong en correspondencia biunívoca con el conjunto $\{1, 2, 3, \dots, k\}$ para algún $k \in \mathbb{N}$ (es decir, las contamos). Decimos que dos cajas contienen el mismo número de pelotas de ping-pong si cada una puede ponerse en correspondencia biunívoca con el mismo conjunto $\{1, 2, 3, \dots, k\}$ y, por lo tanto, entre sí. Si A y B son infinitos, podemos tener dificultad para visualizarlos como cajas de pelotas de ping-pong. Extrapolamos nuestras definiciones para conjuntos infinitos a partir de las definiciones correspondientes para conjuntos finitos. Además, empleamos la siguiente suposición.

Suposición: Si A y B son conjuntos, entonces $|A| \leq |B|$ o $|B| \leq |A|$.

Para A y B finitos, esta suposición es un hecho que puede demostrarse. Si sacamos pelotas de ping-pong, una a la vez, de cada una de dos cajas dadas, eventualmente una (o ambas) de las cajas se vaciará. Debemos tener cuidado, sin embargo, al manejar cajas que contienen infinitas pelotas de ping-pong (véase el Ejercicio 2.43). Para A y B infinitos, aceptamos esta suposición sin demostración. Es equivalente a un axioma de las matemáticas conocido como el Axioma de Elección.

Se sigue de esta suposición que, para cualquier conjunto infinito A , $|\mathbb{N}| \leq |A|$. Esto da lugar a una dicotomía crucial de los conjuntos infinitos: o bien $|\mathbb{N}| = |A|$ o bien $|\mathbb{N}| < |A|$.

Definición 2.17

Un conjunto A es *numerable* si existe una biyección entre A y $\mathbb{N}$.

Definición 2.18

Un conjunto A es *contable* si es finito o numerable. En caso contrario, A es *no contable*.

Proposición 2.1

El conjunto de los números racionales $\mathbb{Q}$ es contable.

Demostración. Claramente, $|\mathbb{N}| \leq |\mathbb{Q}|$ (dado que $\mathbb{N} \subset \mathbb{Q}$). Recíprocamente, cada elemento no nulo de $\mathbb{Q}$ puede escribirse de manera única como una fracción reducida de números naturales multiplicada por $(-1)^m$ para $m = 1$ o 2 . Sea $f : \mathbb{Q} \rightarrow \mathbb{N}$ definida por $f\left(\frac{a}{b}(-1)^m\right) = 2^a 3^b 5^m$ donde $\frac{a}{b}$ está reducida. Además, sea $f(0) = 0$. Entonces f es una función inyectiva de $\mathbb{Q}$ en $\mathbb{N}$. Por definición, $|\mathbb{Q}| \leq |\mathbb{N}|$. Por lo tanto, $\mathbb{Q}$ y $\mathbb{N}$ tienen el mismo tamaño. ■

De manera similar, mostramos en el Ejemplo 2.20 que $\mathbb{N} \times \mathbb{N}$ tiene el mismo tamaño que $\mathbb{N}$. Así, $\mathbb{N} \times \mathbb{N}$ es un conjunto contable. Usamos esto para demostrar el siguiente hecho útil.

Proposición 2.2

La unión de una cantidad contable de conjuntos contables es contable.

Demostración. Para cada $n \in \mathbb{N}$, sea A_n un conjunto contable. Sea U la unión de estos conjuntos. Si cada A_n es numerable y son disjuntos entre sí, entonces U es lo más grande posible. Supongamos que este es el caso. Así, cada A_n puede enumerarse como $\{a_1, a_2, a_3, \dots\}$. Sea $f(m, n)$ el m -ésimo elemento de la enumeración de A_n . Esto define una biyección $f : \mathbb{N} \times \mathbb{N} \rightarrow U$. Concluimos que U tiene el mismo tamaño que $\mathbb{N} \times \mathbb{N}$. Dado que $\mathbb{N} \times \mathbb{N}$ es contable, también lo es U . ■

Un ejemplo de un conjunto no contable lo proporciona el conjunto de todos los subconjuntos de $\mathbb{N}$. Para cualquier conjunto A , el conjunto de todos los subconjuntos de A se llama el *conjunto potencia* de A , denotado por $\mathcal{P}(A)$. Mostramos que $|\mathcal{P}(A)|$ es siempre estrictamente mayor que $|A|$.

Proposición 2.3

Para cualquier conjunto A , $|A| < |\mathcal{P}(A)|$.

Demostración. Para mostrar que $|A| < |\mathcal{P}(A)|$ debemos mostrar tanto que $|A| \leq |\mathcal{P}(A)|$ como que $|A| \neq |\mathcal{P}(A)|$.

La función inyectiva $f : A \rightarrow \mathcal{P}(A)$ definida por $f(a) = \{a\}$ (para cada $a \in A$) muestra que $|A| \leq |\mathcal{P}(A)|$.

Para mostrar que $|A| \neq |\mathcal{P}(A)|$, debemos mostrar que no existe una biyección entre A y $\mathcal{P}(A)$. Sea g una función inyectiva arbitraria de A a $\mathcal{P}(A)$. Mostramos que g necesariamente no es sobreyectiva. (Obsérvese que la función inyectiva f anterior tampoco es sobreyectiva.) Para cada elemento a de A , o bien a está en el conjunto $g(a)$, o bien a no está en $g(a)$. Sea X el conjunto de aquellos elementos a de A para los cuales a no está en $g(a)$. Entonces $a \in X$ si y solo si $a \notin g(a)$. Para cada $a \in A$, no puede ser el caso que $g(a) = X$ (de lo contrario tendríamos $a \in X$ si y solo si $a \notin X$, lo cual es absurdo). Dado que X no está en el rango de g , g no es sobreyectiva. Dado que g era arbitraria, concluimos que ninguna función inyectiva de A a $\mathcal{P}(A)$ es sobreyectiva. ■

Corolario 2.1

Todo conjunto numerable tiene una cantidad no contable de subconjuntos.

En particular, hay una cantidad no contable de subconjuntos de $\mathbb{N}$. Usamos este hecho para mostrar que hay una cantidad no contable de números reales.

Proposición 2.4

El conjunto de los números reales $\mathbb{R}$ es no contable.

Demostración. Definimos una función inyectiva f de $\mathcal{P}(\mathbb{N})$ en $\mathbb{R}$.

Sea X un elemento de $\mathcal{P}(\mathbb{N})$. Entonces, como subconjunto de los números naturales, X contiene a lo sumo 10 números de un dígito, a lo sumo 90 números de dos dígitos, a lo sumo 900 números de tres dígitos, y así sucesivamente. Sea r_X el número real entre 0 y 1 descrito de la siguiente manera. Los primeros dos dígitos después del punto decimal representan el número de números de un dígito en X . A estos les siguen cada uno de los números de un dígito de X , enumerados en orden ascendente. Los siguientes dos dígitos en la expansión decimal de r_X representan el número de números de dos dígitos en X . A estos les siguen la lista de los números de dos dígitos de X . Los siguientes tres dígitos indican cuántos números de tres dígitos hay en X , y así sucesivamente.

Por ejemplo, sea $X = \{2, 4, 5, 6, 7, 8, 9, 10, 24, 213, 3246\}$. Hay 07 números de un dígito en X (a saber, 2, 4, 5, 6, 7, 8 y 9), hay 02 números de dos dígitos (a saber, 10 y 24), hay 001 número de tres dígitos (213), y 0001 número de cuatro dígitos (3246). Así, tenemos

$$r_X = 0,0724567890210240012130001324600000000 \dots$$

El número r_X contiene una descripción completa del conjunto X . Se sigue que la función $f : \mathcal{P}(\mathbb{N}) \rightarrow \mathbb{R}$ definida por $f(X) = r_X$ es una función inyectiva. Por lo tanto, $|\mathcal{P}(\mathbb{N})| \leq |\mathbb{R}|$. Dado que $\mathcal{P}(\mathbb{N})$ es no contable, también lo es $\mathbb{R}$. ■

A continuación mostramos que solo hay una cantidad contable de $\mathcal{V}$ -fórmulas para cualquier vocabulario contable $\mathcal{V}$.

Proposición 2.5

Si el vocabulario $\mathcal{V}$ es contable, entonces también lo es el conjunto de todas las $\mathcal{V}$ -fórmulas.

Demostración. Definimos una función inyectiva f del conjunto de todas las $\mathcal{V}$ -fórmulas en $\mathbb{N}$.

Dado que $\mathcal{V}$ es contable, podemos asignar un número natural distinto a cada símbolo que ocurre en una $\mathcal{V}$ -fórmula. Entonces, a cada $\mathcal{V}$ -fórmula le corresponde una sucesión finita de números naturales. Supongamos que una $\mathcal{V}$ -fórmula dada φ tiene a $a_1, a_2, \dots, a_m$ como su sucesión asociada de números naturales. Definimos $f(\varphi)$ como el producto

$$2^{a_1} \cdot 3^{a_2} \cdot 5^{a_3} \cdots p_n^{a_n}$$

donde p_n denota el n -ésimo número primo. Recordemos dos hechos básicos sobre los números naturales: hay infinitos números primos y hay una única manera de factorizar cualquier número natural dado en primos. Así, podemos factorizar el número natural $f(\varphi)$ para recuperar la sucesión $a_1, \dots, a_n$ y la fórmula φ . Se sigue que f es una función inyectiva, como se requería. ■

Por la Proposición 2.5, la mayoría de los subconjuntos de $\mathbb{N}$ no son definibles en ningún vocabulario contable. La misma idea usada para demostrar la Proposición 2.5 puede usarse para mostrar que hay una cantidad contable de sentencias en español, o en cualquier otro idioma natural. Así, existen incontables números reales que eluden descripción en cualquier idioma natural. Del mismo modo, existen incontables subconjuntos de los números naturales que no pueden definirse. La siguiente proposición muestra que esto también es cierto para las funciones sobre los números naturales.

Proposición 2.6

El conjunto de todas las funciones de $\mathbb{N}$ en $\mathbb{N}$ es no contable.

Demostración. Sea F el conjunto de todas las funciones de $\mathbb{N}$ en $\mathbb{N}$. Mostramos que $|I| \leq |F|$. Recordemos que I es el intervalo $(0, 1)$ que consiste en los números reales entre 0 y 1. Por el Ejemplo 2.22, I y $\mathbb{R}$ tienen el mismo tamaño. Por la proposición anterior, I es no contable. Sea r un elemento arbitrario de I . Sea $f_r : \mathbb{N} \rightarrow \mathbb{N}$ definida dejando que $f_r(n)$ sea el n -ésimo dígito en la expansión decimal de r . Claramente, si r_1 y r_2 son números distintos en I , entonces f_{r_1} y f_{r_2} son funciones distintas. Por lo tanto, la función que asigna f_r a la entrada r es una función inyectiva de I a F . Se sigue que $|I| \leq |F|$ y F es no contable. (De hecho, hemos mostrado que existen incontables funciones de $\mathbb{N}$ al conjunto $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$.) ■

Se dice que una función $f(x)$ es *computable* si existe un programa de computadora que produce $f(x)$ al recibir x como entrada. Aplicando la Proposición 2.5 a los lenguajes de programación, vemos que solo hay una cantidad contable de posibles programas de computadora. Se sigue que existen incontables funciones de $\mathbb{N}$ en $\mathbb{N}$ que no pueden computarse. Esto también es cierto para las funciones sobre los reales. La mayoría de las funciones no son computables. Este hecho desafía la evidencia empírica. La mayoría de las funciones con las que estamos familiarizados (la mayoría de las funciones que uno

encuentra en cálculo, digamos) son computables. La noción de computabilidad se discute en detalle en el Capítulo 7. En la Sección 7.6.1 daremos ejemplos de funciones que están definidas con precisión pero que no son computables.

Al comienzo de esta sección dijimos que tener una sola noción de "infinito" es engañoso. Hemos sustituido esto por dos nociones. Un conjunto infinito es o bien contable o bien no contable. Muchos de los conjuntos infinitos que encontramos tienen el mismo tamaño que $\mathbb{N}$ o el mismo tamaño que $\mathbb{R}$. (Tanto $\mathcal{P}(\mathbb{N})$ como F tienen el mismo tamaño que $\mathbb{R}$; véanse los Ejercicios 2.41 y 2.42.) Esta dicotomía sigue siendo tosca. La Proposición 2.3 garantiza la existencia de conjuntos no contables arbitrariamente grandes, así que tener una sola noción de "no contable" resulta ahora engañoso. En la Sección 4.2 introducimos los números cardinales para representar el tamaño de un conjunto y estudiamos con más profundidad la plétora de números no contables. Por ahora, terminamos nuestra digresión hacia el infinito y volvemos a nuestra discusión sobre las estructuras.

2.6 Relaciones entre estructuras

Consideramos ciertas relaciones que pueden o no darse entre dos estructuras de un mismo vocabulario.

2.6.1. Encajes

Sean M y N estructuras. La notación $f : M \rightarrow N$ se usa para denotar " f es una función de M en N ." Al usar esta notación, se entiende que f no es un símbolo de los vocabularios de M o de N . Cada función unaria del vocabulario de M se interpreta como una función del universo de M en sí mismo. Cuando hablamos de una función de M en N , en realidad nos referimos a una función del conjunto subyacente de M al conjunto subyacente de N . Es decir, a cada elemento a del universo U_M de M , f le asigna un elemento $f(a)$ del universo U_N de N . Nos interesa principalmente el caso en que, para algún vocabulario $\mathcal{V}$, M y N son ambas $\mathcal{V}$ -estructuras y f *preserva* ciertas $\mathcal{V}$ -fórmulas.

Definición 2.19: Preservación

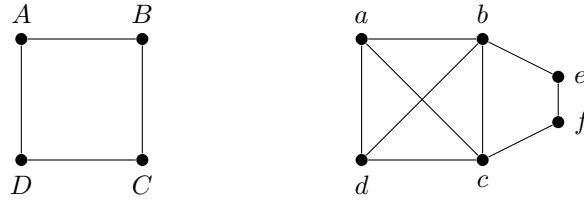
Sea $\mathcal{V}$ un vocabulario y sean M y N $\mathcal{V}$ -estructuras. Una función $f : M \rightarrow N$ *preserva* la $\mathcal{V}$ -fórmula $\varphi(\bar{x})$ si, para cada tupla $\bar{a}$ de elementos de M , $M \models \varphi(\bar{a})$ implica $N \models \varphi(f(\bar{a}))$.

Definición 2.20: Encaje

Sean M y N $\mathcal{V}$ -estructuras y sea $f : M \rightarrow N$ una función. Si f preserva todas las $\mathcal{V}$ -fórmulas que son literales, entonces f es un *encaje literal* (o simplemente un *encaje*). Si f preserva todas las $\mathcal{V}$ -fórmulas, entonces f es un *encaje elemental*.

Ejemplo 2.23

Considere los siguientes dos grafos:



Sea $f : M \rightarrow N$ definida por

$$f(A) = a, f(B) = b, f(C) = c, \text{ y } f(D) = d.$$

Sea $g : M \rightarrow N$ definida por

$$g(A) = b, g(B) = e, g(C) = d, \text{ y } g(D) = f.$$

Entonces g es un encaje literal y f no lo es. ▲

Ejemplo 2.24

Recordemos las estructuras $\mathbb{N}_<$, $\mathbb{Z}_<$, $\mathbb{Q}_<$ y $\mathbb{R}_<$ de la Sección 2.4.3.

Sea $\text{id} : \mathbb{N}_< \rightarrow \mathbb{Z}_<$ la función identidad definida por $\text{id}(x) = x$. Este es un encaje literal. Dado que $\mathbb{N}_< \models \neg \exists x(x < 0)$ y $\mathbb{Z}_< \models \exists x(x < 0)$, este encaje no preserva la fórmula $\neg \exists x(x < y)$, y por lo tanto no es un encaje elemental.

La función identidad $\text{id} : \mathbb{Z}_< \rightarrow \mathbb{Q}_<$ también es un encaje literal que no es elemental (no preserva la fórmula $\neg \exists x(y < x \wedge x < z)$).

La función identidad de $\mathbb{Q}_<$ a $\mathbb{R}_<$, por otro lado, es un encaje elemental. Esto se demostrará en el Capítulo 5. ▲

A continuación mostramos que los encajes literales preservan necesariamente fórmulas que no son literales.

Definición 2.21: Fórmula sin cuantificadores

Una *fórmula sin cuantificadores* es una fórmula en la que no ocurren los cuantificadores $\exists$ ni $\forall$.

Definición 2.22: Fórmula existencial

Una *fórmula existencial* es una fórmula de la forma $\exists y_1 \exists y_2 \dots \exists y_m \varphi(\bar{x}, y_1, y_2, \dots, y_m)$, donde $\varphi(\bar{x}, \bar{y})$ es una fórmula sin cuantificadores y $m \geq 0$.

Mostramos que los encajes preservan las fórmulas existenciales. Primero demostramos la siguiente proposición relativa a las fórmulas sin cuantificadores.

Proposición 2.7

Sea $f : M \rightarrow N$ un encaje. Entonces, para cualquier fórmula sin cuantificadores $\varphi(\bar{x})$ y cualquier tupla $\bar{a}$ de elementos del universo de M ,

$$M \models \varphi(\bar{a}) \text{ si y solo si } N \models \varphi(f(\bar{a})).$$

Demostración. Procedemos por inducción sobre la complejidad de φ .

Supongamos que $\varphi(\bar{x})$ es atómica. Entonces, dado que f preserva literales, si $M \models \varphi(\bar{a})$, entonces $N \models \varphi(f(\bar{a}))$. Recíprocamente, si $N \models \varphi(f(\bar{a}))$ entonces, dado que $\neg\varphi(\bar{x})$ es un literal preservado por f , debe ser el caso que $M \models \varphi(\bar{a})$.

Supongamos ahora que, para las fórmulas ψ y θ ,

$$M \models \psi(\bar{a}) \text{ si y solo si } N \models \psi(f(\bar{a})), \quad \text{y} \quad M \models \theta(\bar{a}) \text{ si y solo si } N \models \theta(f(\bar{a}))$$

para cualquier tupla $\bar{a}$ de elementos del universo de M . Esta es nuestra hipótesis de inducción. Dado que queremos demostrar la proposición solo para fórmulas sin cuantificadores, el paso inductivo, como en lógica proposicional, consta de tres partes correspondientes a $\neg$, $\wedge$ y $\equiv$. Debemos mostrar que $M \models \varphi(\bar{a})$ si y solo si $N \models \varphi(f(\bar{a}))$ cuando φ es $\neg\psi$, cuando φ es $\psi \wedge \theta$, y cuando $\varphi \equiv \psi$. Las dos primeras se siguen inmediatamente de la semántica de la lógica de primer orden, y la última se sigue de la definición de $\equiv$. ■

Proposición 2.8

Los encajes preservan las fórmulas existenciales.

Demostración. Sea $f : M \rightarrow N$ un encaje y sea $\varphi(\bar{x})$ una fórmula existencial. Debemos mostrar que, para cualquier tupla $\bar{a}$ de elementos del universo U_M de M , si $M \models \varphi(\bar{a})$ entonces $N \models \varphi(f(\bar{a}))$. Dado que $\varphi(\bar{x})$ es existencial, tiene la forma $\exists y_1 \exists y_2 \dots \exists y_m \varphi_0(\bar{x}, y_1, y_2, \dots, y_m)$, donde $\varphi_0(\bar{x}, \bar{y})$ es una fórmula sin cuantificadores y $m \geq 0$.

Por la semántica de $\exists$, $M \models \varphi(\bar{a})$ significa que $M \models \varphi_0(\bar{a}, \bar{b})$ para alguna tupla $\bar{b}$ de elementos de U_M . Dado que φ_0 es sin cuantificadores, tenemos $N \models \varphi_0(f(\bar{a}), f(\bar{b}))$ por la proposición anterior. De nuevo por la semántica de $\exists$, $N \models \varphi(f(\bar{a}))$. ■

Obsérvese que si $f : M \rightarrow N$ es un encaje literal, entonces, por la Proposición 2.7, $M \models a = b$ si y solo si $N \models f(a) = f(b)$. Se sigue que todo encaje literal es necesariamente una función inyectiva. Obsérvese también que todo encaje elemental es un encaje literal. En general, "elemental" es un adjetivo mucho más fuerte que "literal." Sin embargo, si f resulta ser sobreyectiva, entonces estas dos nociones coinciden.

Proposición 2.9

Sean M y N $\mathcal{V}$ -estructuras. Si la función $f : M \rightarrow N$ es sobreyectiva, entonces f es un encaje literal si y solo si f es un encaje elemental.

Demostración. Sea $f : M \rightarrow N$ un encaje literal que es sobreyectivo. Entonces f^{-1} es una función inyectiva de N sobre M . Mostramos que tanto f como f^{-1} preservan cada $\mathcal{V}$ -fórmula. Es decir, para cada $\mathcal{V}$ -fórmula $\varphi(\bar{x})$ y cada tupla $\bar{a}$ de elementos de M , $M \models \varphi(\bar{a})$ si y solo si $N \models \varphi(f(\bar{a}))$. Demostramos esto por inducción sobre la complejidad de $\varphi(\bar{x})$. Si $\varphi(\bar{x})$ es atómica, esto es precisamente la Proposición 2.7.

Nuestra hipótesis de inducción es que tanto f como f^{-1} preservan las $\mathcal{V}$ -fórmulas ψ y θ . Si φ es equivalente a ψ , entonces también es preservada por f y f^{-1} . Además, si φ es $\neg\psi$ o $\psi \wedge \theta$, entonces, por la semántica de $\neg$ y $\wedge$, φ es preservada por f y f^{-1} . Resta mostrar que φ se preserva en el caso en que φ es la fórmula $\exists y\psi$.

Sea $\varphi(\bar{x})$ la fórmula $\exists y\psi(\bar{x}, y)$. Primero mostramos que f preserva φ . Supongamos que $M \models \varphi(\bar{a})$ para alguna tupla $\bar{a}$ de elementos de M . Entonces, por la semántica de $\exists$, $M \models \psi(\bar{a}, b)$ para algún elemento b de M . Dado que ψ es preservada por f , $N \models \psi(f(\bar{a}), f(b))$. De nuevo por la semántica de $\exists$, $N \models \varphi(f(\bar{a}))$.

Ahora mostramos que f^{-1} preserva φ . Supongamos que $N \models \varphi(f(\bar{a}))$. Entonces, por la semántica de $\exists$, $N \models \psi(f(\bar{a}), c)$ para algún elemento c de N . Dado que f es sobreyectiva, $c = f(b)$ para algún elemento b de M . Dado que f^{-1} preserva ψ , $M \models \psi(\bar{a}, b)$. Finalmente, de nuevo por la semántica de $\exists$, $M \models \varphi(\bar{a})$. ■

Definición 2.23: Isomorfismo

Sean M y N $\mathcal{V}$ -estructuras. Una función de M a N es un *isomorfismo* si es una correspondencia biunívoca que preserva toda $\mathcal{V}$ -fórmula. Si tal isomorfismo existe, decimos que M y N son *isomorfas*, denotado por $M \cong N$.

Definición 2.24: Equivalencia elemental

Sean M y N $\mathcal{V}$ -estructuras. Si M y N modelan las mismas $\mathcal{V}$ -sentencias, decimos que M y N son *elementalmente equivalentes*, denotado por $M \equiv N$.

Ejemplo 2.25

Las $\mathcal{V}_{<}$ -estructuras $\mathbb{Q}_{<}$ y $\mathbb{R}_{<}$ de la Sección 2.4.3 son elementalmente equivalentes. ▲

Proposición 2.10

Sean M y N $\mathcal{V}$ -estructuras. Si $M \cong N$, entonces $M \equiv N$.

Demostración. Sea $f : M \rightarrow N$ un isomorfismo. Entonces tanto f como f^{-1} preservan toda fórmula. En particular, para cualquier sentencia φ , $M \models \varphi$ si y solo si $N \models \varphi$. ■

Si las $\mathcal{V}$ -estructuras M y N son elementalmente equivalentes, entonces no podemos distinguirlas usando lógica de primer orden. Además, si M y N son isomorfas, entonces son esencialmente la misma. La única diferencia entre estructuras isomorfas son los nombres dados a los elementos de los conjuntos subyacentes (recuerde el Ejemplo 2.18).

2.6.2. Subestructuras

Si B es un conjunto, $A \subset B$ significa que A es un subconjunto de B . Si N es una estructura, $M \subset N$ significa que M es una subestructura de N . Definimos ahora este concepto.

Definición 2.25: Subestructura

Para cualquier estructura N , M es una *subestructura* de N , denotado $M \subset N$, si

1. M es una estructura que tiene el mismo vocabulario que N ,
2. el conjunto subyacente U_M de M es un subconjunto del conjunto subyacente U_N de N ,
y
3. M interpreta el vocabulario de la misma manera que N sobre U_M .

Ejemplo 2.26

Recordemos las estructuras $\mathbb{N}_<$, $\mathbb{Z}_<$, $\mathbb{Q}_<$ y R de la Sección 2.4.3. Tenemos $\mathbb{N}_< \subset \mathbb{Z}_< \subset \mathbb{Q}_< \subset R$. Del mismo modo, para las estructuras discutidas en la Sección 2.4.4, $A \subset Q \subset R \subset C$. ▲

Ejemplo 2.27

Sea G el siguiente grafo:

Vértices: a, b, c, d, e

Aristas: $ab, ac, ad, ae, bc, cd, de$

Si elegimos cualquier subconjunto de estos vértices y cualquier subconjunto de aristas que involucre a los vértices elegidos, obtenemos lo que en teoría de grafos se conoce como un *subgrafo*.

Sea H el siguiente subgrafo de G .

Vértices: a, b, c, d

Aristas: ab, ad, bc, cd

Aunque H es un subgrafo de G , H no es una subestructura de G (viendo a G y a H como $\mathcal{V}_G$ -estructuras). Dado que $G \models R(a, c)$ y $H \models \neg R(a, c)$, H no interpreta la relación binaria R de la

misma manera que G en el conjunto $\{a, b, c, d\}$. La noción de subestructura corresponde a la noción, propia de la teoría de grafos, de subgrafo inducido. ▲

Sea N una $\mathcal{V}$ -estructura y sea U_N el conjunto subyacente de N . No todo subconjunto de U_N puede servir como universo de una subestructura de N . Dado que una subestructura es en sí misma una $\mathcal{V}$ -estructura, debe interpretar cada constante y función de $\mathcal{V}$. Dado que N es una $\mathcal{V}$ -estructura, interpreta cada constante c de $\mathcal{V}$ como un elemento a_c de U_N . Sea C el subconjunto de U_N definido por $C = \{a_c \mid c \text{ es una constante de } \mathcal{V}\}$. Sea f una función n -aria de $\mathcal{V}$. Un subconjunto D de U_N es *cerrado bajo f* si y solo si, para cada n -tupla $\bar{a}$ de elementos de D , $f(\bar{a})$ también es un elemento de D . Para que D sea el universo de una subestructura de N , es necesario y suficiente que D contenga a cada elemento de C y sea cerrado bajo cada función de $\mathcal{V}$.

Ejemplo 2.28

Sea N la estructura $(\mathbb{N} \mid S)$ que interpreta la relación binaria S como la relación sucesor. Es decir, para cualesquiera a y b en $\mathbb{N}$, $N \models S(a, b)$ si y solo si $b = a + 1$. Dado que el vocabulario no contiene ni constantes ni funciones, todo subconjunto de $\mathbb{N}$ es el universo de una subestructura de N . Se sigue que hay una cantidad no contable de subestructuras de N . Además, existen incontables subestructuras, ninguna de las cuales es isomorfa a otra. Dejamos la verificación de este hecho como Ejercicio 2.35. ▲

Ejemplo 2.29

Sea N la estructura $(\mathbb{N} \mid s)$ que interpreta la función unaria s como la función sucesor. Es decir, para cualesquiera a y b en $\mathbb{N}$, $N \models s(a) = b$ si y solo si $b = a + 1$. Solo aquellos subconjuntos de $\mathbb{N}$ que son cerrados bajo s pueden servir como universo de una subestructura. Los subconjuntos cerrados de $\mathbb{N}$ son los conjuntos de la forma $\{n \mid n \geq d\}$ para algún $d \in \mathbb{N}$. Se sigue que hay una cantidad contable de subestructuras de N . Además, todas estas subestructuras son isomorfas. Así, solo hay una subestructura salvo isomorfismo. ▲

Ejemplo 2.30

Sea N la estructura $(\mathbb{N} \mid s, 1)$ que interpreta la función unaria s como la función sucesor y la constante 1 como el elemento 1 de $\mathbb{N}$. Si $D \subset \mathbb{N}$ es el universo de una subestructura de N , entonces D debe contener a 1 y ser cerrado bajo la función s . Se sigue que D debe ser todo $\mathbb{N}$. Por lo tanto, la única subestructura de N es N misma. ▲

Una definición alternativa de subestructura la proporciona la noción de encaje.

Proposición 2.11

Sean N y M estructuras del mismo vocabulario. Entonces M es una subestructura de N si y solo si la función identidad $\text{id} : M \rightarrow N$ definida por $\text{id}(x) = x$ es un encaje.

Demostración. Ejercicio 2.26. ■

Si $M \subset N$, decimos entonces, invirtiendo nuestro punto de vista, que N es una *extensión* de M . Obsérvese la distinción entre una "extensión" y una "expansión" de una estructura. Una estructura tiene tanto un conjunto subyacente como un vocabulario. Una expansión de una estructura tiene el mismo conjunto subyacente, pero el vocabulario puede aumentar. Una extensión de una estructura tiene el mismo vocabulario, pero el conjunto subyacente puede ampliarse.

Definición 2.26

Se dice que la fórmula $\varphi(\bar{x})$ se *preserva bajo extensiones* si, siempre que $M \subset N$ y $\bar{a}$ sea una tupla de elementos del universo de M , si $M \models \varphi(\bar{a})$ entonces $N \models \varphi(\bar{a})$.

Definición 2.27

Se dice que la fórmula $\varphi(\bar{x})$ se *preserva bajo subestructuras* si, siempre que $M \subset N$ y $\bar{a}$ sea una tupla de elementos del universo de M , si $N \models \varphi(\bar{a})$ entonces $M \models \varphi(\bar{a})$.

Proposición 2.12

Las fórmulas sin cuantificadores se preservan bajo subestructuras y bajo extensiones.

Demostración. Esto se sigue inmediatamente de la Proposición 2.7. ■

Proposición 2.13

Las fórmulas existenciales se preservan bajo extensiones.

Demostración. Esto se sigue inmediatamente de la Proposición 2.8. ■

En particular, las sentencias existenciales se preservan bajo extensiones. Intuitivamente, una sentencia existencial afirma que una fórmula sin cuantificadores $\varphi_0(\bar{y})$ se cumple para alguna tupla $\bar{y}$ de elementos del universo. Si esto es cierto en M y $M \subset N$, entonces también debe ser cierto en N , ya que toda tupla de elementos del universo de M es también una tupla de elementos del universo de N . Del mismo modo, si $\varphi_0(\bar{y})$ se cumple para todas las tuplas $\bar{y}$ de elementos del universo de N , entonces, en particular, se cumple para todos los elementos de cualquier subestructura de N . Así, las sentencias de la forma $\forall \bar{y} \varphi_0(\bar{y})$ se preservan bajo subestructuras.

Definición 2.28: Fórmula universal

Una *fórmula universal* es una fórmula de la forma

$$\forall y_1 \forall y_2 \dots \forall y_m \varphi(\bar{x}, y_1, y_2, \dots, y_m),$$

donde $\varphi(\bar{x}, \bar{y})$ es una fórmula sin cuantificadores y $m \geq 0$.

Proposición 2.14

Las fórmulas universales se preservan bajo subestructuras.

Demostración. Ejercicio 2.32. ■

En el Capítulo 4 demostramos los recíprocos de estas proposiciones. Mostramos en la Sección 4.5.1 que si una fórmula φ se preserva bajo subestructuras, entonces φ es equivalente a una fórmula universal. Del mismo modo, si φ se preserva bajo extensiones, entonces φ es equivalente a una fórmula existencial.

La noción de encaje elemental da lugar al siguiente fortalecimiento de la noción de subestructura.

Definición 2.29: Subestructura elemental

Sean N y M estructuras del mismo vocabulario. Entonces M es una *subestructura elemental* de N (o, equivalentemente, N es una *extensión elemental* de M), denotado $M \prec N$, si y solo si la función identidad $\text{id} : M \rightarrow N$ definida por $\text{id}(x) = x$ es un encaje elemental.

Si N es una extensión elemental de M , entonces, para cualquier fórmula $\varphi(\bar{x})$ y cualquier tupla $\bar{a}$ de elementos del universo de M , $M \models \varphi(\bar{a})$ si y solo si $N \models \varphi(\bar{a})$. Se sigue que si $M \prec N$, entonces $M \equiv N$. El recíproco de esto no se cumple. En el siguiente ejemplo, M es una subestructura de N y $M \equiv N$, pero M no es una subestructura elemental de N .

Ejemplo 2.31

Sea N los números naturales con la función sucesor. Es decir, $N = (\mathbb{N} \mid s)$ del Ejemplo 2.29. Sea M la subestructura de N con universo $\{2, 3, 4, \dots\}$. Sea $f : N \rightarrow M$ definida por $f(n) = n + 1$ para cada n en $\mathbb{N}$. Entonces f es un isomorfismo de N sobre M . Tenemos tanto $M \subset N$ como $M \cong N$. Sin embargo, M no es una subestructura elemental de N . Existe un encaje elemental de M en N , pero no es la función identidad. En particular, sea $\varphi(x)$ la fórmula $\neg \exists y (s(y) = x)$, que dice que x no tiene predecesor. Entonces $M \models \varphi(2)$, pero $N \models \neg \varphi(2)$. ▲

2.6.3. Diagramas

El concepto de diagrama (y, más específicamente, de diagrama elemental) de una $\mathcal{V}$ -estructura M es un concepto fundamental que usaremos repetidamente en este libro (principalmente en el Capítulo 4). Intuitivamente, un diagrama de M es un conjunto de sentencias de primer orden que, en conjunto, dicen " M puede encajarse en mí." Es decir, M puede encajarse en cualquier modelo del diagrama de M . Del mismo modo, el diagrama elemental de M es un conjunto de sentencias tal que M puede encajarse elementalmente en cualquier modelo. Definimos ahora explícitamente estos conjuntos de sentencias. Recordemos que $\mathcal{V}(M)$ denota la expansión de $\mathcal{V}$ obtenida al agregar una constante por cada elemento del conjunto subyacente de M , y M_C denota la expansión de M a una $\mathcal{V}(M)$ -estructura que interpreta estas constantes de la manera natural.

Definición 2.30: Diagrama

Sea M una $\mathcal{V}$ -estructura.

El *diagrama elemental* de M , denotado $ED(M)$, es el conjunto de todas las $\mathcal{V}(M)$ -sentencias que se cumplen en M_C .

El *diagrama literal* de M , denotado $D(M)$, es el conjunto de todos los literales de $ED(M)$.

Con frecuencia nos referimos al diagrama literal de M simplemente como el diagrama de M .

Ejemplo 2.32

Considere el grafo definido por la siguiente información:

Vértices: a, b, c, d

Aristas: ab, bc, cd, bd .

Sea G la $\mathcal{V}_G$ -estructura representada por este grafo. El diagrama $D(G)$ contiene las fórmulas atómicas

$$R(a, b), \quad R(b, c), \quad R(c, d), \quad \text{y} \quad R(b, d)$$

que expresan las aristas de G . También contiene las fórmulas atómicas negadas

$$\neg R(a, c) \quad \text{y} \quad \neg R(a, d)$$

que expresan las aristas que no están en G . También hay fórmulas atómicas negadas que indican que a, b, c y d son distintos:

$$\neg(a = b), \quad \neg(a = c), \quad \neg(a = d), \quad \neg(b = c), \quad \neg(b = d), \quad \text{y} \quad \neg(c = d).$$

Obsérvese que G puede encajarse en cualquier grafo que modele estos 12 literales de $D(G)$. Además, $D(G)$ contiene los literales

$$R(b, a), \quad R(c, b), \quad R(d, c), \quad \text{y} \quad R(d, b)$$

junto con

$$\neg R(a, a), \neg R(b, b), \neg R(c, c), \neg R(d, d),$$

y

$$a = a, b = b, c = c, d = d, \neg(b = a), \neg(c = a), \dots$$

y así sucesivamente. En total, hay 32 literales distintos (aunque redundantes) en $D(G)$. Obsérvese que G puede encajarse en cualquier $\mathcal{V}_G$ -estructura que modele todas estas sentencias. ▲

Proposición 2.15

Sean M y N $\mathcal{V}$ -estructuras. Las siguientes son equivalentes:

- (i) M puede encajarse en N .
- (ii) $\widetilde{N} \models D(M)$ para alguna expansión $\widetilde{N}$ de N .
- (iii) $N' \cong N$ para alguna extensión N' de M .

Demostración. Sean U_M y U_N los conjuntos subyacentes de M y N , respectivamente.

Primero, mostramos que (iii) implica (i). Supongamos que $M \subset N'$ y $N' \cong N$. Sea $f : N' \rightarrow N$ un isomorfismo. Entonces f restringida a M es un encaje de M en N .

Para ver que (i) implica (ii), supongamos que $f : M \rightarrow N$ es un encaje. Sea $C = \{c_m : m \in U_M\}$ constantes que no están en $\mathcal{V}$. Sea $\mathcal{V}(C)$ la expansión $\mathcal{V} \cup C$ de $\mathcal{V}$. Sea $\widetilde{N}$ la expansión de N a una $\mathcal{V}(C)$ -estructura que interpreta cada $c_m \in C$ como el elemento $f(m) \in U_N$. Entonces $\widetilde{N} \models D(M)$.

Finalmente, sea $\widetilde{N}$ como en (ii). Queremos mostrar que se cumple (iii). El conjunto U_M podría no ser un subconjunto de U_N . Sin embargo, para cada $m \in U_M$, debe existir $m' \in U_N$ que $\widetilde{N}$ interpreta como la constante c_m . Sea $U_{N'}$ el conjunto obtenido al reemplazar cada $m' \in U_N$ por m . Ahora $U_M \subset U_{N'}$. Sea N' la $\mathcal{V}$ -estructura con conjunto subyacente $U_{N'}$ que interpreta $\mathcal{V}$ de la misma manera que N . Entonces la función f definida por $f(m) = m$ para $m \in U_M$ y $f(x) = x$ para $x \in U_N - U_M$ es un isomorfismo de N sobre N' . ■

Del mismo modo, tenemos lo siguiente.

Proposición 2.16

Sean M y N $\mathcal{V}$ -estructuras. Las siguientes son equivalentes:

- (i) M puede encajarse elementalmente en N .
- (ii) $\widetilde{N} \models ED(M)$ para alguna expansión $\widetilde{N}$ de N .
- (iii) $N' \cong N$ para alguna extensión elemental N' de M .

Demostración. Ejercicio 2.27. ■

Si M es una estructura finita en un vocabulario finito, entonces $D(M)$ es finito. Se sigue que toda estructura finita queda completamente descrita por una única sentencia de la lógica de primer orden.

Proposición 2.17

Sea $\mathcal{V}$ un vocabulario finito. Para cualquier $\mathcal{V}$ -estructura finita M , existe una $\mathcal{V}$ -sentencia φ_M tal que, para cualquier $\mathcal{V}$ -estructura N , $N \models \varphi_M$ si y solo si $N \cong M$.

Demostración. Sea $\{a_1, a_2, \dots, a_n\}$ el conjunto subyacente de M . Sea $\varphi(\bar{a})$ la conjunción de las finitas sentencias de $D(M)$, donde $\bar{a}$ denota la n -tupla $(a_1, a_2, \dots, a_n)$. Sea $\varphi(\bar{x})$ la $\mathcal{V}$ -fórmula obtenida al reemplazar cada a_i en $\varphi(\bar{a})$ por la variable x_i (que suponemos no ocurre en $\varphi(\bar{a})$). Abreviamos la sentencia $\exists x_1 \exists x_2 \dots \exists x_n \varphi(\bar{x})$ simplemente como $\exists \bar{x} \varphi(\bar{x})$.

Sea ψ_n la sentencia

$$\forall x_1 \forall x_2 \dots \forall x_{n+1} \left(\bigvee_{i \neq j} (x_i = x_j) \right)$$

que dice que, dados cualesquiera $n + 1$ elementos, debe haber dos que sean iguales.

Sea ahora φ_M la sentencia $\psi_n \wedge \exists \bar{x} \varphi(\bar{x})$. Debemos verificar que esta sentencia funciona. Supongamos que $N \models \varphi_M$. Entonces, dado que $N \models \exists \bar{x} \varphi(\bar{x})$, N contiene n elementos $b_1, \dots, b_n$ tales que $N \models \varphi(b_1, \dots, b_n)$. Por la Proposición 2.15, M puede encajarse en N . Sea $f : M \rightarrow N$ un encaje. Dado que $N \models \psi_n$, $|N| \leq n$. Se sigue que f debe ser sobreyectiva. Por la Proposición 2.9, f es elemental y, por lo tanto, un isomorfismo. ■

Corolario 2.2

Si M es finita, entonces, para cualquier estructura N , $M \cong N$ si y solo si $M \equiv N$.

Como mencionamos anteriormente, este corolario no es cierto para estructuras infinitas. Si M es infinita, entonces existen muchas estructuras N no isomorfas para las cuales $M \equiv N$. Esto se demuestra en el Capítulo 4. Dicho de otra manera, la lógica de primer orden no es capaz de describir por completo las estructuras infinitas. La lógica de primer orden es, en este sentido, un lenguaje débil. Irónicamente, como consecuencia de esta debilidad, la lógica de primer orden tiene muchas propiedades deseables (discutidas en el Capítulo 4) que la convierten en una lógica prominente. La debilidad de la lógica de primer orden da lugar a la disciplina de la teoría de modelos.

2.7 Teorías y modelos

La teoría de modelos es la rama de la lógica que se ocupa de la interacción entre las estructuras matemáticas y las sentencias de un lenguaje formal. La lógica de primer orden sirve como lenguaje principal para esta disciplina. Cualquier estructura M determina un conjunto de sentencias de primer orden $\text{Th}(M)$ llamado la *teoría* de M .

Definición 2.31: Teoría de una estructura

Para cualquier $\mathcal{V}$ -estructura M , la *teoría* de M , denotada $\text{Th}(M)$, es el conjunto de todas las $\mathcal{V}$ -sentencias φ tales que $M \models \varphi$.

Recíprocamente, cualquier conjunto de sentencias de primer orden Γ determina una clase de estructuras $\text{Mod}(\Gamma)$.

Definición 2.32: Modelo

Para cualquier conjunto de $\mathcal{V}$ -sentencias Γ , un *modelo* de Γ es una $\mathcal{V}$ -estructura que modela cada sentencia de Γ . La clase de todos los modelos de Γ se denota por $\text{Mod}(\Gamma)$.

Nota: se usa la palabra *clase* en lugar de *conjunto* para $\text{Mod}(\Gamma)$ debido al siguiente tecnicismo: $\text{Mod}(\Gamma)$ a veces no está acotado. No está acotado precisamente cuando Γ tiene un modelo infinito. Por "no acotado" queremos decir que, para cualquier conjunto X , $\text{Mod}(\Gamma)$ es estrictamente más grande que X . Si este es el caso, entonces $\text{Mod}(\Gamma)$ no puede ser un conjunto (no puede ser estrictamente más grande que sí mismo).

Bajo ciertas condiciones sobre Γ , la teoría de cualquier modelo de Γ es Γ misma. Si este es el caso, entonces $\text{Th}(M) = \Gamma$ si y solo si $M \in \text{Mod}(\Gamma)$. Esto ocurre solo si Γ es una *teoría completa*, noción que definimos a continuación.

Definición 2.33: Teoría completa

Sea Γ un conjunto de $\mathcal{V}$ -sentencias. Entonces Γ es una *$\mathcal{V}$ -teoría completa* si, para cualquier $\mathcal{V}$ -sentencia φ , o bien φ o bien $\neg\varphi$ está en Γ , y no es el caso que tanto φ como $\neg\varphi$ estén en Γ .

Proposición 2.18

Para cualquier $\mathcal{V}$ -estructura M , $\text{Th}(M)$ es una $\mathcal{V}$ -teoría completa.

Demostración. Mostramos que, para cualquier vocabulario $\mathcal{V}$, cualquier $\mathcal{V}$ -estructura M , y cualquier $\mathcal{V}$ -sentencia φ :

(†) o bien φ o bien $\neg\varphi$ está en $\text{Th}(M)$, y no es el caso que tanto φ como $\neg\varphi$ estén en $\text{Th}(M)$.

Sin pérdida de generalidad, podemos suponer que φ no contiene ocurrencias de $\forall$, $\rightarrow$, $\leftrightarrow$ ni $\exists$. Esto se debe a que estos símbolos se definen en función de los símbolos primitivos $\neg$, $\wedge$ y $\exists$. Procedemos por inducción sobre el número total de ocurrencias de $\neg$, $\wedge$ y $\exists$ en φ .

Si φ no contiene ninguna ocurrencia de los símbolos primitivos, entonces φ tiene la forma $R(t_1, \dots, t_n)$ o $t_1 = t_2$ donde $t_1, \dots, t_n$ son $\mathcal{V}$ -términos. Es decir, φ es atómica. Dado que φ es una sentencia, cada t_i

está libre de variables. Dado que M es una $\mathcal{V}$ -estructura y cada t_i es un $\mathcal{V}$ -término libre de variables, M interpreta cada t_i como un elemento a_i del universo U de M . Por la definición de $\models$, $M \models t_1 = t_2$ si y solo si a_1 y a_2 son el mismo elemento de U , y $M \models R(t_1, \dots, t_n)$ si y solo si la tupla $(a_1, \dots, a_n)$ está en el subconjunto de U^n que la interpretación de M asigna a R .

En cualquier caso, vemos que $M \models \varphi$ o $M \models \neg\varphi$, y no ambas.

Hemos verificado ($\dagger$) para cualquier vocabulario $\mathcal{V}$, cualquier $\mathcal{V}$ -estructura M , y cualquier $\mathcal{V}$ -sentencia atómica φ . Supongamos ahora que hemos mostrado esto para cualquier $\mathcal{V}$ -sentencia que contenga a lo sumo m ocurrencias totales de $\neg$, $\wedge$ y $\exists$. Esta es nuestra hipótesis de inducción.

Supongamos que φ tiene la forma $\neg\psi$ o $\psi \wedge \theta$. Por nuestra hipótesis de inducción, ($\dagger$) se cumple tanto para ψ como para θ . Por la semántica de $\neg$ y $\wedge$, el enunciado anterior también se cumple para φ . Finalmente, supongamos que φ tiene la forma $\exists x\psi(x)$. Por la semántica de $\exists$, $M \models \varphi$ si y solo si $M_C \models \psi(c)$ para alguna constante c del vocabulario de M_C . De nuevo por nuestra hipótesis de inducción, el enunciado anterior se cumple para $\psi(c)$, y por lo tanto se cumple también para φ .

Se sigue por inducción que ($\dagger$) se cumple para todas las sentencias φ . ■

Esta proposición, aunque bastante elemental, tiene una importancia fundamental. Esta proposición verifica que la lógica de primer orden evita las ambigüedades y paradojas que surgen en los lenguajes naturales. En cualquier conjunto de sentencias de primer orden que describa una estructura dada, no hay nada contradictorio.

Definición 2.34: Consistente

Se dice que un conjunto de sentencias Γ es *consistente* si no puede deducirse ninguna contradicción de Γ .

La palabra "deducirse" se define formalmente para la lógica de primer orden en el siguiente capítulo, pero la idea es análoga a la noción de "deducirse" para la lógica proposicional.

Definición 2.35: Teoría

Una *teoría* es un conjunto consistente de sentencias. Si T es una teoría, entonces $\text{Mod}(T)$ se llama una *clase elemental*.

Sea $\mathcal{V}$ un vocabulario. Entonces una $\mathcal{V}$ -teoría es un conjunto consistente de $\mathcal{V}$ -sentencias. Una $\mathcal{V}$ -teoría T es una *teoría completa* si es maximal en el siguiente sentido: ningún conjunto de $\mathcal{V}$ -sentencias que contenga a T como subconjunto propio es consistente. Esto concuerda con nuestra definición anterior de "teoría completa."

La teoría de modelos estudia las teorías y los modelos, y la interacción entre ellos. Comprender la teoría de una estructura brinda información sobre la estructura. La teoría describe la estructura. Por otro lado, comprender los modelos de una teoría brinda información sobre la teoría. Una teoría T puede clasificarse con base en diversas propiedades de $\text{Mod}(T)$.

Continuamos nuestro estudio de la teoría de modelos en los Capítulos 4 a 6. El Capítulo 4 considera las propiedades de la lógica de primer orden que la hacen un lenguaje apropiado para la teoría

de modelos. En el Capítulo 5 nos enfocamos en las teorías y consideramos algunas propiedades que una teoría puede o no poseer. En el Capítulo 6, consideramos modelos individuales de una teoría que tienen propiedades especiales. Antes de esto, en el Capítulo 3, consideramos el problema básico de determinar si una sentencia dada de la lógica de primer orden es satisfacible. Con ese fin, desarrollamos las demostraciones formales y la resolución para la lógica de primer orden.

Ejercicios

Ejercicio 2.1 Sea $\mathcal{V}$ el vocabulario $\{+, <, 1, 2, 3\}$ donde $+$ es una función binaria, $<$ es una relación binaria, y 1, 2 y 3 son constantes. Escribimos $(x + y)$ para $+(x, y)$ y $x < y$ para $<(x, y)$. Considere las siguientes $\mathcal{V}$ -fórmulas:

1. $\forall x \exists y ((x + y) = 1)$
2. $\forall x \neg (x < 1)$
3. $((1 + 1) = 2)$
4. $2 < 1$
5. $\forall x (2 < 1) \rightarrow (x + 2 < x + 1)$
6. $\forall x \forall y \exists z (x + y = z)$
7. $\forall x \forall y \forall z (((x + 3 = y) \wedge (x + 3 = z)) \rightarrow (y = z))$
8. $\forall x \forall y \forall z (((x + y = 3) \wedge (x + z = 3)) \rightarrow (y = z))$
9. $\forall x \forall y (((x + 3) < (y + 3)) \rightarrow (x < y))$
10. $\forall x \forall y ((x < 2) \rightarrow ((x + 3) = 4))$

- (a) ¿Cuáles de estas 10 fórmulas son sentencias?
- (b) ¿Cuáles de estas 10 fórmulas son satisfacibles?
- (c) ¿Cuáles de estas 10 fórmulas son tautologías?
- (d) Sea $\mathbb{N}_+$ la $\mathcal{V}$ -estructura con universo $\mathbb{N}$ que interpreta los símbolos de $\mathcal{V}$ de la manera usual. ¿Cuáles de las sentencias anteriores modela $\mathbb{N}_+$?
- (e) Sea $\mathbb{R}_+$ la $\mathcal{V}$ -estructura con universo $\mathbb{R}$ que interpreta los símbolos de $\mathcal{V}$ de la manera usual. ¿Cuáles de las sentencias anteriores modela $\mathbb{R}_+$?
- (f) Enumere los términos que ocurren en las fórmulas anteriores.
- (g) Para cada una de las diez fórmulas, indique el número de subfórmulas. ¿Cuántas subfórmulas atómicas tiene cada fórmula?

Ejercicio 2.2 Sea $\mathcal{V}$ el vocabulario que consiste en una relación binaria P y una relación unaria F . Interprete $P(x, y)$ como " x es padre o madre de y " y $F(x)$ como " x es mujer."

- (a) Defina una $\mathcal{V}$ -fórmula $\varphi_B(x, y)$ que diga que x es hermano de y .
- (b) Defina una $\mathcal{V}$ -fórmula $\varphi_A(x, y)$ que diga que x es tía de y .
- (c) Defina una $\mathcal{V}$ -fórmula $\varphi_C(x, y)$ que diga que x y y son primos.
- (d) Defina una $\mathcal{V}$ -fórmula $\varphi_O(x)$ que diga que x es hijo o hija único(a).
- (e) Defina una $\mathcal{V}$ -fórmula $\varphi_T(x)$ que diga que x tiene exactamente dos hermanos.
- (f) Dé un ejemplo de una relación familiar que no pueda definirse mediante una $\mathcal{V}$ -fórmula.

Ejercicio 2.3 El *espectro finito* de una sentencia de primer orden φ es el conjunto de números naturales n tales que φ tiene un modelo de tamaño n . Halle una sentencia de primer orden φ que tenga a S como espectro finito para cada uno de los siguientes conjuntos S :

- (a) S es el conjunto de los números naturales pares.
- (b) S es el conjunto de los números naturales impares.
- (c) S es el conjunto de los números primos.
- (d) S es el conjunto de los cuadrados perfectos.

Ejercicio 2.4 Refiérase al Ejemplo 2.17.

- (a) Muestre que φ_1 no es consecuencia de φ_2 y φ_3 .
- (b) Muestre que φ_3 no es consecuencia de φ_1 y φ_2 .

Ejercicio 2.5 Sea $\mathcal{V}_{gp}$ el vocabulario $\{+, 0\}$ donde $+$ es una función binaria y 0 es una constante. Usamos la notación $x + y$ para denotar el término $+(x, y)$. Considere las siguientes $\mathcal{V}_{gp}$ -sentencias.

$$\begin{aligned} \forall x \forall y \forall z (x + (y + z) &= (x + y) + z) \\ \forall x ((x + 0 = x) \wedge (0 + x &= x)) \\ \forall x (\exists y (x + y = 0) \wedge \exists z (z + x &= 0)), \end{aligned}$$

Sea γ la conjunción de estas tres sentencias.

- (a) Muestre que γ es satisfacible exhibiendo un modelo.
- (b) Muestre que γ no es una tautología.
- (c) Sea α la sentencia $\forall x \forall y ((x + y) = (y + x))$. Muestre que α no es consecuencia de γ .
- (d) Muestre que γ no es equivalente a la conjunción de dos cualesquiera de las tres sentencias anteriores.

Ejercicio 2.6 Una fórmula de primer orden $\varphi(x)$ se dice satisfacible si y solo si la sentencia $\forall x\varphi(x)$ es satisfacible. Demuestre que una fórmula $\varphi(x)$ es una tautología si y solo si la sentencia $\exists x\varphi(x)$ es una tautología.

Ejercicio 2.7 Sea $\mathcal{V}_N = \{+, \cdot, 1\}$. Sea N la $\mathcal{V}_N$ -estructura con conjunto subyacente $\mathbb{N}$ que interpreta este vocabulario de la manera usual.

- (a) Defina una $\mathcal{V}_N$ -fórmula $\varepsilon(x)$ tal que, para cualquier $a \in \mathbb{N}$, $N \models \varepsilon(a)$ si y solo si a es par.
- (b) Defina una $\mathcal{V}_N$ -fórmula $\pi(x)$ tal que, para cualquier $a \in \mathbb{N}$, $N \models \pi(a)$ si y solo si a es primo.
- (c) Defina una $\mathcal{V}_N$ -fórmula $\mu(x, y)$ tal que, para cualesquiera a y b en $\mathbb{N}$, $N \models \mu(a, b)$ si y solo si a y b son primos relativos (es decir, el máximo común divisor de a y b es 1).
- (d) Defina una $\mathcal{V}_N$ -fórmula $\nu(x, y, z)$ tal que, para cualesquiera a , b y c en $\mathbb{N}$, $N \models \nu(a, b, c)$ si y solo si c es el menor número divisible tanto por a como por b .

Ejercicio 2.8 La conjetura de Goldbach afirma que todo entero par mayor que 2 es la suma de dos números primos. No se sabe si esto es verdadero o falso; es una pregunta abierta de la teoría de números. Enuncie la conjetura de Goldbach como una $\mathcal{V}_{ar}$ -sentencia, donde $\mathcal{V}_{ar} = \{+, \cdot, 0, 1\}$.

Ejercicio 2.9 Sea $\mathcal{V}_{ar} = \{+, \cdot, 0, 1\}$ el vocabulario de la aritmética. Sea R la $\mathcal{V}_{ar}$ -estructura que tiene universo $\mathbb{R}$ e interpreta el vocabulario de la manera usual.

- (a) Defina una $\mathcal{V}_{ar}$ -fórmula $\alpha(x)$ tal que, para cualquier $a \in \mathbb{R}$, $R \models \alpha(a)$ si y solo si a es positivo.
- (b) Defina una $\mathcal{V}_{ar}$ -fórmula $\beta(x, y)$ tal que, para cualesquiera a y b en $\mathbb{R}$, $R \models \beta(a, b)$ si y solo si $a \leq b$.
- (c) Defina una $\mathcal{V}_{ar}$ -fórmula $\gamma(x)$ tal que, para cualquier a en $\mathbb{R}$, $R \models \gamma(a)$ si y solo si el valor absoluto de a es menor que 1.

Ejercicio 2.10 Sean $\mathcal{V}_{ar}$ y R como en el ejercicio anterior. Sea $\mathcal{V}^+ = \mathcal{V}_{ar} \cup \{f\}$ la expansión de $\mathcal{V}_{ar}$ obtenida al agregar una función unaria f . Defina una $\mathcal{V}^+$ -sentencia ζ tal que, para cualquier expansión R^+ de R a una $\mathcal{V}^+$ -estructura, $R^+ \models \zeta$ si y solo si R^+ interpreta f como una función continua.

Ejercicio 2.11 Sean A y B subconjuntos definibles de una estructura M . Suponga que A y B son ambos conjuntos de n -tuplas de elementos del conjunto subyacente de M .

- (a) Muestre que $A \cup B$ es definible.
- (b) Muestre que $A \cap B$ es definible.
- (c) Muestre que $A - B = \{a \mid a \in A \text{ y } a \notin B\}$ es definible.

Ejercicio 2.12 Sea U_M el conjunto subyacente de la estructura M . Suponga que $A \subset (U_M)^3$ y $B \subset (U_M)^3$ son subconjuntos definibles de M .

- (a) Muestre que $A \times B \subset (U_M)^6$ es definible.
- (b) Suponga que reordenamos las n -tuplas. Considere el conjunto de todos los (z, x, y) tales que (x, y, z) está en A . Muestre que este conjunto es definible.
- (c) Muestre que $C \subset (U_M)^2$ es definible, donde C es el conjunto de pares ordenados (x, y) tales que (x, y, z) está en A para algún z .
- (d) Muestre que $D \subset (U_M)^2$ es definible, donde D es el conjunto de pares ordenados (x, y) tales que tanto $(x, y, z) \in A$ para algún z como $(x, y, z) \in B$ para algún z .
- (e) Muestre que $E \subset (U_M)^2$ es definible, donde E es el conjunto de pares ordenados (x, y) tales que, para algún z , (x, y, z) está tanto en A como en B .

Ejercicio 2.13 Definimos la distancia $d(a, b)$ entre dos vértices a y b de un grafo como el menor número de aristas en un camino de a a b . Si no existe tal camino, entonces $d(a, b) = \infty$. Recordemos que $\mathcal{V}_G$ es el vocabulario de los grafos.

- (a) Muestre que, para cualquier $n \in \mathbb{N}$, existe una $\mathcal{V}_G$ -fórmula $\delta_n(x, y)$ tal que, para cualquier grafo G , $G \models \delta_n(a, b)$ si y solo si $d(a, b) = n$. (Defina las fórmulas $\delta_n(x, y)$ por inducción sobre n .)
- (b) ¿Existe una $\mathcal{V}_G$ -fórmula $\delta_\infty(x, y)$ tal que, para cualquier grafo G , $G \models \delta_\infty(a, b)$ si y solo si $d(a, b) = \infty$? Explique su respuesta.

Ejercicio 2.14

- (a) Defina una $\mathcal{V}_G$ -sentencia φ tal que φ tenga modelos finitos arbitrariamente grandes y, para cualquier modelo G , G sea un grafo conexo.
- (b) Halle un grafo conexo que no modele la sentencia φ que halló en la parte (a).

Ejercicio 2.15

- (a) Defina una $\mathcal{V}_G$ -sentencia φ tal que $\neg\varphi$ tenga modelos finitos arbitrariamente grandes, y $G \models \varphi$ para todo grafo conexo G .
- (b) Halle un grafo que no sea conexo y que modele la sentencia φ de la parte (a).

Ejercicio 2.16

- (a) Defina una $\mathcal{V}_G$ -sentencia φ tal que φ tenga modelos finitos arbitrariamente grandes, y para cualquier modelo finito G de φ , $|G|$ sea par.

- (b) Halle un grafo finito G tal que $|G|$ sea par y G no modele la sentencia φ de la parte (a).

Ejercicio 2.17

- (a) Defina una $\mathcal{V}_G$ -sentencia φ tal que $\neg\varphi$ tenga modelos finitos arbitrariamente grandes, y tal que, para cualquier grafo finito G , si $|G|$ es par, entonces $G \models \varphi$.
- (b) Halle un modelo finito G para la sentencia φ de la parte (a) tal que $|G|$ sea impar.

Ejercicio 2.18

- (a) Explique la diferencia entre los prefijos de primer orden $\exists x\forall y$ y $\forall x\exists y$.
- (b) Explique la diferencia entre los prefijos de primer orden $\exists x\forall y\exists z$ y $\forall x\exists y\forall z$.
- (c) Explique la diferencia entre los prefijos de primer orden $\forall x\exists y\forall z\exists w$ y $\exists x\forall y\exists z\forall w$.

Ejercicio 2.19 Muestre que las sentencias $\forall x\exists y\forall z(R(x, y) \wedge R(x, z) \wedge R(y, z))$ y $\exists x\forall y\exists z(R(x, y) \wedge R(x, z) \wedge R(y, z))$ no son equivalentes, exhibiendo un grafo que modele una pero no ambas de estas sentencias.

Ejercicio 2.20 Para cada $n \in \mathbb{N}$, $\exists^{\geq n}$ denota un cuantificador de conteo. Intuitivamente, $\exists^{\geq n}$ significa "existen al menos n tales que." La lógica de primer orden con cuantificadores de conteo es la lógica que se obtiene al agregar estos cuantificadores (para cada $n \in \mathbb{N}$) a los símbolos fijos de la lógica de primer orden. La sintaxis y la semántica de esta lógica se definen de la siguiente manera.

Sintaxis: para cualquier fórmula φ de la lógica de primer orden con cuantificadores de conteo, $\exists^{\geq n}x\varphi$ también es una fórmula.

Semántica: $M \models \exists^{\geq n}\varphi(x)$ si y solo si $M \models \varphi(a_i)$ para cada uno de n elementos distintos $a_1, a_2, \dots, a_n$ del universo de M .

- (a) Usando cuantificadores de conteo, defina una sentencia φ_7 tal que $M \models \varphi_7$ si y solo si $|M| > 7$.
- (b) Usando cuantificadores de conteo, defina una sentencia φ_{23} tal que $M \models \varphi_{23}$ si y solo si $|M| \leq 23$.
- (c) Usando cuantificadores de conteo, defina una sentencia φ_{45} tal que $M \models \varphi_{45}$ si y solo si $|M| = 45$.
- (d) Defina una sentencia de primer orden φ (sin usar cuantificadores de conteo) que sea equivalente a la sentencia $\exists^{\geq n}x(x = x)$.
- (e) Muestre que toda fórmula que usa cuantificadores de conteo es equivalente a una fórmula que no usa cuantificadores de conteo. Concluya que la lógica de primer orden con cuantificadores de conteo tiene el mismo poder expresivo que la lógica de primer orden.

Ejercicio 2.21 Supongamos que se nos presenta un grafo G que tiene aristas múltiples. Esto significa que puede haber más de una arista entre dos vértices de G (así, según nuestra definición estricta de "grafo," un grafo con aristas múltiples no es un grafo). Describa a G como una $\mathcal{V}$ -estructura de primer orden para un vocabulario $\mathcal{V}$ adecuado.

Ejercicio 2.22 Sea K_n la n -clica para algún $n \in \mathbb{N}$. Entonces cualquier grafo con a lo sumo n vértices es un subgrafo de K_n .

- (a) ¿Cuántas subestructuras tiene K_n ?
- (b) ¿Cuántas subestructuras tiene K_n salvo isomorfismo?
- (c) ¿Cuántas subestructuras elementales tiene K_n ?

Ejercicio 2.23 Defina una estructura infinita que tenga exactamente n subestructuras, donde n es un número natural mayor que 1.

Ejercicio 2.24 Sea G el Grafo 1 de la Sección 2.4.1.

- (a) ¿Cuántas sentencias hay en el diagrama de G ?
- (b) Halle una sentencia φ_G tal que $H \models \varphi_G$ si y solo si $H \cong G$.

Ejercicio 2.25 Repita el Ejercicio anterior con el Grafo 4 de la Sección 2.4.1.

Ejercicio 2.26 Demuestre la Proposición 2.11.

Ejercicio 2.27 Demuestre la Proposición 2.16.

Ejercicio 2.28

- (a) Sea $N = (\mathbb{N} \mid S, 1)$. Muestre que ninguna subestructura propia de N es elementalmente equivalente a N .
- (b) Sea $\mathbb{N}_{<}$ la estructura $(\mathbb{N} \mid <)$ de la Sección 2.4.3. Muestre que toda subestructura infinita de $\mathbb{N}_{<}$ es elementalmente equivalente a $\mathbb{N}_{<}$, pero que ninguna subestructura propia es una subestructura elemental de $\mathbb{N}_{<}$.

Ejercicio 2.29 Sean A , B y C $\mathcal{V}$ -estructuras con $A \subset B \subset C$. Para cada uno de los siguientes enunciados, demuéstrela o dé un contraejemplo.

- (a) Si $A \prec B$ y $B \prec C$, entonces $A \prec C$.
- (b) Si $A \prec C$ y $B \prec C$, entonces $A \prec B$.

(c) Si $A \prec B$ y $A \prec C$, entonces $B \prec C$.

Ejercicio 2.30 Sea $\mathcal{V} = \{s, P\}$ el vocabulario que consiste en una función unaria s y una relación unaria P . Sea M la $\mathcal{V}$ -estructura con universo $\mathbb{N}$ que interpreta s como la función sucesor y P como el predicado "par." Es decir, para números naturales a y b , $M \models s(a) = b$ si y solo si $a + 1 = b$, y $M \models P(a)$ si y solo si a es par.

Sea N la $\mathcal{V}$ -estructura con universo $\mathbb{N}$ que interpreta s como la función sucesor y P como el predicado "impar." Es decir, N interpreta s de la misma manera que M , pero $N \models P(a)$ si y solo si a es impar.

(a) Muestre que existen encajes $f_1 : M \rightarrow N$ y $f_2 : N \rightarrow M$.

(b) Muestre que M y N no son isomorfas.

Ejercicio 2.31 Defina estructuras M y N en el mismo vocabulario de modo que existan encajes elementales $f : M \rightarrow N$ y $g : N \rightarrow M$, pero $M \not\cong N$.

Ejercicio 2.32 Usando el hecho de que las fórmulas existenciales se preservan bajo extensiones, demuestre que las fórmulas universales se preservan bajo subestructuras.

Ejercicio 2.33 Sean M y N $\mathcal{V}$ -estructuras. Una función $f : M \rightarrow N$ se dice un *homomorfismo* si preserva las $\mathcal{V}$ -fórmulas atómicas. Suponga que f es sobreyectiva (es decir, cada elemento del universo de N está en el rango de f). Sea φ una $\mathcal{V}$ -fórmula que no contiene los símbolos $\neg$, $\rightarrow$ ni $\leftrightarrow$. Muestre que f preserva φ .

Ejercicio 2.34 Sea M una $\mathcal{V}$ -estructura con conjunto subyacente U . Para cualquier n -tupla $\bar{a} = (a_1, \dots, a_n)$ de elementos de U , sea $\langle \bar{a} \rangle$ la subestructura de M generada por $\bar{a}$. Es decir, el conjunto subyacente de $\langle \bar{a} \rangle$ es el menor subconjunto de U que contiene a cada a_i , contiene además a todas las constantes de $\mathcal{V}$, y es cerrado bajo cada función de $\mathcal{V}$. Sean $\bar{a}$ y $\bar{b}$ dos n -tuplas de elementos de U . Muestre que las siguientes son equivalentes:

(i) Para toda $\mathcal{V}$ -fórmula sin cuantificadores $\varphi(\bar{x})$, $M \models \varphi(\bar{a})$ si y solo si $M \models \varphi(\bar{b})$.

(ii) $\langle \bar{a} \rangle \cong \langle \bar{b} \rangle$.

Ejercicio 2.35 Sea N la estructura $(\mathbb{N} \mid S)$ que interpreta la relación binaria S como la relación sucesor. Muestre que N tiene una cantidad no contable de subestructuras no isomorfas entre sí.

Ejercicio 2.36 Sea A un conjunto. Demuestre que los siguientes enunciados son equivalentes.

(i) A es infinito.

(ii) $|\mathbb{N}| \leq |A|$.

(iii) $|A \cup B| = |A|$ para cualquier conjunto finito B .

- (iv) $|\mathcal{P}_F(A)| = |A|$ donde $\mathcal{P}_F(A)$ es el conjunto de todos los subconjuntos finitos de A .
- (v) Existe una función $f : A \rightarrow A$ que es inyectiva pero no sobreyectiva.
- (vi) Para cualquier B con $|B| < |A|$ y cualquier función $f : A \rightarrow B$, existe $b \in B$ tal que $f(a) = b$ para infinitos $a \in A$.

Ejercicio 2.37 Halle una $\mathcal{V}_<$ -sentencia φ tal que los únicos modelos de φ interpreten $<$ como un orden lineal denso. Muestre que φ solo tiene modelos infinitos.

Ejercicio 2.38 Sea $\mathcal{V}_f$ el vocabulario que consiste en una única función unaria f . Halle una $\mathcal{V}_f$ -sentencia que tenga solo modelos infinitos.

Ejercicio 2.39 Halle un conjunto de sentencias que tenga solo modelos no contables.

Ejercicio 2.40

- (a) Sea F el conjunto de todas las cadenas finitas de letras del alfabeto. Muestre que F es contable.
- (b) Sea I el conjunto de todas las cadenas infinitas de letras del alfabeto. Muestre que I es no contable.

Ejercicio 2.41

- (a) Sea $U = \{1, 2, 3\}$. Enumere los elementos de $\mathcal{P}(U)$.
- (b) Muestre que, para cualquier conjunto finito U , si $|U| = n$ entonces $|\mathcal{P}(U)| = 2^n$.
- (c) Muestre que el conjunto potencia de los números naturales $\mathcal{P}(\mathbb{N})$ y los números reales $\mathbb{R}$ tienen el mismo tamaño.

Ejercicio 2.42 Sea F el conjunto de todas las funciones de $\mathbb{N}$ en $\mathbb{N}$. Muestre que F y $\mathbb{R}$ tienen el mismo tamaño.

Ejercicio 2.43 La caja A contiene infinitas pelotas de ping-pong numeradas $1, 2, 3, \dots$

- (a) Meta la mano en la caja A y saque 100 pelotas y póngalas en su regazo. Luego devuelva una. Repita esto. Saque otras 100 pelotas, póngalas en su regazo, y luego devuelva una. Supongamos que hacemos esto una cantidad numerable de veces. ¿Cuántas pelotas tendrá en su regazo?
- (b) Suponga que, en la parte (a), comenzó sacando las pelotas numeradas 1 a 100 y luego devolvió la pelota 1. Suponga que luego sacó las pelotas 101 a 200 y devolvió la pelota 2. Después llevó a su regazo las pelotas 201 a 300, encontró la pelota 3, y la devolvió. Y así sucesivamente. Después de hacer esto una cantidad numerable de veces, ¿qué pelotas quedan en su regazo?

- (c) Ahora suponga que repetidamente sacamos 99 pelotas de la caja A y nunca devolvemos ninguna de estas pelotas a la caja. Primero llevamos las pelotas 1 a 99 a nuestro regazo y, en lugar de devolver la pelota 1, tomamos un marcador, le agregamos dos ceros, y la convertimos en la pelota 100. Luego sacamos las pelotas 101 a 199 de A , tomamos la pelota 2 de nuestro regazo, la convertimos en la pelota 200, y las conservamos todas en nuestro regazo. Después de repetir esto una cantidad numerable de veces, ¿cuántas pelotas hay en nuestro regazo y qué números tienen?
- (d) ¿Los procesos de (b) y (c) tienen resultados distintos? Si es así, explique por qué es el caso (si no, vuelva a mirar (b) y (c)). Observe que, después de cada etapa, tenemos las mismas pelotas numeradas en nuestro regazo. Suponga que alguien más puso las pelotas de ping-pong en nuestro regazo y no sabemos si se usó un marcador o no. ¿Qué pasa entonces? ¿Por qué debería el uso de un marcador afectar el resultado?